



UNIVERSITÀ DEGLI STUDI DI CATANIA

DIPARTIMENTO DI MATEMATICA E INFORMATICA

CORSO DI LAUREA MAGISTRALE IN INFORMATICA

Petronio Petronio Davide e Rosano Antonio

Principi di Programmazione Parallela

APPUNTI LEZIONI

Prof. Vitello e Prof.ssa Sciacca

Anno Accademico 2025 - 2026

Indice

1	Architetture Moderne	9
1.1	Memory Wall	10
1.1.1	Gerarchia di Memoria	10
1.1.2	Prefetching	11
1.2	Parallelismo Interno al Processore	11
1.2.1	Pipelining	11
1.2.2	Instruction-Level Parallelism (ILP)	12
1.2.3	SIMD: Parallelismo Vettoriale	12
1.2.4	SMT: Simultaneous Multithreading	13
1.3	Sistemi Multicore e Modelli di Accesso alla Memoria	13
1.3.1	Il Modello MIMD	14
1.3.2	SMP vs NUMA	14
1.4	Sistemi a Memoria Condivisa e Distribuita	14
1.4.1	Sistemi Ibridi e HPC	15
1.4.2	Reti e Acceleratori	15
1.5	Modelli di Analisi: Il Roofline Model	16
2	Software Stack in HPC	17
2.1	Filesystem nei sistemi HPC	19
2.2	Compilatori in ambienti HPC	20
2.3	Librerie	21
2.4	Scheduler	22
2.5	Workflow per sistemi HPC	23
3	Ottimizzazione	25
3.1	Misurazione delle performance	25
3.1.1	Misure sul clock	25
3.1.2	Misure sul Tempo	26
3.1.3	Profiling	27
3.2	Ottimizzare	27
3.2.1	Memory Aliasing	30

4	Ottimizzazione Single-Core	32
4.1	Memoria cache	32
4.2	CPU	34
4.2.1	Pipeline di Esecuzione	34
4.3	Evitare le Inefficienze	36
4.3.1	Esempio che seguiremo	36
4.3.2	1. Evitare Chiamate a Funzioni Costose	36
4.3.3	2. Hoisting delle espressioni	37
4.3.4	3. Chiarire lo Scope delle Variabili	38
4.3.5	4. Suggestisci cosa è Importante	39
4.3.6	5. Non ripetere controlli o referenziamenti non necessarie	39
4.3.7	Avvertenza su Numeri in Virgola	41
4.4	Ottimizzazione basata sulla Cache	41
4.4.1	Mappatura della Cache	42
4.4.2	Modello di accesso alla Memoria	43
4.4.3	Organizzazione dei Dati	45
4.5	Branch condizionali	47
4.5.1	Branch Predictor	48
4.5.2	Tecniche per limitare i Branch	49
4.6	Ottimizzazione dei Loop	51
4.6.1	Ottimizzazioni per Cicli a Bassa Intensità: Loop Fusion	52
4.6.2	Ottimizzazioni per Cicli $O(N^2)$: Unroll & Jam	53
4.6.3	Srotolamento del Ciclo (Loop Unrolling) e Aritmetica Floating Point	53
4.6.4	Esempio Pratico $O(N^3)/O(N^2)$: Moltiplicazione Matrice- Matrice (MMM)	54
5	Parallelismo e Scalabilità	56
5.1	Modelli di parallelismo	56
5.2	Scalabilità	58
6	OpenMP	61
6.1	Cosa è OpenMP	62
6.2	Modello Fork-Join	63
6.2.1	Direttive	64
6.3	Regioni Parallele	65
6.3.1	Gestione della Regione Parallela	67
6.3.2	Controllo del Numero di Thread	69
6.3.3	Sincronizzazione dell'Esecuzione	70
6.3.4	Ordinamento dei Thread	74
6.3.5	Assegnazione del lavoro	76

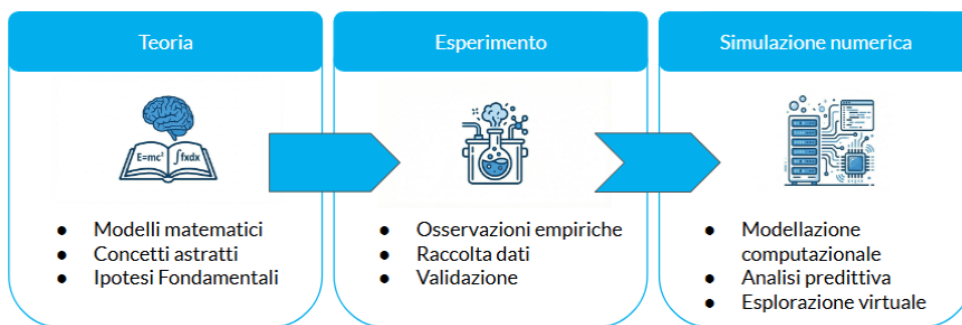
6.4	Basi della Parallelizzazione dei Cicli con OpenMP	78
6.5	Assegnazione del Lavoro	79
6.5.1	Guida alla Scelta dello Scheduler	80
6.6	Clausole Avanzate nei For Paralleli	80
6.6.1	La clausola <code>reduction</code>	81
6.6.2	La clausola <code>collapse</code>	82
6.6.3	La clausola <code>nowait</code>	82
6.7	Anatomia di un Data Race	82
6.7.1	Il Problema del Livello Macchina (Read-Modify-Write)	83
6.7.2	Sincronizzazione vs Scalabilità	83
6.8	Il Problema del False Sharing	84
6.8.1	Risoluzione Tramite Padding	85
6.9	Direttive Annidate: Un Dettaglio Importante	86
6.9.1	Caso A: Condivisione del Lavoro (<code>omp for</code>)	86
6.9.2	Caso B: Creazione di Nuove Regioni (<code>omp parallel for</code>)	86
6.10	Caso di Studio: Calcolo del Pi-greco	87
6.10.1	L'Algoritmo Sequenziale di Base	87
6.10.2	Tentativi di Parallelizzazione ed Errori Comuni	87
6.10.2.1	Errore 1: Il Data Race (Variabile Condivisa)	87
6.10.2.2	Errore 2: L'Oblio (Variabile Privata)	88
6.10.2.3	Errore 3: Il Furto (Variabile Lastprivate)	88
6.10.3	La Soluzione Ottimale: Reduction	88
6.11	NUMA Awareness	89
6.11.1	Affinità dei Thread	89
6.11.2	Placecs	91
6.11.3	Binding	94
6.11.4	Policy di Allocazione della Memoria	97
6.11.5	Strumenti per l'Analisi della Topologia e il Pinning	99
6.11.6	Workflow per la NUMA-awareness	100
7	Tecniche di Compilazione Ottimizzata e Profiling	102
7.1	Compilatore	102
7.1.1	Pipeline di Compilazione	102
7.1.2	Il Memory Aliasing	105
7.2	Profiling	107
7.2.1	PMU	108
7.2.1.1	Performance Events for Linux (Perf)	109
7.2.1.2	Altri strumenti di Profiling	112

8	OpenMPI	115
8.1	Il Problema della Scalabilità	115
8.2	Cos'è MPI?	115
8.3	Struttura di un Programma e Comunicatori	116
8.4	Tipi di Dati e Messaggi	117
8.5	Modalità di Comunicazione Point-to-Point	118
8.5.1	Garanzie di Comunicazione: Ordering, Fairness e Progress	120
8.5.2	Gestione della Ricezione: <code>MPI_Status</code> e <code>MPI_Get_count</code>	120
8.5.3	Comunicazione Bloccante vs Non-Bloccante	121
8.5.4	Deadlock e la Soluzione <code>MPI_Sendrecv</code>	122
8.5.5	Compilazione ed Esecuzione in Ambiente HPC	122
8.6	Misurazione delle Prestazioni: <code>MPI_Wtime</code>	123
8.6.1	Modelli di Costo e Scalabilità (Legge di Amdahl e Modello $\alpha - \beta$)	123
8.6.1.1	L'impatto sulla Scalabilità	123
8.6.2	Il Modello di costo $\alpha - \beta$	123
8.6.3	Il problema delle Deadlock	124
8.6.4	Comunicazioni non bloccanti	125
8.6.5	Linee guida per la scelta: Bloccanti vs. Non-Bloccanti	127
8.7	Comunicazioni Collettive in MPI	128
8.7.1	Operazioni di Distribuzione e Raccolta	128
8.7.2	Operazioni di Riduzione (<code>MPI_Reduce</code> e <code>MPI_Allreduce</code>)	129
8.8	Comunicatori	131
8.8.1	Topologia Cartesiana	133

Introduzione

Con HPC intendiamo ci riferiamo a **High-Performance Computing** che rappresenta il calcolo ad alte prestazioni. Il problema nasce dal fatto che, al giorno d'oggi, il computer tradizionale non basta più a causa di:

- **Limiti del singolo computer:** al giorno d'oggi, i problemi sono troppo grandi per un singolo computer e un singolo computer è troppo lento.
- **Vincoli Sperimentali:** I problemi sono troppo costosi o impossibile da testare sperimentalmente.
- **Terzo pilastro:** La simulazione numerica è, piano piano, diventato un pilastro della scienza grazie al fatto che rende possibile effettuare calcoli e testare funzioni sperimentali a supporto del metodo scientifico.



Tramite l'HPC possiamo andare a superare i problemi del calcolo sequenziale attraverso il **parallelismo** che diventa quindi una necessità strutturale a causa di quanto detto precedentemente. Infatti, quello che accade molto spesso è che i problemi hanno un fattore di crescita **non lineare** rendendo la computazione parecchio lenta nel caso in cui si aggiungano un certo numero di parametri.

L'**High Performance Computing (HPC)** indica l'insieme di **tecniche**

hardware e software utilizzate per risolvere problemi ad alta intensità computazionale, di memoria o di dati. Questa disciplina integra:

- architetture di **calcolo parallele**;
- **modelli di programmazione**;
- **algoritmi e metodi numerici**.

L'obiettivo è, dunque, quello di rendere risolvibili in tempi accettabili problemi che, di norma, non lo sarebbero in un computer normale. Le prestazioni in HPC dipendono dal corretto **bilanciamento tra hardware, software e algoritmi**. È bene notare che il calcolo ad alte prestazioni non coincide con l'uso esclusivo di supercomputer di grandi dimensioni, bensì riguarda il modo di utilizzare le risorse sfruttando **parallelismo e gerarchia di memoria**. Le stesse tecniche HPC sono applicate infatti anche a diversi tipi di sistemi che usiamo anche quotidianamente come CPU multicore, unità vettoriali **SIMD** o acceleratori con GPU.

Il classico modello di **Von Neumann** fa delle assunzioni che sono troppo limitanti:

- esecuzione di **una istruzione alla volta**;
- un **singolo flusso di controllo**;
- accesso a **una memoria uniforme**.

E, per quanto questo modello sia utile per descrivere algoritmi e dimostrare la loro complessità e correttezza, questa è una semplificazione troppo grossa rispetto all'hardware reale. Infatti, al giorno d'oggi il tempo di esecuzione di un programma è dominato dall'attesa dei dati in accesso dalla memoria e non dal calcolo in sé per sé. Inoltre, il modello sequenziale assume un accesso alla memoria uniforme e immediato, che non riflette la realtà.

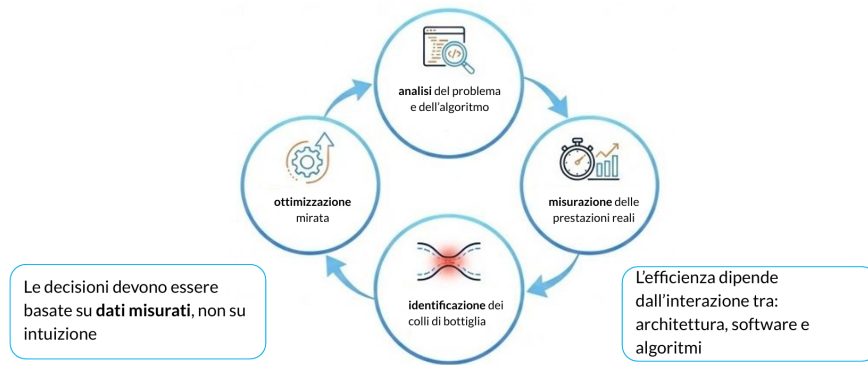
Si può fare riferimento a due leggi:

1. **Legge di Moore**: il numero di transistor raddoppia ogni 18-24 mesi.
2. **Scaling di Dennard**: frequenza può aumentare mantenendo potenza e dissipazione costanti migliorando l'hardware.

Il risultato di queste due leggi è che abbiamo un aumento automatico delle prestazioni grazie a più transistor e, quindi, un' esecuzione più veloce del codice sequenziale. Tuttavia, nel 2006 lo scaling di Dennard si è arrestato e si arriva al punto in cui non si può fare meglio di così sfruttando solo codice sequenziale. È dunque necessario **sfruttare l'hardware in parallelo**

e sfruttare in modo efficiente la **gerarchia di memoria**. Per cui, e prestazioni diventano una responsabilità del programmatore e si rende necessario adottare il **performance engineering**.

Il **performance engineering** è un approccio sistematico alla progettazione di software efficiente



L'hardware moderno è intrinsecamente parallelo e sfrutta sistemi **multi-core**, fa uso di **Hyper-Threading** cioè possono coesistere più thread hardware per core e sfrutta **Acceleratori GPU**. Per fruttare a pieno queste architetture, è necessario che il software sia **esplicitamente parallelo**. Esistono più livelli di parallelismo:

1. **Bit-level parallelism**: operazioni su parole di dimensione crescente;
2. **Instruction-level parallelism (ILP)**: pipeline ed esecuzione fuori ordine;
3. **Data-level parallelism**: operazioni vettoriali (SIMD) cioè che vengono effettuate contemporaneamente su dati diversi;
4. **Thread-level parallelism**: esecuzione concorrente di più thread in parallelo;
5. **Process-level parallelism**: esecuzione distribuita su più processi o nodi;

Ogni livello contribuisce alle prestazioni complessive e ignorarne anche solo uno porta a un uso non efficiente dell'hardware.

Un sistema HPC è tipicamente organizzato come un cluster di nodi di calcolo i cui nodi sono collegati tramite una rete ad alte prestazioni a **bassa latenza e alta banda**. I dati sono spesso gestiti tramite **storage condivisi** accessibili a tutti. Un cluster è un **insieme di nodi di calcolo indipendenti**, collegati tramite una rete ad alte prestazioni. Ogni nodo dispone di

una propria CPU, memoria e acceleratori e un sistema operativo completo. Il sistema è gestito tramite uno **stack software stratificato** che garantisce correttezza, prestazioni e scalabilità. Abbiamo diversi livelli dello stack:

- **Applicazioni:** che rappresentano i codici scientifici da eseguire.
- **Librerie:** che sono librerie che forniscono supporto al calcolo numerico e al parallelismo.
- **Runtime:** ciò che gestisce l'esecuzione parallela.
- **Scheduler:** che alloca le risorse e gestisce i job. Nel nostro caso è bene segnalare che sfrutteremo lo scheduler SLURM che rappresenta l'attuale standard del settore.
- **Compilatori:** che traducono il codice in codice macchina ottimizzato per una specifica architettura.

Tra questi, un ruolo fondamentale vedremo che lo avrà lo **scheduler** che si occuperà di gestire le risorse del cluster. Così facendo, gli utenti non eseguono i programmi direttamente, bensì sottomettono i job allo scheduler che li gestirà e ordinerà a seconda di quelle che sono le sue policy e delle risorse che può allocare. Questo viene fatto per far sì che sia lo scheduler a garantire l'uso efficiente delle risorse e l'equità tra gli utenti. Come abbiamo già detto, dunque, non basterà scrivere codice corretto, abbiamo anche bisogno di **pensare in termini di parallelismo** e gestire esplicitamente dati, comunicazione e sincronizzazione.

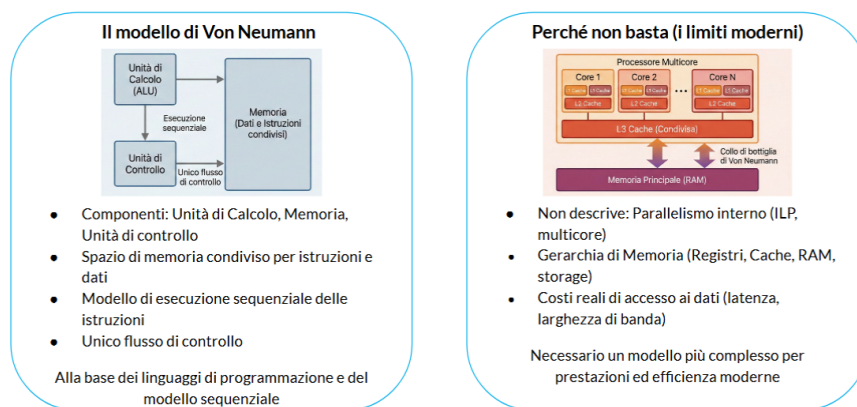
Capitolo 1

Architetture Moderne

Le prestazioni di un programma parallelo dipendono dall'architettura hardware e lo stesso codice può avere prestazioni molto diverse su architetture diverse. Ciò dipende dal fatto che l'architettura determina:

- Costi di accesso alla memoria.
- Disponibilità di parallelismo.
- Costo della computazionale e sincronizzazione.

Ignorare questi dettagli potrebbe comportare scelte inefficienti. Come già detto, il modello Von Neumann è piuttosto limitante rispetto a quella che è la realtà:



Infatti, la struttura di un core moderno è molto diversa rispetto a quella ideata da Von Neumann poiché troviamo:

- Unità di **Fetch/Decode**.

- Pipeline di esecuzione.
- Unità di esecuzione ALU, FPU e unità vettoriali.
- Registri e Cache di primo livello

1.1 Memory Wall

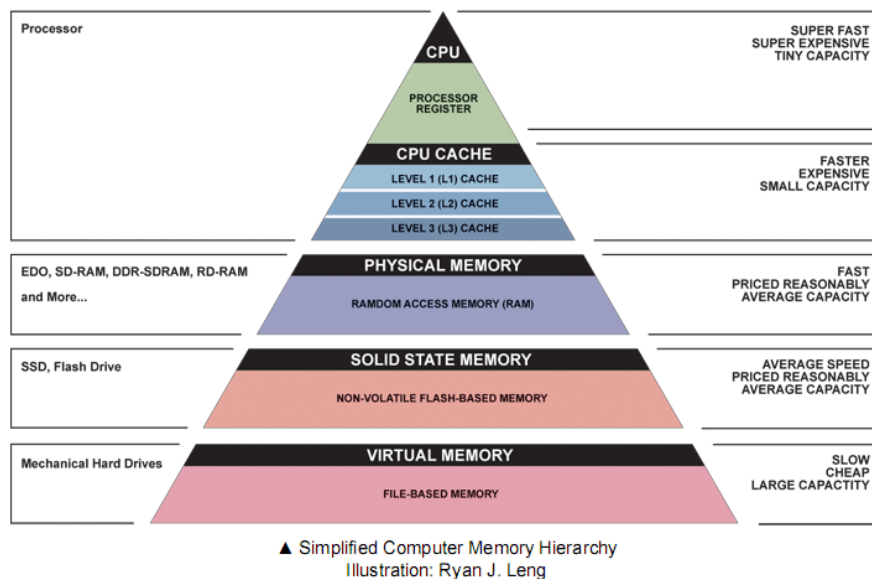
Il problema principale che dovremo risolvere è quello del **Memory Wall** per cui il calcolo della CPU sia diventato molto più veloce rispetto alla velocità di accesso alla memoria principale risultando così in sprechi di tempo dovuti al fatto che è necessario attendere l'accesso alla memoria. Per cercare di risolvere questo problema, utilizziamo due principi fondamentali:

- **Località temporale:** dati usati di recente vengono spesso riutilizzati.
- **Località spaziale:** dati vicini in memoria vengono utilizzati insieme.

Le architetture moderne sfruttano già i principi di località attraverso **cache**, **prefetching** e **gerarchia di memoria**.

1.1.1 Gerarchia di Memoria

I sistemi moderni usano una gerarchia di memoria a più livelli che segue, tipicamente, il seguente schema:



Ogni livello si differenzia dal precedente in termini di latenza, banda e capacità. Generalmente, minore è la capacità, minore sarà la latenza d'accesso.

Le memorie cache sono memorie molto veloci che memorizzano copie di dati presenti in memoria principale (seguendo il principio di località temporale) e i dati vengono trasferiti tra memoria e cache in **blocchi** (seguendo il principio di località spaziale). Abbiamo un **cache hit** quando il dato è presente in cache, mentre parliamo di **cache miss** quando il dato non è presente. A causa del costo elevato di accesso in memoria centrale, i cache miss hanno un costo estremamente elevato.

1.1.2 Prefetching

Il **prefetching** è un meccanismo che anticipa il caricamento dei dati e può essere effettuato tramite **hardware** o **software**. Questo può essere parecchio efficace quando abbiamo accessi ai dati regolari con pattern prevedibili ma svantaggio in caso di accessi indiretti o irregolari. Possiamo vedere, ad esempio, i seguenti due snippet di codice:

```
for (i = 0; i < N; i++)  
    sum += A[i];
```

```
for (i = 0; i < N; i++)  
    sum += A[index[i]];
```

Quello che vediamo nel secondo caso è che prima di accedere all'indice di A, abbiamo bisogno di accedere a un ulteriore indirizzo di memoria prima. Ciò rallenta chiaramente le prestazioni rispetto al primo caso.

1.2 Parallelismo Interno al Processore

Per aumentare il throughput senza aumentare la frequenza di clock, si sfruttano diverse tecniche di parallelismo.

1.2.1 Pipelining

La **pipeline** suddivide l'esecuzione di un'istruzione in più stadi:

- **Fetch**: prelevamento dell'istruzione.
- **Decode**: decodifica dell'istruzione.

- **Execute:** accesso ai registri ed esecuzione.
- **Memory Access:** accesso in memoria.
- **Write-back:** scrittura risultato sui registri.

Più istruzioni possono essere in esecuzione contemporaneamente in stadi diversi ciò con l'obiettivo di aumentare il throughput. Ci teniamo a sottolineare che il sistema di pipeline non migliora la latenza del sistema poiché questa è una caratteristica intrinseca della gestione della memoria del sistema. Vedremo più avanti come questo sistema di pipeline in realtà complichino il nostro sistema a causa di possibili stalli dovuti alle dipendenze tra istruzioni o anche a causa di errate guess nel possibile if che verrà eseguito.

1.2.2 Instruction-Level Parallelism (ILP)

L'**Instruction-Level Parallelism (ILP)** è il parallelismo tra istruzioni indipendenti. Un processore moderno sfrutta l'ILP potendo eseguire più istruzioni nello stesso ciclo di clock e riordinandole dinamicamente (out-of-order execution). Questo meccanismo è sfruttato tramite:

- **Pipeline multiple**
- **Unità di esecuzione multiple**

Il grado di ILP, sebbene sia una tecnica trasparente al programmatore, presenta limiti intrinseci dettati da: dipendenze tra le istruzioni, accessi alla memoria e struttura stessa del codice.

1.2.3 SIMD: Parallelismo Vettoriale

Il **SIMD (Single Instruction, Multiple Data)** rappresenta un set di istruzioni che permette l'applicazione della stessa operazione a più dati contemporaneamente. Le CPU moderne implementano questo paradigma includendo unità vettoriali dotate di registri di grande ampiezza. La vettorizzazione del codice può essere:

- **Automatica:** a carico del compilatore.
- **Esplicita:** a carico del programmatore. Sebbene molto efficace, la velocità ottenibile dal SIMD dipende strettamente dalla regolarità degli accessi in memoria e dall'assenza di dipendenze tra le iterazioni.

1.2.4 SMT: Simultaneous Multithreading

Il **SMT (Simultaneous Multithreading)** permette a un singolo core di eseguire più thread hardware contemporaneamente. In questo scenario, i thread condividono pipeline, unità di esecuzione e cache. Lo SMT mira a:

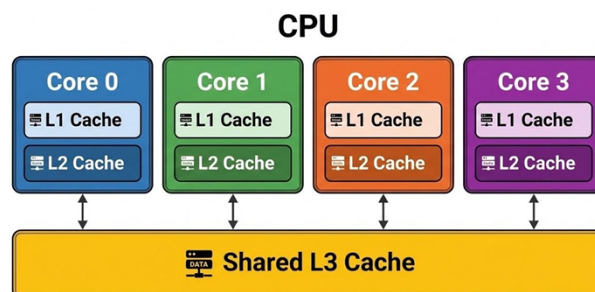
- Migliorare l'utilizzo delle risorse del core.
- Nascondere le latenze (come gli accessi in memoria).

È fondamentale notare che lo SMT non raddoppia le prestazioni, poiché i thread competono per le medesime risorse. La sua reale efficacia dipende dal tipo di carico di lavoro e dal comportamento della memoria. Sarebbe bene che thread hardware che condividono lo stesso core, utilizzino gli stessi dati così da evitare di poter sfruttare al meglio la cache L1 che condividono. Se non facessero a questo modo, si rischierebbe che i buttano fuori i dati dalla memoria a vicenda.

1.3 Sistemi Multicore e Modelli di Accesso alla Memoria

I **processori multicore** integrano più core di calcolo sullo stesso chip. Questa architettura rappresenta la principale risposta hardware alla fine dello scaling di Dennard e all'impossibilità fisica di continuare ad aumentare la frequenza di clock. In un sistema multicore:

- **Risorse private:** ogni core dispone di proprie pipeline, unità di esecuzione e cache (tipicamente L1 e L2).
- **Risorse condivise:** la cache di ultimo livello (L3) e la memoria principale sono condivise tra i core.



Le prestazioni finali di un processore multicore dipendono pesantemente dalla capacità del software di sfruttare più core in parallelo e di gestire in modo efficiente la condivisione delle risorse.

1.3.1 Il Modello MIMD

L'architettura **MIMD (Multiple Instruction, Multiple Data)** prevede che ogni unità di elaborazione esegua un flusso di istruzioni indipendente operando su dati differenti. Questo modello a parallelismo coarse-grained offre elevata flessibilità, è adatto a problemi eterogenei e rappresenta la base dei sistemi moderni.

1.3.2 SMP vs NUMA

Nei sistemi **SMP (Symmetric Multiprocessing)**, più processori condividono la stessa memoria fisica e lo stesso spazio di indirizzamento. Si basano su un modello **UMA (Uniform Memory Access)**, dove l'accesso alle risorse è paritario per tutti i processori ed equivalenti per il sistema operativo.

- **Vantaggi:** modello concettualmente semplice, comunicazione veloce intra-nodo e forte supporto da parte di OS e runtime.
- **Svantaggi:** scalabilità limitata (difficile andare oltre poche decine di core) a causa della contesa sul bus di memoria e del traffico hardware dovuto a protocolli necessari a mantenere la coerenza della cache.

Per superare i limiti di scalabilità dell'SMP, si utilizza l'**architettura NUMA (Non-Uniform Memory Access)**, in cui la memoria è fisicamente distribuita. Ogni gruppo di core possiede una memoria locale (con accesso più veloce), ma può accedere alla memoria remota (con accesso più lento).

I sistemi moderni sono spesso ccNUMA (cache-coherent NUMA): l'hardware mantiene la coerenza delle cache tra i vari nodi NUMA, offrendo al programmatore l'illusione di una memoria condivisa. Le prestazioni reali dipendono fortemente dalla località NUMA dei dati e dei thread.

1.4 Sistemi a Memoria Condivisa e Distribuita

In base al paradigma di accesso alla memoria, possiamo dividere i sistemi in due macro-categorie:

- **Sistemi a memoria condivisa:** Tutti i thread vedono lo stesso spazio di indirizzamento e comunicano implicitamente tra loro leggendo e scrivendo gli stessi dati. Il parallelismo è espresso tramite thread. La programmazione è generalmente più semplice, ma presenta svantaggi legati alla competizione per le risorse, la necessità di sincronizzazione e i costi di memoria non uniformi introdotti dalle architetture NUMA.
- **Sistemi a memoria distribuita:** Ogni processo dispone di una memoria privata e non esiste alcuno spazio di indirizzamento globale. La comunicazione tra processi avviene obbligatoriamente tramite lo scambio esplicito di messaggi. Questo approccio garantisce un'elevata scalabilità e rimuove la contesa diretta sulla memoria, al prezzo di una programmazione più complessa e di un costo esplicito (latenza) per ogni comunicazione.

1.4.1 Sistemi Ibridi e HPC

I sistemi ibridi costituiscono l'architettura standard per i moderni sistemi HPC:

- Sono composti da nodi multicore che operano internamente a memoria condivisa.
- Questi nodi sono poi collegati tra loro formando un cluster a memoria distribuita.

Di conseguenza, la gestione del parallelismo avviene su due livelli distinti e complementari:

- All'interno del singolo nodo (Intra-nodo): il parallelismo viene espresso tramite thread che accedono alla memoria condivisa locale (spessore regolato da architettura NUMA).
- Tra nodi diversi (Inter-nodo): il parallelismo è gestito tramite processi separati, e lo scambio di dati richiede una comunicazione esplicita (come l'invio di messaggi via rete).

1.4.2 Reti e Acceleratori

Nei sistemi HPC i nodi sono collegati tramite **reti ad alte prestazioni** e questa influenza la **latenza** della comunicazione, la **banda** disponibile e la **scalabilità** delle applicazioni. È anche possibile fare uso di **acceleratori** come dispositivi specializzati per il calcolo parallelo come la **GPU**.

1.5 Modelli di Analisi: Il Roofline Model

Il **Roofline model** è un modello prestazionale semplificato. Questo descrive il limite massimo di prestazioni di un'applicazione che è solitamente limitato a causa di capacità di calcolo o dalla banda di memoria. Questo modello mette in relazione **intensità computazionale** e **prestazioni ottenibili**. Esistono due tipi di sistemi:

- **memory bound.**
- **compute bound.**

E questo modello ci aiuta a capire dove intervenire per ottimizzare al variare di memoria messa a disposizione o potenza computazionale messa a disposizione.

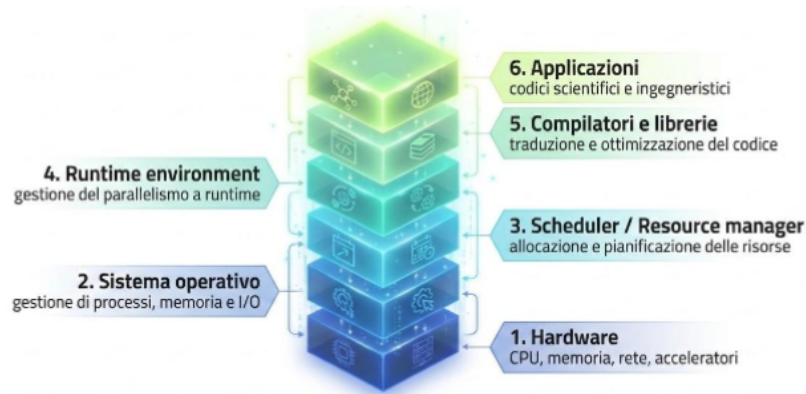
Capitolo 2

Software Stack in HPC

L'utilizzo di un cluster HPC differisce radicalmente da quello di un normale PC o di un server tradizionale. La necessità di introdurre uno stack software specifico nasce da tre fattori chiave:

- **La natura delle risorse:** I sistemi HPC mettono a disposizione risorse (CPU, memoria, rete e storage) che sono per definizione condivise, estremamente costose e spesso destinate a task critici.
- **La competizione:** A differenza di un PC personale, in un cluster ci sono costantemente molti utenti che competono per utilizzare queste stesse risorse nello stesso momento.
- **Il rischio del caos:** Senza un livello software in grado di orchestrare tutto questo, i risultati sarebbero disastrosi, portando a:
 - Spreco di risorse (es. nodi lasciati inattivi o sovraccaricati).
 - Conflitti tra utenti (programmi che si "rubano" memoria a vicenda o si bloccano).
 - Impossibilità di garantire le prestazioni previste e l'equità (fairness) nell'accesso al sistema da parte di tutti.

La regola d'oro è che **ogni livello interagisce con quelli adiacenti e influenza il comportamento prestazionale dell'intero sistema.**



Partendo dalle fondamenta fisiche e salendo fino all'utente, troviamo 6 livelli:

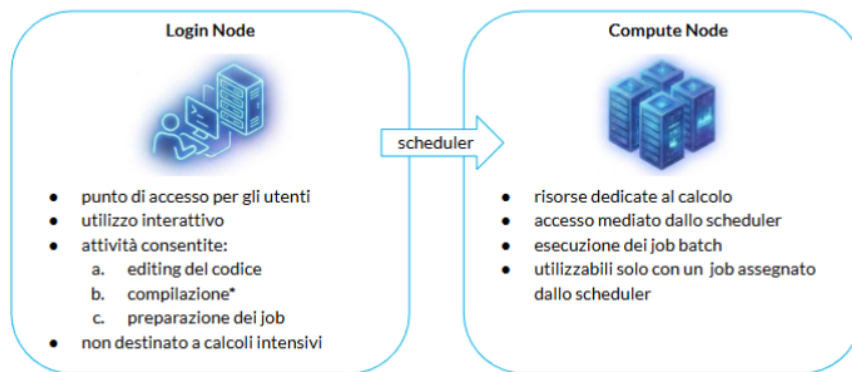
- **1. Hardware:** costituisce la base fisica (CPU, memoria, rete, acceleratori).
- **2. Sistema operativo:** si occupa della gestione di base dei processi, della memoria e dell'I/O.
- **3. Scheduler / Resource manager:** è il livello responsabile dell'allocazione e lo **scheduling** delle risorse del cluster.
- **4. Runtime environment:** gestisce il parallelismo a runtime durante l'esecuzione vera e propria.
- **5. Compilatori e librerie:** sono fondamentali per la traduzione e, soprattutto, l'ottimizzazione del codice affinché sfrutti l'hardware.
- **6. Applicazioni:** rappresentano i programmi finali, come i codici scientifici e ingegneristici degli utenti.

Dal punto di vista dell'utente, un sistema HPC non deve essere visto come una singola macchina, bensì come un vero e proprio **ambiente di calcolo controllato**.

Esso rappresenta, infatti, l'integrazione strutturata di hardware, software e specifiche **policy di utilizzo**. L'intera infrastruttura è stata pensata e progettata con il duplice scopo di eseguire **job di lunga durata** e di **massimizzare l'utilizzo complessivo** delle risorse a disposizione. Trattandosi di un sistema intrinsecamente **condiviso**, è importante tenere a mente che le risorse hardware non sono sempre immediatamente disponibili. Di conseguenza, l'accesso alla potenza di calcolo non avviene in modo diretto, ma è

sempre **mediato**: l'utente interagisce inizialmente solo con un nodo di accesso detto **nodo di Login** che si occuperà di gestire le richieste al cluster di calcolo vero e proprio.

Il **Login Node** funge da punto di accesso interattivo per gli utenti le attività consentite su questa macchina si limitano alla preparazione del lavoro, come l'editing del codice, la sua eventuale compilazione e l'impostazione dei job. È fondamentale ricordare che questo nodo **non è destinato all'esecuzione di calcoli**. Al contrario, i **Compute Node** sono le vere e proprie risorse dedicate al calcolo puro. L'accesso a questi nodi non è mai diretto, ma avviene sempre in modo mediato: essi si occupano dell'esecuzione dei job batch e sono utilizzabili esclusivamente in seguito all'assegnazione di un job da parte dello **scheduler**.



Questa rigida separazione non è casuale, ma rappresenta una scelta architetturale obbligata per gestire e proteggere un hardware estremamente costoso. Nello specifico, isolare le attività di preparazione (spesso interattive e imprevedibili) da quelle di calcolo serve a garantire la **stabilità** dell'intero sistema e a proteggere le risorse computazionali. Inoltre, questa barriera impedisce che le operazioni o gli errori di un singolo utente possano causare interferenze con i lavori in esecuzione degli altri, assicurando un utilizzo **equo ed efficiente** delle macchine e permettendo allo scheduler di effettuare una pianificazione controllata e ottimale dei job.

2.1 Filesystem nei sistemi HPC

La gestione dei dati all'interno dei sistemi HPC è affidata a un **filesystem condiviso** tra i login node e i compute node, garantendo così che i dati siano accessibili contemporaneamente da molti nodi. Questa infrastruttura è appositamente progettata per offrire un **elevato throughput** e per supportare

accessi paralleli, motivo per cui viene spesso implementata come un vero e proprio **filesystem parallelo**. A livello pratico, gli utenti interagiscono principalmente con due directory tipiche: la `$HOME`, dedicata alla conservazione dei dati personali e dei file di configurazione, e la `$SCRATCH`, pensata appositamente per ospitare i dati temporanei e per memorizzare l'output generato dai job in esecuzione.

Le modalità con cui le applicazioni effettuano le operazioni di Input/Output (I/O) hanno un impatto diretto e profondo sulle prestazioni e sulla scalabilità dell'intero sistema. Ad esempio, l'elaborazione di **molti file di piccole dimensioni** comporta un elevato overhead, il quale si traduce in una forte degradazione delle prestazioni globali. Analogamente, gli **accessi casuali** in memoria penalizzano sia il throughput che la latenza, non riuscendo a sfruttare l'ottimizzazione offerta dalla struttura del filesystem parallelo. Per poter gestire grandi volumi di dati in maniera efficiente, è strettamente necessario ricorrere all'**I/O parallelo**, che garantisce un accesso coordinato ai file da parte di molteplici processi contemporaneamente. In conclusione, la gestione dell'I/O rappresenta molto spesso il **vero limite alla scalabilità** di un programma, persino in quegli scenari in cui la componente di puro calcolo riesce a scalare in modo perfettamente corretto.

2.2 Compilatori in ambienti HPC

In un ambiente HPC sono tipicamente messi a disposizione degli utenti molteplici **compilatori**, tra cui troviamo le opzioni più note come GCC o Clang/LLVM, affiancate spesso da compilatori **vendor-specific** (come quelli forniti da Intel, AOCC o NVIDIA HPC). Il ruolo del compilatore va ben oltre la semplice traduzione del codice sorgente poiché si occupa di applicare **ottimizzazioni automatiche** e di generare un codice che sia specificamente ottimizzato per l'architettura su cui andrà in esecuzione. Per questo motivo, il compilatore non è un semplice strumento di traduzione, ma deve essere considerato a tutti gli effetti come **parte integrante delle prestazioni** finali di un'applicazione.

La ragione per cui all'interno di questi sistemi coesistono più compilatori è dovuta in primis alla grande diversità delle **architetture hardware** sottostanti. Ogni compilatore implementa, infatti, differenti **strategie di ottimizzazione** e gestisce in modo diverso i modelli di vettorizzazione e di parallelismo. Inoltre, è frequente che determinate librerie risultino maggiormente ottimizzate solo se trattate con compilatori specifici. Nella pratica, chi sviluppa si trova spesso a dover gestire una naturale **tensione** tra la massima portabilità del codice e la ricerca delle massime prestazioni. Proprio per

questo motivo, è fondamentale ricordare che **non esiste un compilatore "migliore" in assoluto**, ma bisogna saper scegliere quello più adatto al proprio caso d'uso e al proprio hardware.

È bene segnalare che si potrebbe provare a compilare il codice con più compilatori per vedere quale ci ritorna delle prestazioni migliori.

2.3 Librerie

Nei sistemi HPC, le **librerie** rappresentano dei componenti centrali dell'intero stack software, in quanto forniscono implementazioni già ampiamente ottimizzate per le operazioni fondamentali. Il loro grande vantaggio è quello di incapsulare al loro interno sia una profonda conoscenza dell'architettura hardware sia le ottimizzazioni più avanzate, tanto che spesso determinano le prestazioni finali molto più di quanto non faccia il codice applicativo stesso. Tra gli esempi più importanti a disposizione degli sviluppatori troviamo:

- **MPI**: fondamentale per gestire la comunicazione tra processi in sistemi a memoria distribuita.
- **OpenMP**: standard di fatto per gestire il parallelismo in architetture a memoria condivisa.
- **BLAS / LAPACK**: librerie di base, essenziali per le operazioni di algebra lineare.

Questa enorme ricchezza di strumenti, tuttavia, genera il cosiddetto **problema dell'ambiente software**. Rispetto a un computer classico, l'ambiente di un cluster HPC è intrinsecamente molto più complesso. Su queste macchine devono **necessariamente coesistere molteplici versioni di compilatori e innumerevoli versioni delle stesse librerie**, poiché le varie applicazioni degli utenti presentano dipendenze specifiche e requisiti spesso incompatibili tra loro. Un concetto fondamentale da tenere a mente è che il compilatore altera il risultato finale ad esempio una libreria MPI compilata con GCC è incompatibile e profondamente diversa da una compilata con strumenti Intel. Diventa quindi imperativo che l'ambiente sia sempre controllato, riproducibile e configurabile dinamicamente per ogni singolo utente e per ogni specifico job.

La soluzione standard adottata universalmente nei sistemi HPC per domare questa complessità prende il nome di **Environment Modules**. Si tratta di un meccanismo che permette di "caricare" dinamicamente solo le versioni specifiche del software necessario per il proprio lavoro, configurando

l'ambiente su misura. All'atto pratico, i moduli agiscono andando a modificare in modo sicuro le variabili d'ambiente del sistema operativo (come ad esempio i percorsi `PATH` o `LD_LIBRARY_PATH`). Questo approccio elegante garantisce la coesistenza pacifica di più compilatori e versioni dello stesso software, favorendo al contempo un elevato grado di controllo, grande flessibilità e, soprattutto, la totale riproducibilità degli esperimenti

2.4 Scheduler

Il **runtime environment** rappresenta il livello software responsabile della gestione e dell'esecuzione di un programma parallelo a tempo di run-time. Pur non facendo parte del kernel del sistema operativo (opera infatti in user-space al di sopra di esso), svolge compiti fondamentali come la **creazione e gestione dei thread** (es. tramite OpenMP), **l'inizializzazione e il coordinamento dei processi** (es. tramite MPI), e la **gestione della sincronizzazione e della comunicazione tra processi**. Inoltre, si occupa di applicare politiche cruciali per le prestazioni come il **CPU binding e la memory affinity**. Il comportamento di questo ambiente è pesantemente influenzato dalle variabili d'ambiente (come `OMP_NUM_THREADS`), dai parametri del job passati allo scheduler e dall'architettura fisica del nodo stesso.

Perché tutto questo ecosistema funzioni senza collassare, la presenza di uno **scheduler** è assolutamente indispensabile. In un ambiente in cui le risorse sono limitate, costose e condivise tra moltissimi utenti che effettuano richieste concorrenti e spesso incompatibili, operare senza questo componente porterebbe inevitabilmente al caos operativo, allo spreco di risorse e all'instabilità del sistema. Lo scheduler diviene quindi necessario per **pianificare l'uso delle risorse, evitare conflitti, garantire equità e massimizzare l'efficienza** complessiva del cluster.

Nello specifico, lo scheduler riceve le richieste di risorse (sotto forma di **job**) dagli utenti e agisce da vero e proprio arbitro:

- Decide autonomamente **quando** un job può effettivamente partire e **dove** (ovvero su quali nodi fisici) deve essere eseguito.
- **Assegna e isola** fisicamente le risorse necessarie per quel job, bloccando ad esempio un certo numero di CPU/core, un quantitativo specifico di memoria e le eventuali GPU richieste.
- Applica rigorosamente le **policy di sistema**, gestendo le priorità in coda, imponendo i limiti di tempo massimi consentiti per l'esecuzione e garantendo il *fair-share* (l'equità di utilizzo tra i vari utenti).

Attualmente, **Slurm** rappresenta lo scheduler standard di fatto che vedremo anche in questo corso.

2.5 Workflow per sistemi HPC

Nei sistemi HPC l'unità fondamentale di esecuzione non è il singolo programma o processo tradizionale, ma prende il nome di **job**. Un job può essere visto come un contenitore descrittivo che specifica allo scheduler esattamente **quali risorse** hardware sono necessarie, per **quanto tempo** lo saranno e **quale programma** dovrà essere effettivamente eseguito. Si tratta a tutti gli effetti di un'entità **schedulata**: una volta preparato, il job viene sottomesso allo scheduler, viene posizionato in una coda di attesa e verrà eseguito in totale autonomia solo nel momento in cui le risorse richieste si libereranno. La regola d'oro da ricordare è che in un ambiente HPC **non si esegue mai direttamente un programma, ma si richiedono le risorse per poterlo fare**.

All'atto pratico, l'interazione quotidiana dell'utente con il cluster segue un ciclo ben definito, noto come **workflow tipico HPC**, che viene ripetuto innumerevoli volte durante le fasi di sviluppo:

1. **Accesso al sistema**: si effettua il login interattivo esclusivamente sul login node.
2. **Preparazione dell'ambiente**: si caricano i moduli software (compilatori, librerie) necessari al proprio lavoro.
3. **Sviluppo e compilazione**: si scrive il codice sorgente e lo si compila utilizzando gli strumenti scelti al passo precedente.
4. **Preparazione del job**: si definiscono le risorse hardware richieste e si crea materialmente lo "script di job".
5. **Sottomissione del job**: si invia lo script appena creato allo scheduler affinché venga preso in carico.
6. **Esecuzione**: lo scheduler fa girare il job sui compute node non appena le macchine sono pronte.
7. **Analisi dei risultati**: una volta terminato il calcolo, si analizzano i file di output e di log generati per valutare i risultati o per fare eventuale debugging.

Alcuni errori da evitare assolutamente sono:

- Eseguire calcoli sul login node.
- Bypassare lo scheduler tramite SSH.
- Richiedere troppe o troppe poche risorse.
- Non considerare che l'ambiente non è quello standard di un normale PC.

Capitolo 3

Ottimizzazione

L'High Performance Computing richiede, come suggerisce il nome stesso, di ottenere la massima efficacia dal codice sulla macchina su cui viene eseguito. L'ottimizzazione in HPC non è un'operazione singola, ma un bilanciamento di diversi vincoli: tempo di esecuzione, uso della memoria, efficienza energetica, robustezza e costi di I/O.

3.1 Misurazione delle performance

Quando parliamo di performance cerchiamo di andare a fare una stima di quanto il nostro codice riesca a sfruttare delle specifiche risorse della CPU. In particolare, ci concentriamo su:

- Quanto è **veloce** il nostro codice.
- Quanto bene sfrutta il parallelismo a livello di istruzione.

3.1.1 Misure sul clock

Per quanto riguarda la velocità, sappiamo che la CPU non lavora in maniera continua ma lavora seguendo **cicli di clock** il cui ritmo è di parecchi clock al secondo. La frequenza può anche variare a seconda del carico di lavoro aumentando o diminuendo per risparmiare energia. Nello specifico, ci riferiremo a questo fenomeno come **throttling**.

Proprio perché il ciclo di clock varia, misurare semplicemente il tempo di esecuzione potrebbe portare a stime imprecise. Può essere più comodo andare a vedere **quanti cicli di clock sono stati spesi** su una certa sezione di codice. In particolare, ci concentriamo su sezioni di codice che ripetono

blocchi d'istruzioni su un array che rappresentano i punti più pesanti computazionalmente del nostro software. Questa metrica prende il nome di **Cyles Per Element** o **CPE**. Se vogliamo, il CPE rappresenta il numero di cicli di clock necessari all'analisi di una singola porzione di dato come, ad esempio, un singolo elemento dell'array.

Al contrario, nel caso volessimo stimare quanto bene stiamo sfruttando la capacità super-scalare della CPU, ha più senso usare la matrice di **Istruzioni per Ciclo (IPC)** per valutare quante istruzioni per ciclo vengono eseguite. Questo ci darà l'informazione riguardo a quante operazioni stiamo riuscendo a fare contemporaneamente all'interno di un ciclo di clock.

3.1.2 Misure sul Tempo

Nonostante le misure sul clock siano sicuramente molto utili e precise, spesso si preferisce andare a ragionare in termini di **Tempo di esecuzione**. Fondamentalmente, guarderemo tre diversi tipi di tempo:

- **Wall-clock time**: Fondamentalmente questo è il tempo d'orologio dall'inizio alla fine del processo.
- **System-Time**: quantità di tempo che l'intero sistema trascorre eseguendo il codice. Questo comprende il tempo in **kernel mode**.
- **Process user-Time**: Metrica che tiene il tempo speso dalla CPU per eseguire le istruzioni del codice eliminando, in pratica, i tempi morti d'attesa. Questo è il tempo in **user mode**
- **CPU time**: Somma di system time e process user time.

Indipendentemente dalle metriche che stiamo accumulando, affrontare solo una misura non è una buona stima: bisogna fare la media su molte esecuzioni e sottrarre l'overhead della funzione di tempo stessa.

Per misurare questi tempi in C possiamo utilizzare principalmente funzioni come *gettimeofday()* che restituisce il wall clock time, *clock()* che restituisce la somma tra lo user time e il system time e, infine, *clock_gettime()* che ritorna vari tipi di tempi tra cui real time, monotonic ecc.

```
#define CPU_TIME (clock_gettime( CLOCK_PROCESS_CPUTIME_ID, &ts ),\
                  (double)ts.tv_sec + \
                  (double)ts.tv_nsec * 1e-9)

....

Tstart = CPU_TIME ;
// your code segment here

Time = CPU_TIME - Tstart;
```

Come si vede in figura, è possibile misurare il tempo effettuato in una sezione di codice incapsulandolo tra i due CPU time.

Un'altra cosa da tenere in considerazione è che il nostro processore, al giorno d'oggi, è **multi-core** e, spesso, i processi possono spostarsi tra un core e l'altro. Di fatto ciò comporta dei rallentamenti a causa del fatto che è necessario nuovamente riprendere i dati in memoria e metterli nella cache allocata per ogni singolo core (vedremo dopo in dettaglio). Per evitare questo, si usano delle funzioni particolare che effettuano un binding di un processo ad uno o più core.

3.1.3 Profiling

Il **profiling** è una fase cruciale del *Performance Engineering*. Consiste nell'analizzare l'esecuzione di un programma utilizzando carichi di lavoro (*workload*) reali per identificare i **punti critici** (*hotspots*) e i **colli di bottiglia** del codice.

Invece di procedere per tentativi, il profiling permette di guidare il processo di ottimizzazione basandosi su dati oggettivi. Un'analisi accurata permette di individuare:

- **Intensità di calcolo e dati:** identifica quali specifiche linee di codice eseguono il maggior numero di operazioni o richiedono lo spostamento di grandi volumi di dati.
- **Stalli delle risorse:** rileva i punti in cui la CPU è ferma in attesa (ad esempio a causa di *pipeline stalls* o latenze della memoria).
- **Efficienza dell'I/O:** traccia il tempo speso nelle operazioni di input/output, evidenziando schemi di accesso al disco inefficienti.
- **Bilanciamento del carico (Workload balance):** fondamentale in ambito parallelo, serve a capire se il lavoro è distribuito equamente tra i core o se ci sono processi che rallentano l'intera esecuzione.

Questa pratica la vedremo dopo nel dettaglio.

3.2 Ottimizzare

Come abbiamo detto, ottimizzare vuol dire **estrarre la massima efficacia del codice sulla macchina su cui viene eseguito**. È importante notare come sottolineiamo il fatto che questo processo sia riferito a una specifica

macchina questo perché, a causa di dimensioni di memoria e simili, alcune scelte prese potrebbero essere ottimali su una macchina ma non sull'altra.

Inoltre, è necessario definire su cosa vogliamo ottimizzare. Questo perché potrebbe non essere possibile ottimizzare un codice a tutto tondo, ma potrebbe essere necessario dover trovare compromessi. Tra le varie condizioni di cui tener conto quando vogliamo ottimizzare, troviamo:

- Vincoli di tempo;
- Vincoli di memoria;
- Vincoli di I/O;
- Desiderio di robustezza del sistema;
- Limiti energetici.

Per quanto adesso il nostro focus sia incentrato sull'ottimizzazione del codice, è bene ricordare che prima di tutto dobbiamo:

- Verificare la **correttezza** del codice. Cioè che faccia già correttamente quello che deve fare.
- Usare il giusto **modello di dati** in termini di struttura dati ecc.
- Scegliere il giusto **algoritmo**.
- Avere un **buon codice** che sia chiaro, pulito, conciso e documentato. Un codice sporco è difficile da ottimizzare perché nasconde le inefficienze. Usare il principio **DRY (Don't Repeat Yourself)** evita il debito tecnico e facilita il debug. Inoltre, è bene che il codice sia **commentato**.
- Un'altra cosa da tenere in considerazione è il **Design** che comprende, in genere:
 - Teoria, quindi lettura di come si svolgono solitamente le cose.
 - Analisi dei vincoli e delle risorse che si hanno a disposizione.
 - Traduzione della teoria in codice computabile.
 - Testing che consiste nel verificare che tutto il codice o specifiche sezioni funzionino come ci si aspetta.
 - Validazione per assicurarci che il codice faccia ciò che deve fare e verificare per assicurarci che il codice faccia ciò che fa correttamente.

Un codice meravigliosamente ottimizzato che adotta l'algoritmo sbagliato è un controsenso, perché un algoritmo migliore, anche se implementato male, batterà (quasi) sempre il codice precedente per un numero di casi sufficientemente ampio.

In sintesi dovremmo lavorare come segue:

1. Scrivere un programma che ci dia le risposte corrette e che si comporti correttamente sempre.
2. Adottare gli algoritmi e le strutture dati più adatti ai nostri vincoli.
3. Avere un codice sorgente pulito e ottimizzabile dal compilatore.
4. Ottenere un flusso di lavoro basato sui dati e sui compiti.
5. Verificare il codice in diverse condizioni anche non direttamente dipendenti dal codice stesso come i vari carichi di lavoro ecc. per trovare i colli di bottiglia e le inefficienze.
6. Applicare tecniche di ottimizzazioni modificando i punti critici nel codice per rimuovere i blocchi di ottimizzazione.
7. Tornare a 1 se necessario.

Il **compilatore** è un programma che traduce il linguaggio sorgente in un linguaggio target equivalente. Un **interprete** è un programma di elaborazione del linguaggio che esegue direttamente un programma in un **linguaggio sorgente**. La compilazione fa parte di una **pipeline** che traduce il codice sorgente in un codice eseguibile dalla macchina. Questa pipeline comprende:

- **Preprocessore**: Analizza il sorgente includendo gli header ed espandendo le macro.
- **Front-end**: Traduce il sorgente in un linguaggio ad alto livello di astrazione specifico. Spesso assembly.
- **Assembler**: Produce il codice macchina dall'assembly generato.
- **Linker**: Risolve i riferimenti e gli indirizzi di memoria tra diverse sezioni del codice e librerie esterne.

I compilatori sono anche in grado di eseguire un'analisi sofisticata del codice sorgente in modo da produrre un codice target (solitamente un codice assembly) altamente ottimizzato per una data architettura target. Quindi, per prima cosa, lasciamo fare il lavoro al compilatore. Esistono diversi livelli di ottimizzazione generati automaticamente dal compilatore mediante specifico flag **-On**:

- `-O0`: Nessuna ottimizzazione (default), utile per il debug.
- `-O1`, `-O2`, `-O3`: Livelli crescenti di aggressività. **Nota bene**: non è detto che `-O3`, sebbene spesso generi un codice più veloce, sia ciò di cui hai realmente bisogno; su alcune architetture può generare codice troppo voluminoso che satura le cache, rendendo preferibile `-Os` (ottimizzazione per dimensione).
- `-march=native`: Istruisce il compilatore a generare codice specifico per l'architettura della CPU su cui sta girando, sfruttando tutti i set di istruzioni (ISA) disponibili. Infatti, il compilatore generalmente produce un codice portabile, cioè un codice che può essere eseguito su qualsiasi CPU appartenente a quella classe di architettura.

Esistono anche diversi modelli tipici di compilazione come, ad esempio **fbrach-probabilities** o **fprofile-generate**.

Altra cosa utile che approfondiremo maggiormente anche dopo è l'uso di **memoria contigua** che, in quanto contigua, evita continui salti da un punto di memoria a un altro.

Esistono anche diverse classi di storage:

- `extern`: variabili globali che esistono per sempre.
- `auto`: variabili locali allocate sullo stack.
- `register`: variabili che suggeriamo al compilatore di tenere in registro perché magari molto usata.

Inoltre, esistono anche dei quantificatori di variabili:

- **const**: indica che il valore dentro la variabile non cambia mai.
- **volatile**: indica che la variabile è modificabile dall'esterno del programma.
- **restrict**: indica che l'indirizzo di memoria è accessibile solo tramite specifico puntatore.

3.2.1 Memory Aliasing

Il **Memory Aliasing** si verifica quando due o più riferimenti (puntatori) puntano alla stessa posizione di memoria. Questo è uno dei maggiori blocchi alle prestazioni in C/C++.

```

void func1 ( int *a, int *b ) {
    *a += *b;    -> *a e *b adesso contengono 2
    *a += *b;    -> *a e *b adesso contengono 4
}

void func2 ( int *a, int *b )
    *a += 2 * *b; -> *a e *b adesso contengono 3
}

```

Consideriamo queste due funzioni. In effetti, se *a* e *b* puntano a due indirizzi diversi, il risultato delle due funzioni è lo stesso. Il discorso cambia, invece, quando *a* e *b* puntano allo stesso indirizzo come mostrato in figura.

Se il compilatore non può garantire che due puntatori siano distinti, non può ottimizzare gli accessi alla memoria perché teme che una scrittura su uno possa influenzare il valore dell'altro. Questo costringe la CPU a ricaricare continuamente i dati dalla memoria centrale anziché usare i registri o la cache. Per risolvere questo problema usiamo il qualificatore *restrict*.

In generale è utile:

- Scrivere codice evitando aliasing di memori rendendo chiaro a cosa serve una variabile e quando e mantenendo sotto controllo i branch condizionali.
- Usare strutture dati buone cache-friendly facendo sì che dati che vengono usati insieme, siano insieme. Inoltre, è necessario evitare il **false-sharing** che accade quando due thread lavorano su variabili diverse che però sfortunatamente risiedono nella stessa linea di cache, costringendo i core a invalidarsi la memoria a vicenda continuamente.
- Avere un buon flusso di lavoro così che il compilatore possa ottimizzare i branch e gli accessi alla memoria.
- Sfruttare le architetture multi-core superscalari e out of order.

Capitolo 4

Ottimizzazione Single-Core

Prima di tutto è bene iniziare dal punto più piccolo che si occupa di eseguire le nostre istruzioni cioè il **core**. Infatti, sebbene attualmente le CPU siano multi core, è importante cercare di capire come sfruttare ogni singolo core al meglio per tirare fuori il massimo potenziale da ogni risorsa a nostra disposizione.

Le prestazioni del software moderno dipendono dalla comprensione del divario tra l'architettura ideale di Von Neumann e la complessità dei core attuali. Mentre il modello classico prevede una memoria "piatta" con costi di accesso uniformi, le architetture moderne sono caratterizzate da gerarchie profonde e parallelismo interno per contrastare il **Data Starvation** ossia la mancanza dei dati necessari al lavoro attualmente in corso.

4.1 Memoria cache

Come già detto, a partire dai primi anni '90, la velocità delle CPU è cresciuta molto rapidamente rispetto a quella della RAM provocando un collo di bottiglia noto come **Memory Wall** in cui la CPU è costretta ad aspettare i tempi di accesso in memoria per continuare la sua elaborazione.

Per risolvere questo problema, si utilizza una **gerarchia di memoria** con costi in termini di cicli di clock crescenti:

1. **Registri:** questa è banalmente la memoria che utilizza il singolo core per effettuare le proprie operazioni e, ogni elemento su cui deve essere fatta un'operazione, deve passare da qui.
2. **Livello 1:** questa è una memoria cache presente all'interno di ogni singolo core e il suo accesso richiede circa 2 cicli di clock.
3. **Livello 2:** questa è una cache che può appartenere a uno o a più core e per effettuare l'accesso necessita di 10 cicli di clock.

4. **Livello 3:** infine troviamo una cache di terzo livello che è all'interno della CPU ed è condivisa da tutti i core.
5. **RAM:** La RAM è l'ultima "speranza" necessaria a recuperare le informazioni se non presenti in cache.

L'hit rate della memoria cache comporta grossi cambiamenti in termini di efficienza del sistema poiché basti pensare che passando dal 97% al 100% si passa dall'avere fino al 50% o 150% di prestazioni in più. Se volessimo fare i conti, dovremmo usare questa formula:

$$L1_{cost} * L1_{hitrate} + L1_{missrate} * RAM_{cost} = Cycles$$

Definiamo dei dati **locali** come dei dati che risiedono in una piccola porzione dello spazio degli indirizzi acceduta in un breve periodo di tempo. Da qui definiamo i due principi di località:

- **Località temporale:** dati referenziati di recente vengono spesso riutilizzati.
- **Località spaziale:** dati vicini in memoria vengono spesso referenziati insieme.

Da qui possiamo definire alcuni tipi di miss che possono andare a degradare le prestazioni della cache:

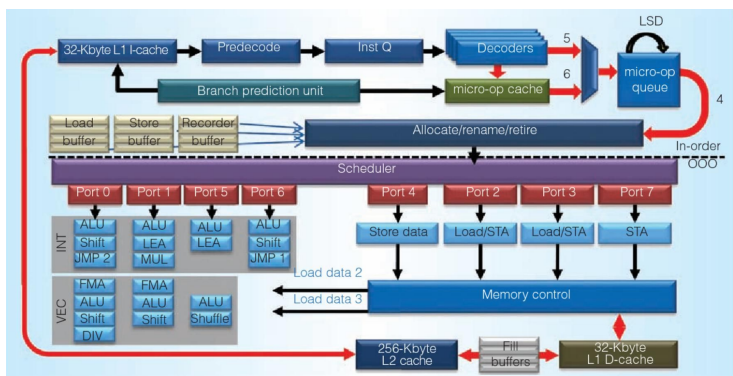
1. **Compulsory:** accessi inevitabili alla prima referenziazione a un dato.
2. **Capacità:** cache troppo piccola per contenere i dati necessari al lavoro.
3. **Conflitto:** svuotamento della cache dovuto alla mappatura dei dati sulle stesse linee di cache necessari per scambi di dati.

A questo punto, possiamo andare a dedurre anche quelli che sono dei modi per migliorare le prestazioni, banalmente, bilanciando i problemi di cui abbiamo parlato prima:

1. **Riorganizzare (codice e dati):** progettare il layout per migliorare la località temporale e spaziale.
2. **Ridurre (dimensione):** dimensione dei dati più piccola, meno istruzioni e per essere più facilmente contenuta in cache.
3. **Riutilizzare (linee di cache):** aumentare la località per mantenere i dati residenti per più operazioni.

Come abbiamo detto prima, sappiamo che le CPUCore, al giorno d'oggi contengono delle memorie cache "locali". Questo comporta il problema che se due cache di due core diversi contengono lo stesso dato, se uno accede in scrittura potrebbe modificare il dato senza che l'altro se ne renda conto. Questo problema comporta un certo costo difficoltà per essere gestito correttamente.

A metà degli anni '90 le CPU abbiamo già detto che diventano molto più veloci della memoria. Questo è dovuto principalmente all'uso di **CPU superscalari**:



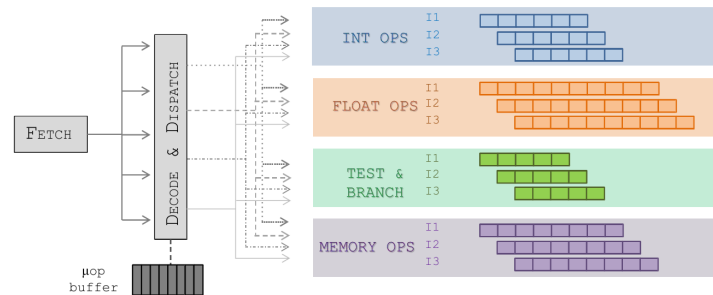
La capacità superscalare di questo tipo di CPU è data dall'avere diverse **porte di accesso** per mandare in esecuzione le istruzioni e diverse unità logiche per eseguirle. Notiamo una divisione in:

- **FrontEnd:** è la parte che recupera le istruzioni dalle cache le decodifica, predice i branch e smista le istruzioni sulle diverse porte.
- **BackEnd:** è la parte responsabile dell'esecuzione effettiva delle istruzioni e della riscrittura in memoria nel caso in cui sia necessario. Questa parte si occupa anche di gestire l'esecuzione delle operazioni **out-of-order** a seconda delle dipendenze che possono esserci.

Le operazioni effettuate dalla nostra CPU seguono in realtà una serie di passaggi ben definiti:

- **Fetch:** fase in cui vengono richiamate le istruzioni dalla memoria/cache.
- **Decode:** fase in cui l'istruzione è interpretata e compresa.
- **Esecuzione:** in cui l'operazione viene eseguita.
- **Writeback:** fase in cui il risultato deve essere registrato in memoria.

Queste operazioni possono essere eseguite da una singola istruzione per volta e, chiaramente richiedono un certo tempo per essere eseguite. Per evitare di dover aspettare che ogni singola istruzione finisca la Pipeline, facciamo sì che ogni istruzione sia in un certa fase diversa della pipeline, così da per aumentare il **throughput**. Un altro modo per ottimizzare la pipeline è quello di utilizzare **pipeline multiple** che si dividono in base al tipo di istruzione che è necessario effettuare. Infatti, abbiamo detto che esistono varie unità di esecuzione che sono indipendenti tra di loro e che, quindi, possono lavorare in parallelo.



Il pipelining è sicuramente ottimo quando le istruzioni sono tutte indipendenti l'una dalle altre, tuttavia introducono alcuni problemi:

1. **Data hazard:** un'istruzione dipende dal risultato di un'istruzione precedente che non ha ancora completato la pipeline. **Soluzione:** La CPU deve inserire stalli (bubble) nella pipeline, perdendo cicli produttivi.
2. **Control hazard:** quando la CPU incontra un branch condizionale, non sa quale ramo eseguirà finché non valuta la condizione, ma nel frattempo la pipeline ha già iniziato a fare il prefetch delle istruzioni successive. **Soluzione:** branch predictor, un componente hardware che cerca di indovinare quale ramo verrà preso, basandosi sulla storia dei branch precedenti.

Infine, un'ulteriore ottimizzazione viene introdotta alla fine degli anni '90 con i **registri vettoriali**. Questi sono grandi registri nella CPU che permettono di eseguire una singola istruzione su tutti gli elementi del registro vettoriale in parallelo. Infine, all'inizio degli anni 2000 troviamo che i sistemi sono dotati di più core avvicinandoci quindi alla struttura più comune a oggi.

4.3 Evitare le Inefficienze

Scrivere codice pulito e logico è il prerequisito fondamentale per scrivere del buon codice, tuttavia, per ottenere le massime prestazioni da esso è necessario evitare inefficienze legate a salti, pipeline, cicli e controlli condizionali. Esistono cinque regole pratiche da applicare durante la stesura del codice.

4.3.1 Esempio che seguiremo

Supporremo di avere una distribuzione di punti casuali in un punto 2D che è diviso in sottoregioni. Per ogni punto, vogliamo selezionare tutte le celle della griglia il cui centro è più vicino a p di un certo raggio r ed eseguire su di esse alcune operazioni. Partiamo, dunque, da questo codice:

```
for (p = 0; p < Np; p++) {
    for (i = 0; i < Ng; i++) {
        for (j = 0; j < Ng; j++) {
            for (k = 0; k < Ng; k++) {
                dist = sqrt(
                    pow(x[p] - (double)i/Ng + half_size, 2) +
                    pow(y[p] - (double)j/Ng + half_size, 2) +
                    pow(z[p] - (double)k/Ng + half_size, 2)
                );
                if (dist < R) {
                    fai_qualcosa;
                }
            }
        }
    }
}
```

4.3.2 1. Evitare Chiamate a Funzioni Costose

Alcune chiamate a funzioni, sono parecchie costose anche a causa del doversi collegare a delle librerie esterne. In questo codice, ad esempio, poiché calcoliamo la distanza tra vari punti, troviamo la funzione *pow* e la funzione

sqrt per fare il quadrato e la radice. Notiamo che possiamo anche evitare questa cosa poiché ci basta considerare la distanza al quadrato dato che, alla fine, ci basta che vengano presi quegli specifici punti. Inoltre, sappiamo che i computer non sono molto bravi a effettuare le operazioni di divisione poiché hanno la necessità di calcolare l'inverso. Per evitare questo, possiamo precalcolarlo se è necessario effettuare la divisione più volte. Infine, poiché $(double)i * Ng_inv + half_size$ è indipendente dal punto preso, possiamo anche questo, precalcolarlo in un array.

```
for (i = 0; i < Ng; i++) {
    for (j = 0; j < Ng; j++) {
        for (k = 0; k < Ng; k++) {
            dx = x[p] - (double)i * Ng_inv + half_size;
            dy = y[p] - (double)j * Ng_inv + half_size;
            dz = z[p] - (double)k * Ng_inv + half_size;

            dist2 = dx*dx + dy*dy + dz*dz;
            if (dist2 < R2) {
                /* fai qualcosa */
            }
        }
    }
}
```

Nella versione presenta, ancora non è stata ancora inserita l'ultima ottimizzazione.

4.3.3 2. Hoisting delle espressioni

Guardando il codice precedente, possiamo soffermarci su alcuni specifici calcoli. Infatti, nel ciclo più interno noi calcoliamo dx e dy senza che questi dipendano effettivamente da k . Per quanto questo possa sembrare semplicemente una scelta per non rendere il codice confusionario, in realtà provoca grossi ritardi a causa del fatto che questi vengono calcolati k volte più del necessario. Possiamo dunque spostarli nel ciclo più in alto così da evitare questo problema.

```
double ijk[Ng];
for (i = 0; i < Ng; i++) {
```

```

    ijk[i] = (double)i * Ng_inv + half_size; // Pre-calcolo
}

for (i = 0; i < Ng; i++) {
    dx2 = x[p] - ijk[i];
    dx2 = dx2 * dx2; // Spostato nel ciclo 'i'

    for (j = 0; j < Ng; j++) {
        dy2 = y[p] - ijk[j];
        dy2 = dy2 * dy2;
        dist2_xy = dx2 + dy2; // Spostato nel ciclo 'j'

        for (k = 0; k < Ng; k++) {
            dz = z[p] - ijk[k];
            dist2 = dist2_xy + dz * dz; // Solo questo in k

            if (dist2 < Rmax2) { /* fai qualcosa */ }
        }
    }
}

```

4.3.4 3. Chiarire lo Scope delle Variabili

Nell'esempio considerato, troviamo delle variabili che sono funzionali alla specifica operazione che stiamo facendo. In particolare ci riferiamo a dx , dy , dz e $dist$. Una cosa che possiamo fare è andare a definirle all'interno stesso del ciclo in cui vengono utilizzate. Questo aiuta di molto il compilatore che sa con certezza che non è necessario conservarne il valore al di fuori di quelle parentesi.

```

for(int i = 0; i < Ng; i++) {
    // dx2 vive solo all'interno del ciclo 'i'
    double dx2 = x[p] - (double)i * Ng_inv + half_size;
    dx2 *= dx2;

    for(int j = 0; j < Ng; j++) {
        // dy2 e dist2_xy vivono solo qui
        double dy2 = y[p] - (double)j * Ng_inv + half_size;
        double dist2_xy = dx2 + dy2 * dy2;

        for(int k = 0; k < Ng; k++) {
            // dz e dist2 vivono solo qui

```

```
double dz = z[p] - (double)k * Ng_inv + half_size;
double dist2 = dist2_xy + dz * dz;

if (dist2 < Rmax2) {
    // fai qualcosa con sqrt(dist2);
}
}
}
```

4.3.5 4. Suggerisci cosa è Importante

Quando una variabile viene calcolata e riutilizzata frequentemente all'interno di un loop critico, può essere utile suggerire al compilatore di dedicarle un registro hardware utilizzando la parola chiave `register`. È fondamentale ricordare che si tratta solo di un **suggerimento non vincolante**: il compilatore, dopo aver analizzato il codice, potrebbe comunque decidere una strategia di allocazione diversa a causa di ragionamenti interni suoi sul come avrà intenzione di gestire la pipeline.

4.3.6 5. Non ripetere controlli o referenziazioni non necessarie

Spesso, anche a causa delle nostre abitudini di lavoro, ci ritroviamo spesso a ripetere dei controlli che spesso sono ridondanti rispetto a quello che già controlliamo. Infatti, questo è dovuto al fatto che la maggior parte delle volte siamo convinti di fare un bene al nostro codice rendendolo più sicuro quando, in realtà, non stiamo facendo altro che rallentarlo. Vediamo un esempio:

```
char * find_char_in_string( char *string, char c )
{
    int i = 0;
    // strlen() viene chiamata a ogni iterazione
    while ( i < strlen(string) ) {
        if ( string[i] == c )
            break;
        else
            i++;
    }
    if ( i < strlen(string) )
```

```
        return &string[i];
    else
        return NULL;
}
```

In questo caso, il problema di performance è causato dal fatto che `strlen` attraversa l'intera stringa a ogni ciclo per calcolarne la lunghezza, portando a una complessità quadratica. La stringa, ovviamente, non cambia nel corso delle iterazioni, dunque, ha senso calcolarla prima così da non ripetere una chiamata a funzione esterna. Esistono due soluzioni parecchio interessanti:

```
//V.1 La lunghezza è pre-calcolata
int i = 0;
int len = strlen(string);
while (i < len) {
    if(string[i] == c) break;
    else i++;
}

//V.2 Controllo del terminatore tramite puntatori
char *pos = string;
while((*pos != '\0') && (*pos != c))
    pos++;
return (*pos == '\0'?NULL:pos);
```

Simile è quello che accade con questa funzione riguardo alla referenziazione della memoria:

```
// Accesso in memoria ad ogni iterazione
void reduce_vector(int n, double *array, double *sum) {
    for (int i = 0; i < n; i++) *sum += array[i];
    // Scrive e legge l'indirizzo di 'sum' N volte
}
```

L'introduzione di un accumulatore locale separato permette di sfruttare i registri veloci della CPU, limitando la scrittura in memoria a una sola istruzione una volta terminato il ciclo:

```
// Uso di un accumulatore locale
void reduce_vector(int n, double *array, double *sum) {
    double temp = 0; // Variabile locale
    for (int i = 0; i < n; i++) {
        temp += array[i]; // Elaborazione veloce e locale
    }
    *sum = temp; // Un solo accesso in memoria
}
```

4.3.7 Avvertenza su Numeri in Virgola

Si potrebbe pensare che un compilatore con un alto livello di ottimizzazione (es. -O3) sia sempre in grado di riorganizzare i calcoli matematici per renderli più convenienti ma **questo non è vero per i numeri in virgola mobile**.

Mentre l'aritmetica intera in complemento a 2 è commutativa e associativa, la matematica in virgola mobile non è mai associativa:

$$(a + b) + c \neq a + (b + c) \quad (4.1)$$

A causa del numero limitato di cifre (precisione finita), l'ordine delle operazioni altera il risultato finale. Ad esempio, considerando una precisione limitata, sommare $1.00 + 0.001$ restituisce 1.00 , causando una perdita del decimale che invece non si avrebbe sommando prima i numeri più piccoli. Di conseguenza, il compilatore **non rimescolerà mai l'ordine delle operazioni floating point**, costringendo lo sviluppatore a scrivere direttamente la forma algebrica più efficiente nel codice sorgente.

4.4 Ottimizzazione basata sulla Cache

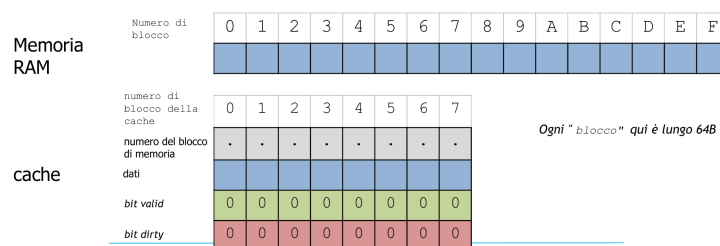
Abbiamo già visto la cache e il motivo della sua introduzione per risolvere il problema del **Memory Wall** e abbiamo già parlato della **gerarchia di memoria** che questa introduce. Per cercare di capire come ottimizzare al meglio l'utilizzo della cache ripetiamo adesso i due principi fondamentali:

- **Località temporale:** dati referenziati di recente vengono spesso riutilizzati.
- **Località spaziale:** dati vicini in memoria vengono spesso referenziati insieme.

Teniamo bene a mente questi due principi per capire comprendere al meglio le scelte che vengono prese nella mappatura della cache.

4.4.1 Mappatura della Cache

Supponiamo che la memoria RAM e la cache siano divisi in **blocchi** di uguale dimensione. Quando andremo a spostare un blocco in cache, non sposteremo dunque il singolo bit, bensì tutto il singolo blocco.



In cache oltre ai blocchi in sé per sé troviamo:

- **bit valid:** che è un bit che ci indica se quel dato è ancora valido o se non è più presente nella RAM e, quindi, può essere anche eliminato nella cache se necessario.
- **bit dirty:** è un bit che ci indica che il dato presente in cache è stato modificato rispetto al dato presente in RAM e che, quindi, è necessaria una scrittura quando possibile. Possono esserci due strategie riguardo a questo bit, la prima è che si scrive sempre sulla RAM una volta che il dato è stato modificato la seconda è che si posticipa la scrittura in RAM fino a quando si può. La seconda opzione è consigliata quando si hanno tanti cambiamenti di dati così da non dover fare molte scritture.

Tornando alla mappatura da memoria RAM a cache troviamo le seguenti strategie:

- **Completamente associativa:** I dati possono andare in qualsiasi blocco libero. È efficiente per evitare conflitti in scrittura, ma lenta in lettura perché richiede la ricerca in tutta la cache.
- **Mappatura diretta:** Un indirizzo RAM può risiedere in un solo specifico blocco. Estremamente veloce da trovare, ma massimizza i conflitti se più dati utili competono per lo stesso blocco.
- **Set-associativa a N-vie:** È il compromesso tipico delle architetture moderne (es. associativa a 8 vie). I dati possono essere inseriti in uno specifico sottoinsieme (set) di blocchi.

Per scrivere codice efficiente bisogna gestire le tre C e le tre R:

- **Compulsory miss** → Inevitabili al primo accesso.
- **Capacity miss** (Spazio insufficiente) → Si contrasta con la regola **Ridurre**: usare tipi di dati più piccoli e lavorare su working set limitati.
- **Conflict miss** (Sovrascrittura di linee cache utili) → Si contrasta con le regole **Riorganizzare** (layout di memoria per migliorare la località) e **Riutilizzare** (completare tutte le operazioni su un dato mentre è ancora residente in cache).

4.4.2 Modello di accesso alla Memoria

Le prestazioni della memoria formano una superficie tridimensionale nota come *Memory Mountain*, che dipende strettamente da due fattori: la **dimensione del *working set*** (quanti dati stiamo manipolando) e lo **Stride** (il "passo" con cui leggiamo gli elementi in memoria).

Lavorare con un set di dati piccolo e uno stride unitario (elementi contigui) massimizza la banda passante (*Bandwidth*). Al contrario, scorrere grandi array saltando posizioni di memoria (stride elevato) distrugge le prestazioni perché spreca le linee di cache caricate.

Esempio: Impatto dello Stride nell'Accesso alla Memoria

La CPU non carica in cache singoli elementi interi, ma intere linee da 64 byte. Nel primo ciclo (`stride = 1`), un singolo accesso alla RAM carica in cache un blocco che contiene anche gli elementi successivi, rendendo le iterazioni seguenti istantanee.

```
#define N 1000000
int a[N];
int sum = 0;

/* Accesso sequenziale (Stride = 1) */
/* Veloce: sfrutta appieno l'intera cache line*/
for (int i = 0; i < N; i++) {
    sum += a[i];
}
```

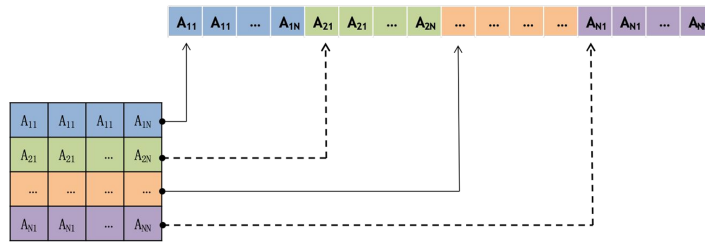
Se incrementiamo l'indice in modo non sequenziale, stiamo ignorando i dati portati in cache. Ad ogni salto, la CPU va in *cache miss* ed è costretta a pagare la penalità di centinaia di cicli per interrogare nuovamente la RAM.

```

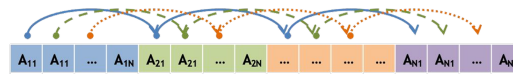
/* Accesso sparso (Stride grande) */
/* Lento: spreca quasi tutta la cache line e genero miss*/
for (int i = 0; i < N; i += 16) {
    sum += a[i];
}

```

Questo principio diventa critico quando si opera su **matrici multidimensionali**. In linguaggi come C e C++, le matrici non sono tabelle magiche, ma blocchi unidimensionali e continui di memoria. La convenzione di questi linguaggi è il **row-major order** (ordine per righe): gli elementi della stessa riga sono memorizzati consecutivamente in RAM. Al contrario, leggere una matrice scorrendola per colonne significa effettuare salti enormi (*stride*) nella memoria fisica, abbattendo radicalmente le prestazioni del programma (uno *stride* pari a $N \times S$ byte, dove N è la dimensione della riga e S la dimensione del dato).



La lettura della matrice quindi per colonne è chiaramente estremamente costosa dato che è necessario effettuare grandi salti andando quindi a passare da un punto della memoria a un altro anche provocando molti cache miss.



In genere, l'indice interno è quello che varia più velocemente e traccia la contiguità della memoria e, quindi, abbiamo che $M[i][j][k]$ è contiguo a $M[i][j][k+1]$. Considerando tutto quello che abbiamo detto, analizziamo questa sezione di codice:

Esempio: Lettura vs Scrittura con Stride

Nel primo caso leggiamo in modo contiguo ma scriviamo a salti. Nel secondo caso scriviamo in modo contiguo ma leggiamo a salti.

```
// Modalità 1: Lettura contigua, Scrittura con stride
for(int row = 0; row < N; row++) {
    for(int col = 0; col < N; col++) {
        A[col * N + row] = B[row * N + col];
    }
}

// Modalità 2: Scrittura contigua, Lettura con stride
for(int row = 0; row < N; row++) {
    for(int col = 0; col < N; col++) {
        A[row * N + col] = B[col * N + row];
    }
}
```

In entrambi i casi abbiamo uno stride, tuttavia, questi non sono uguali. Quando si modifica il contenuto di una variabile, tale modifica equivale a sovrascrivere una posizione di memoria. Quando la variabile viene caricata nella memoria cache, ciò che viene caricata è una copia della variabile, che mantiene la sua posizione originale nella memoria principale. Quindi, se modifichiamo il suo valore, dobbiamo modificarlo solo nella cache o anche nella memoria principale?

- **Write-Through:** La modifica viene scritta immediatamente sia in cache che in RAM mantenendoli coerenti.
- **Write-Back:** La modifica viene apportata solo in cache e la RAM viene aggiornata solo quando la linea di cache viene sostituita.

In caso di *write-miss* in una cache di tipo write-through possiamo allocare prima il blocco in cache e poi effettuare la scrittura in entrambi (write-allocate) o scrivere direttamente sulla memoria principale (non-write-allocate). Nell'ultimo caso la scrittura a raffica è conveniente per evitare continue allocazioni in cache.

4.4.3 Organizzazione dei Dati

Poiché lavoriamo con cache che hanno generalmente dimensioni molto piccole, organizzare i dati per far sì che questi stiano dentro la cache diventa fondamentale per sfruttare il principio di località spaziale.

Scansione per blocchi

Oltre alla classica scansione per righe/colonne esiste un altro tipo di scansione detta **scansione per blocchi** fatta appositamente per essere sfruttata quando lavoriamo con operazioni molto pesanti come moltiplicazioni o inversioni di matrice. L'idea è che invece di attraversare intere righe o colonne, dividiamo la matrice in **sottomatrici più piccole** dette blocchi scegliendo la dimensione facendo sì che i blocchi delle matrici coinvolte siano tutti in cache così da abbattere i cache miss.

```

0-1-2-3 16-17-18-19
4-5-6-7 20-21-22-23
8-9-10-11 24-25-26-27
12-13-14-15 28-29-30-31
32-33-34-35 48-49-50-51
36-37-38-39 52-53-54-55
40-41-42-43 56-57-58-59
44-45-46-47 60-61-62-63

```

Anziché codificare manualmente la dimensione del blocco, è possibile memorizzare direttamente la matrice in RAM utilizzando pattern non lineari, come la **Curva di Morton** (o Z-order). Questo ordinamento mappa dati multidimensionali in 1D mantenendo fisicamente vicini in memoria i punti che sono vicini nello spazio 2D o 3D, migliorando le prestazioni medie su architetture diverse senza bisogno di un *tuning* manuale dei blocchi.

Campi Caldi e Campi Freddi

Un errore comune nell'ottimizzazione è progettare strutture dati (`struct`) che mescolano dati ad accesso frequente ("caldi") con dati raramente utilizzati ("freddi"). Nelle liste concatenate, ad esempio, i puntatori e le chiavi di ricerca vengono letti ad ogni iterazione, mentre il carico utile (payload) viene letto solo in caso di successo.

Esempio: Ottimizzazione delle Strutture Dati

Nella prima versione, l'inserimento di un grosso array tra la chiave e il puntatore al nodo successivo allontana fisicamente i due dati. Questo distrugge la località spaziale: la cache verrà inondata da byte inutili caricati per via del payload (`my_data`).

```
// Versione NON ottimizzata (Campi mischiati)
struct my_node {
    double key;                // HOT
    char my_data[300];         // COLD: Allontana key e next
    my_node *next_node;       // HOT
};
```

La soluzione consiste nell'estrarre i dati "freddi" in una struttura separata allocata altrove, lasciando nella struttura principale solo un puntatore. In questo modo, `key` e `next_node` diventano adiacenti in memoria e verranno caricati insieme in una singola e veloce linea di cache.

```
// Versione OTTIMIZZATA (Separazione Hot/Cold)
struct my_data {
    char data[300];
};

struct my_node {
    double key;                // HOT
    my_node *next_node;        // HOT: Ora è adiacente alla chiave!
    struct my_data *payload;    // COLD: Puntatore ai dati esterni
};
```

4.5 Branch condizionali

Ogni volta che la sequenza di operazioni da eseguire o i dati da elaborare dipendono da una condizione (ad esempio, l'esito di un test 'if'), il programma effettua un'esecuzione condizionale. Le architetture moderne gestiscono questa situazione a basso livello in due modi distinti: modificando il flusso di controllo oppure modificando il flusso di dati.

Flusso di Controllo

A livello macchina, il controllo viene passato a una sezione diversa del codice tramite istruzioni di salto (*jump*). Queste possono essere:

- **Incondizionate** (`jmp`): Il salto avviene sempre.
- **Condizionali** (`je`, `jne`, `jg`, `jl`, ecc.): L'esecuzione dipende dall'esito di un test precedente. Queste istruzioni leggono i bit del **registro dei flag** (es. *Zero Flag*, *Sign Flag*, *Carry Flag*), che memorizza lo stato dell'ultima operazione logico-aritmetica.

In questo scenario, il codice generato dal compilatore è pieno di bivi e diramazioni fisiche.

Flusso di Dati

Le CPU moderne offrono un meccanismo alternativo: invece di saltare a blocchi di codice diversi, calcolano in anticipo i possibili risultati e poi spostano nei registri finali solo il dato corretto (es. Tramite l'istruzione assembly `cmov - conditional move`). Questo approccio non genera bivi nel codice, mantenendo un flusso di esecuzione lineare e altamente prevedibile. Tuttavia, è applicabile solo a scenari semplici poiché altrimenti il numero di operazioni necessarie al calcolo supera il vantaggio dato dal calcolare tutti i risultati.

4.5.1 Branch Predictor

Per mantenere le pipeline di esecuzione sempre piene, le CPU utilizzano un componente hardware chiamato **Branch Predictor**, che cerca di indovinare in anticipo l'esito di una diramazione prima che questa venga effettivamente valutata per anticiparne i calcoli.

I migliori predittori moderni hanno un'accuratezza del 95%. Tuttavia, quando il predittore sbaglia (**Branch Miss** o *Misprediction*), la CPU è costretta a svuotare l'intera pipeline scartando il lavoro fatto in anticipo. Questa penalità costa tipicamente dai 10 ai 30 cicli di clock per ogni errore.

L'impatto statistico è enorme: supponiamo di avere un blocco di 140 istruzioni in cui 1 istruzione su 7 è un branch (quindi 20 branch in totale). Assumendo eventi indipendenti, la probabilità che la pipeline *non* venga mai svuotata con un predittore accurato al 95% crolla drasticamente:

$$P(\text{noflush}) = 0.95^{20} \approx 0.3585 \Rightarrow 35.85\%$$

Con un predittore al 90%, la probabilità scende ad appena il 12.16%. Per questo motivo, i branch condizionali all'interno dei cicli (*loop*) sono i nemici principali delle prestazioni e vanno evitati o ristrutturati.

Ecco perché la seconda strategia che abbiamo visto, la modifica condizionale del flusso di dati, è preferibile quando possibile e il compilatore cercherà di utilizzarla il più possibile poiché non genera istruzioni di salto e il flusso di esecuzione è lineare e perfettamente prevedibile. Tuttavia, come abbiamo detto, si applica solo a un limitato sottoinsieme di casi.

Una cosa da tenere in considerazione che provoca un miglioramento delle prestazioni è che il branch con la condizione vera è quello che sta più vicina rispetto all'if stesso. Dunque, per evitare salti da una parte all'altra del codice, sarebbe bene che il branch vero sia quello che è più probabile che venga utilizzato.

Idealmente vorremmo evitare i branch condizionali il più possibile all'interno dei cicli:

- Spostandoli fuori da Loop e scrivendo Loop più specializzati.
- Esecuzioni di variabili/quantità impostate fuori dal ciclo.
- Utilizzando puntatori alle funzioni invece di selezionare funzioni all'interno di un ciclo.
- Uso di operazioni diverse per evitare branch.

4.5.2 Tecniche per limitare i Branch

In genere ci sono due casi possibili:

- Le variabili testate nelle condizioni non cambiano durante il ciclo. Questo è il caso semplice e con poca ristrutturazione si hanno grandi miglioramenti.
- Le variabili cambiano nel ciclo. Qui abbiamo bisogno di molto lavoro in più per cercare di risolvere il problema.

Vediamo adesso una carrellata di esempi.

Sollevamento del loop

Se una condizione testata all'interno di un ciclo non dipende dall'iteratore, è un grave errore ripeterla ad ogni passo. La diramazione va spostata *fuori* dal ciclo, scrivendo versioni specializzate del loop stesso.

```
// Versione NON ottimizzata: il test viene ripetuto 'top' volte
for(int i = 1; i < top; i++) {
    if (value > 1) do_something();
    else          do_something_else();
}

// Versione OTTIMIZZATA: il test viene eseguito UNA sola volta
if (value > 1) {
    for (int i = 1; i < top; i++) do_something();
} else {
    for (int i = 1; i < top; i++) do_something_else();
}
```

Nota: Nei casi di scelte multiple basate su uno stato costante, è nettamente preferibile usare un costrutto `switch` invece di una cascata di `if/else`. Il compilatore traduce lo `switch` in una tabella statica di puntatori detta **jump table** a cui si accede in tempo costante $O(1)$ senza diramazioni condizionali.

Flussi di dati imprevedibili

Quando la condizione dipende dai dati stessi (es. l'elemento corrente di un array casuale), il predittore fallirà continuamente. Spesso si può sostituire l'if con un'operazione booleana moltiplicativa.

```
// Versione NON ottimizzata: Rischio altissimo di Branch Miss
for (int i = 0; i < SIZE; i++) {
    if (data[i] > PIVOT)
        sum += data[i];
}

// Versione OTTIMIZZATA: Nessun salto! Il flusso è lineare.
for (int i = 0; i < SIZE; i++) {
    sum += (data[i] > PIVOT) * data[i];
}
```

Poiché ($data[i] > PIVOT$) viene valutato come 1 (Vero) o 0 (Falso), è possibile sfruttare questa logica per eliminare il branch.

Ordinamento degli elementi

Un caso classico è lo scambio condizionale di valori tra due array per mantenere un ordinamento. L'uso esplicito dell'if distrugge le prestazioni. È preferibile usare operatori condizionali ternari o maschere bit a bit che il compilatore può tradurre in istruzioni `cmov` (flusso di dati condizionale senza salti).

```
// Versione NON ottimizzata
for (int i = 0; i < SIZE; i++) {
    if (A[i] < B[i]) {
        int t = B[i];
        B[i] = A[i];
        A[i] = t;
    }
}

// Versione OTTIMIZZATA: Sfrutta min e max
for (int i = 0; i < SIZE; i++) {
    int min = (A[i] > B[i]) ? B[i] : A[i];
    int max = (A[i] >= B[i]) ? A[i] : B[i];
}
```

```
A[i] = max;  
B[i] = min;  
}
```

Cambiare il POV

A volte il modo migliore per rimuovere i branch è ripensare i limiti stessi dei cicli, invece di usare ‘if’ interni per discriminare le aree spaziali (es. in una matrice).

Vogliamo riempire la diagonale di una matrice $M \times N$ con 0.0, il triangolo superiore con 1.0 e quello inferiore con -1.0 .

```
// Versione NON ottimizzata: if/else dentro i loop annidati  
for (int j = 0; j < N; j++) {  
    for (int i = 0; i < M; i++) {  
        if (i > j)          matrix[j][i] = 1.0;  
        else if (i <= j) matrix[j][i] = -1.0;  
        else                matrix[j][i] = 0.0;  
    }  
}  
  
// Versione OTTIMIZZATA: Loop separati basati sulla geometria  
for (int j = 0; j < N; j++) {  
    int i;  
    for (i = 0; i < j; i++)          // Fino alla diagonale  
        matrix[j][i] = -1.0;  
  
    matrix[j][i] = 0.0;              // Sulla diagonale esatta  
  
    for (i = j + 1; i < M; i++)      // Dopo la diagonale  
        matrix[j][i] = 1.0;  
}
```

4.6 Ottimizzazione dei Loop

Ottimizzare i cicli significa massimizzare il riutilizzo dei dati e mantenere le pipeline della CPU sempre piene. Per capire quanto un ciclo sia ottimizzabile, si introduce il concetto di **Intensità Aritmetica** (A_I), definita come

il rapporto tra il numero di operazioni eseguite $f(n)$ e la quantità di dati trasferiti n :

$$A_I = \frac{f(n)}{n} \quad (4.2)$$

In base all'intensità aritmetica, i cicli si classificano in tre categorie:

1. $O(N)/O(N)$: Es. Addizioni vettoriali o prodotti scalari. Il potenziale di ottimizzazione è limitato perché il limite è imposto dalla larghezza di banda della memoria. Le migliori derivano dall'evitare accessi ripetuti (es. *Loop Fusion*).
2. $O(N^2)/O(N^2)$: Es. Moltiplicazione matrice-vettore. Offre maggiori opportunità di ottimizzazione sfruttando la località spaziale e temporale.
3. $O(N^3)/O(N^2)$: Es. Moltiplicazione matrice-matrice. Ha un enorme potenziale di ottimizzazione perché i dati caricati in cache possono essere riutilizzati molteplici volte per fare calcoli.

4.6.1 Ottimizzazioni per Cicli a Bassa Intensità: Loop Fusion

Quando si elaborano cicli $O(N)/O(N)$, l'obiettivo principale è ridurre il traffico verso la memoria principale. Se due cicli consecutivi scorrono gli stessi array, fonderli in un unico ciclo (**Loop Fusion**) dimezza i caricamenti dalla RAM.

Esempio: Fusione dei Loop (Loop Fusion)

```
//B[i] viene caricato dalla RAM due volte
for(int i=0; i<N; i++)
    A[i] = B[i] * C[i];
for(int i=0; i<N; i++)
    Q[i] = B[i] + D[i];

//OTTIMIZZATA: Fusione, B[i] è letto una sola volta.
for(int i=0; i<N; i++) {
    A[i] = B[i] * C[i];
    Q[i] = B[i] + D[i];
}
```

4.6.2 Ottimizzazioni per Cicli $O(N^2)$: Unroll & Jam

Nei cicli annidati, è possibile ridurre l'overhead e migliorare il riutilizzo dei dati srotolando il ciclo esterno e fondendolo con quello interno (**Unroll & Jam**).

Tuttavia, bisogna prestare attenzione al **Register Spilling**: se si srotola troppo (es. un fattore m troppo grande), la CPU esaurisce i registri hardware disponibili ed è costretta a riversare continuamente i dati temporanei nella cache, rallentando drasticamente il calcolo (*codice gonfiato*).

Esempio: Unroll & Jam in una Moltiplicazione Matrice-Vettore

```
// 1. Versione Base (3*N^2 accessi)
for(int i=0; i<N; i++)
    for(int j=0; j<N; j++)
        C[i] += A[i][j] * B[j];

// 2. Non accediamo a C[i] ma a una variabile locale
for(int i=0; i<N; i++) {
    double c_temp = C[i];
    for(int j=0; j<N; j++)
        c_temp += A[i][j] * B[j];
    C[i] = c_temp;
}

// 3. Unroll & Jam
for(int i=0; i<N; i+=2) {
    for(int j=0; j<N; j++) {
        double b_temp = B[j];
        C[i]   += A[i][j]   * b_temp;
        C[i+1] += A[i+1][j] * b_temp;
    }
}
```

4.6.3 Srotolamento del Ciclo (Loop Unrolling) e Aritmetica Floating Point

Lo srotolamento (*unrolling*) è una trasformazione fondamentale che espone il parallelismo a livello di istruzione (ILP) e riduce il costo di aggiornamento del contatore.

Esiste però un'enorme differenza tra variabili intere e in virgola mobile. Come discusso in precedenza, **la matematica in virgola mobile non è associativa**. Se si compila una semplice somma di array ('S += A[i]') usando il

tipo `int`, il compilatore vettorizza automaticamente il codice massimizzando le prestazioni. Se si usa il tipo `double`, il compilatore **non** ottimizzerà il ciclo, perché alterare l'ordine delle somme cambierebbe il risultato numerico a causa degli arrotondamenti. È quindi responsabilità del programmatore riassociare esplicitamente le operazioni.

Esempio: Srotolamento e Riassociazione Esplicita per i Double

Per sfruttare le porte vettoriali della CPU con i 'double', dobbiamo srotolare manualmente il ciclo e forzare un nuovo ordine di valutazione tramite le parentesi, esponendo operazioni indipendenti.

```
// la CPU deve aspettare la fine della somma precedente
for (int i=0; i<N; i++)
    S = S + A[i];

// Versione Srotolata 2x1
// (Riduce l'overhead del ciclo, ma resta sequenziale)
for (int i=0; i<N-2; i+=2)
    S = (S + A[i]) + A[i+1];

// Versione Srotolata e RIASSOCIATA (Parallelismo esposto!)
for (int i=0; i<N-2; i+=2)
    S = S + (A[i] + A[i+1]);
```

Nota: Modificando l'ordine in '(A[i] + A[i+1])', le due somme elementari sono indipendenti dal valore precedente di 'S'. La CPU può calcolare questo blocco in parallelo nelle proprie pipeline vettoriali (es. istruzioni AVX `vaddpd`) e sommarlo ad 'S' solo alla fine, abbattendo drasticamente il tempo di esecuzione. Ricordarsi sempre di gestire la "coda" (tail) del ciclo per gli elementi residui non multipli del fattore di srotolamento.

4.6.4 Esempio Pratico $O(N^3)/O(N^2)$: Moltiplicazione Matrice-Matrice (MMM)

La moltiplicazione di due matrici quadrate $C = A \times B$ richiede $2N^3$ operazioni floating point (flop). A causa dell'ordine *row-major* del linguaggio C, attraversare la matrice B per colonne provoca un'enorme quantità di *cache miss*. Un'implementazione ingenua genera quasi un cache miss per ogni flop, degradando le prestazioni.

Esistono tre fasi progressive per ottimizzare questo problema computazionale:

1. **Trasposizione preventiva:** Trasporre la matrice B in memoria prima dei calcoli permette di accedere ad entrambe le matrici sequenzialmente (per righe).
2. **Scambio dei cicli (Loop Swapping):** Cambiare l'ordine di annidamento dai classici $i-j-k$ a $i-k-j$. Questo approccio accede agli elementi di C in modo contiguo e legge B in ordine di riga, riducendo le letture dalla RAM.
3. **Elaborazione a Blocchi (Tiling):** Mantenere in cache sottomatrici fisse (blocchi o *tiles*). Invece di calcolare un'intera riga per volta, si accumulano risultati parziali su un blocco ristretto di C sfruttando mini-porzioni di A e B caricate in L1.

Impatto sulle metriche: I profiler hardware dimostrano che passando dall'implementazione ingenua (`O0 naive`) all'implementazione a blocchi (`O3 tailed`), i miss della cache L1 crollano vicino allo zero, l'IPC (Instructions Per Cycle) si avvicina al massimo teorico dell'architettura e il tempo di esecuzione scala perfettamente senza subire crolli all'aumentare di N .

Capitolo 5

Parallelismo e Scalabilità

Definiamo il parallelismo come l'**esecuzione simultanea** da parte di più unità di lavoro al fine di **ridurre il tempo di esecuzione o aumentare la dimensione del problema risolvibile**. Facciamo una distinzione tra esecuzione in **contemporanea (concurrency)** ed esecuzione in **parallelo**:

- **Concurrency**: La concurrency è la capacità di un sistema di gestire più compiti (task) nello stesso intervallo di tempo, ma **non necessariamente nello stesso istante**.
- **Parallelism**: Il parallelismo si verifica quando più compiti vengono eseguiti fisicamente nello stesso istante.

Adesso che abbiamo parlato dell'ottimizzazione, ci rendiamo facilmente conto di quanto il parallelismo sia ormai fondamentale nelle architetture odierne. Senza questo strumento sarebbe impossibile risolvere problemi complessi a cui lavoriamo oggi e, nonostante questo strumento, siamo ancora molto lontani dall'avere le prestazioni che vorremmo.

5.1 Modelli di parallelismo

Parliamo dunque nel dettaglio di **modelli di parallelismo**. Questi definiscono:

- Come il lavoro viene suddiviso;
- Come le unità di elaborazione interagiscono tra loro;
- Il modello di memoria da adottare;
- I costi di **comunicazione, sincronizzazione e gestione dei dati**.

Il modello ha, dunque, un'influenza riguardo al come dobbiamo progettare il nostro codice e di come questo possa scalare. Possiamo dividere il parallelismo sulla base di **come si suddivide il lavoro**:

- **Data Parallelism**: La stessa operazione è applicata a porzioni o **chunk** di dati diversi. Questo tipo di parallelismo è dominante in HPC scientifico.
- **Task Parallelism**: Compiti diversi vengono eseguiti in parallelo. Tipico in applicazioni eterogenee.

O sulla base di come è organizzata la memoria:

- **Shared memory**: Secondo questa organizzazione abbiamo uno **spazio di indirizzamento unico**. Abbiamo quindi che i thread/processi guardano tutti la stessa porzione di memoria e, dunque, comunicano **implicitamente** proprio guardando la memoria. Questo modello di memoria è generalmente adottato sui singoli nodi poiché non è possibile collegare più di un certo numero di nodi alla stessa memoria per problemi di sovraffollamento e controlli di coerenza tra di essi.
- **Distributed memory**: Ogni processo ha una memoria **privata** che, quindi, nel caso in cui avesse bisogno di comunicare con un altro processo, dovrà farlo mediante messaggi **espliciti**. Questo modello è generalmente usato nei cluster così che ogni nodo del cluster abbia la sua memoria dato che questi sono indipendenti gli uni dagli altri, ciò è molto più scalabile.
- **Ibrido**: I sistemi HPC moderni sono spesso **ibridi** per cercare di prendere il meglio dall'uno e dall'altro.

Modelli a Memoria Condivisa

Come già detto, nel modello a memoria condivisa, tutte le unità di esecuzione condividono lo spazio degli indirizzi. Le variabili sono dunque visibili da tutte le unità di esecuzione e la comunicazione avviene implicitamente tramite l'accesso a memoria condivisa.

Questo modello ha il vantaggio di avere una comunicazione intra-nodo estremamente rapida al costo, però, di dover **gestire i meccanismi di sincronizzazione** dovuti ai problemi di race condition ecc.

Modelli a Memoria Distribuita

Nei modelli a memoria distribuita ogni processo ha la sua memoria privata e i dati che si scambiano i vari processi devono essere scambiati esplicitamente mediante messaggi o scambi di dati. Questo garantisce un **maggiore controllo sulla distribuzione dei dati** a patto di avere una maggiore complessità riguardo alla programmazione degli stessi.

Questo modello è **altamente scalabile** e adatto a **sistemi multi-nodo**.

5.2 Scalabilità

La scalabilità è la capacità di un sistema parallelo di migliorare le prestazioni al crescere delle risorse utilizzate.

Distinguiamo tra **Strong Scaling** e **Weak Scaling**:

- **Strong Scaling**: Questo scenario suppone di avere un problema di dimensione fissa e che, all'aumentare del numero di processi di lavoro, si misura una riduzione del tempo di risoluzione del problema.
- **Weak Scaling**: Qui, invece, abbiamo uno scenario con un problema che cresce all'aumentare dei processi. In questo caso, il nostro obiettivo sarà quello non di andare a diminuire il tempo di risoluzione del problema, bensì quello di farlo rimanere costante all'aumentare della dimensione del problema.

Sia $T(1)$ il tempo di risoluzione con un processo e $T(p)$ il tempo di risoluzione con p processi, definiamo:

- **Speedup**: Come il rapporto tra il tempo di esecuzione con un singolo processo e il tempo con p processi, cioè:

$$S(p) = \frac{T(1)}{T(p)}$$

Questo va a misurare quanto il nostro problema migliora in termini di tempo all'aumentare dei processi che lo elaborano. Idealmente, vorremmo che $S(p) = p$.

- **Efficienza**: Questa è definita come il rapporto tra lo Speedup e p , cioè:

$$E(p) = \frac{S(p)}{p}$$

Questo va a misurare quanto il nostro problema scala bene all'aumentare dei processi che lo elaborano. Idealmente, vorremmo che $E(p) = 1$.

In realtà, per quanto vorremmo che lo Speedup sia lineare, non è così poiché:

- **Parte Seriale:** Una parte del programma potrebbe non essere parallelizzabile e, dunque, l'aumentare i processi non è una soluzione.
- **Comunicazione:** Inevitabilmente, a causa dell'uso di molti processi, questi hanno la necessità di comunicare e, ciò, introduce una latenza.
- **Sincronizzazione:** Poiché lavoriamo con sezioni critiche ecc. abbiamo la necessità di introdurre delle attese.
- **Overhead di gestione:** Dovuto allo scheduling o alla creazione dei thread/processi.
- **Load imbalance:** Cioè alcuni processi terminano prima di altri.

Legge di Amdahl

Sia α la frazione parallelizzabile e $1 - \alpha$ la parte seriale. Il tempo di esecuzione con p processi è:

$$T(p) = T(1) * [(1 - \alpha) + \frac{\alpha}{p}]$$

Lo Speedup è, invece:

$$S(p) = \frac{1}{(1 - \alpha) + \frac{\alpha}{p}}$$

Il limite teorico per p che tende a infinito è:

$$\lim_{p \rightarrow \infty} S(p) = \frac{1}{1 - \alpha}$$

Questo dipende dal fatto che, anche con infiniti processi, la parte seriale rimane un limite non superabile.

Legge di Gustafson

La legge di Gustafson è applicabile nel caso di **weak scaling** in cui il problema cresce con il numero di processi ma la parte seriale rimane costante. Quella che, dunque, non sarà costante, sarà proprio la parte parallelizzabile che crescerà con p . Lo Speedup diventerà:

$$S(p) = p - (1 - \alpha)(p - 1)$$

Se α fosse vicino a 1, cioè la parte parallelizzabile fosse quasi la totalità del programma, allora lo Speedup crescerebbe quasi linearmente. Per questa legge non abbiamo un limite come nel caso della legge di Amdahl.

Abbiamo un vantaggio nell'aggiungere processi quando:

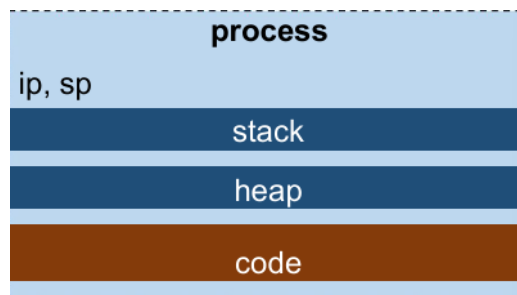
- La parte seriale è minima.
- Il problema è sufficientemente grande a tal punto che il calcolo domina la comunicazione.
- Il carico è ben bilanciato.

Capitolo 6

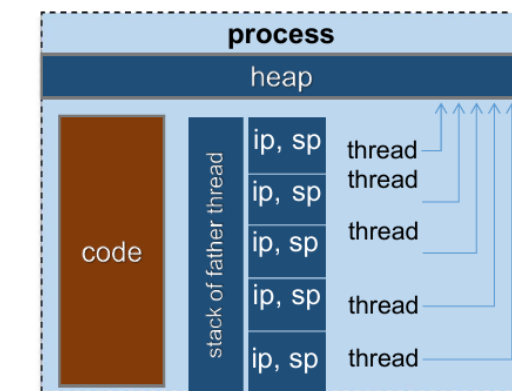
OpenMP

Definiamo un **processo** come una sequenza indipendente di istruzioni e l'insieme di risorse necessarie alla sua esecuzione. Questi, infatti, devono accedere a uno spazio di memoria protetto e a delle risorse del sistema.

Possono esistere più processi in esecuzione che eseguono lo stesso programma in maniera del tutto indipendente che possiedono, quindi, diverse risorse del sistema indipendenti li uni dalle altre.



Al contrario, un **thread** è un'istanza indipendente di esecuzione di un codice all'**interno di un processo**. Possono esserci, dunque, più thread in esecuzione sullo stesso processo. Ognuno di questi, condivide lo stesso codice, lo stesso spazio di indirizzamento e le stesse risorse del processo padre. Siamo in un modello a **memoria condivisa** in cui ogni unità di elaborazione (i thread) condivide la stessa memoria ma con un piccolo spazio **privato**, come in un'architettura NUMA.



In generale, generare thread all'interno di un processo è molto meno costoso che creare processi.

A seconda dell'architettura hardware, si utilizzano paradigmi software differenti.

- **Shared-Memory (es. OpenMP):** Un singolo processo univoco genera un certo numero di thread. Poiché esiste uno spazio di memoria univoco (l'Heap del processo), la comunicazione tra thread avviene in modo *implicito* leggendo e scrivendo le stesse variabili in memoria.
- **Distributed-Memory (es. MPI):** Vengono creati N processi distinti, ognuno con la propria copia del codice e il proprio spazio di memoria privato e isolato. Un processo non può fisicamente accedere alla memoria di un altro. La comunicazione deve avvenire obbligatoriamente in modo *esplicito* tramite lo scambio di messaggi sulla rete.

6.1 Cosa è OpenMP

OpenMP (Open specifications for MultiProcessing) è un'API standard per la programmazione parallela a memoria condivisa. Consente di scrivere programmi multithread con un comportamento standard attraverso l'utilizzo di un set di direttive del compilatore da inserire nel codice sorgente. OpenMP permette di parallelizzare il codice sia suddividendo automaticamente piccoli lavori (come le iterazioni di un ciclo), sia assegnando esplicitamente compiti più grandi ai singoli thread.

OpenMP è composto da 3 componenti:

1. **Direttive del Compilatore:** In C/C++ assumono la forma di `#pragma` e forniscono indicazioni su come gestire i thread.

2. **Librerie di Runtime:** Funzioni richiamabili dal codice (es. per sapere quanti thread stanno lavorando).
3. **Variabili d'Ambiente:** Parametri impostati dall'utente a livello di sistema operativo (es. per definire quanti thread generare o per gestire l'affinità core-memoria).

I vantaggi di un approccio basato sulle direttive sono:

- **Astrazione:** Le sottigliezze di pthread e gli aspetti specifici dell'hardware sono nascosti. È possibile concentrarsi sui dati e sul flusso di lavoro molto più facilmente.
- **Efficienza:** La curva di apprendimento per ottenere risultati ragionevoli è molto più semplice. La progettazione del codice è più semplice, il rapporto risultato/sforzo è favorevole rispetto a pthread.
- **Approccio incrementale:** Non è necessario riscrivere l'intero codice. Si inizia concentrandosi solo su alcune sezioni, seguendo i suggerimenti del profiling.
- **Portabilità:** Il compilatore si occuperà di questo per noi. È comunque necessario sviluppare una progettazione in grado di adattarsi a diverse topologie.
- **Un'unica fonte:** Attraverso la compilazione condizionale, le versioni seriali e parallele possono coesistere facilmente.

6.2 Modello Fork-Join

OpenMP basa il suo funzionamento sul modello **Fork-Join**. Il programma inizia l'esecuzione come un normale programma sequenziale guidato da un **singolo thread master**.

- **Fork:** Quando il thread master incontra una direttiva specifica (una regione parallela), genera un *team di thread* figli per eseguire il lavoro in parallelo. Il master sospende la sua attività singola e si unisce al pool lavorando come "thread 0".
- **Join:** Al termine della regione parallela, i thread si sincronizzano. I thread figli terminano (o si mettono in pausa) e il solo thread master riprende l'esecuzione seriale del codice.

Il modello offre flessibilità nella gestione delle risorse, ma presenta vincoli stringenti riguardo alla struttura del codice:

- **Dipendenze nei Loop:** Le dipendenze "trasportate" dai cicli (quando un'iterazione ha bisogno del risultato della precedente) rappresentano il principale ostacolo all'accelerazione parallela.
- **Parallelismo Annidato (Nested):** È esplicitamente consentito creare regioni parallele all'interno di altre regioni parallele.
- **Modifica Dinamica:** Il numero di thread non è necessariamente fisso ma può essere modificato dinamicamente prima di entrare in una nuova regione parallela.

6.2.1 Direttive

Le direttive OpenMP si applicano a **blocchi di codice strutturati**, ovvero blocchi che hanno un singolo punto di ingresso e un singolo punto di uscita, senza istruzioni di salto (branch) incontrollati al loro interno. La direttiva principale per avviare il parallelismo è:

```
#pragma omp parallel{  
    // Codice eseguito in parallelo da tutti i thread del team  
}
```

Queste direttive permettono di:

- Creare team di Thread per eseguire in parallelo.
- Gestire il carico di lavoro tra i thread.
- Specificare quali sono le parti di memoria condivisa e quali no.
- Gestire l'aggiornamento delle regioni condivise.
- Sincronizzare thread e determinare operazioni atomiche e/o esclusive.

Possiamo fare riferimento a due cose in particolare:

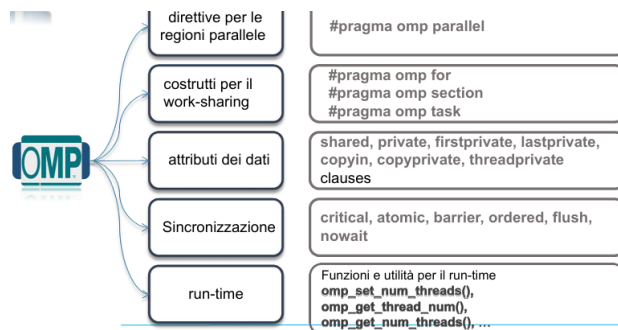
- **Static Extent:** È l'ambito lessicale del blocco strutturato. Riguarda solo le righe di codice scritte fisicamente e visivamente all'interno delle parentesi graffe subito dopo il `#pragma`.

- **Dynamic Extent:** Se all'interno della regione parallela viene chiamata una funzione esterna, anche il codice di quella funzione verrà eseguito in parallelo da tutti i thread. L'estensione dinamica include l'estensione statica originale *più* tutte le istruzioni e le ulteriori chiamate lungo l'albero delle chiamate.

Le funzioni chiamate nell'estensione dinamica possono contenere direttive OpenMP aggiuntive dette **direttive orfane**.

Uno dei grandi vantaggi di OpenMP è l'approccio incrementale. Non dobbiamo riscrivere tutto il codice, ma possiamo aggiungere gradualmente i pragma nei colli di bottiglia. Inoltre, lo stesso identico codice sorgente può essere compilato sia in modalità parallela che in modalità puramente sequenziale ignorando i pragma.

Quando al compilatore viene passata l'istruzione di attivare OpenMP (es. tramite il flag `-fopenmp` in GCC), esso definisce automaticamente una macro di sistema chiamata `_OPENMP`.



6.3 Regioni Parallele

Come abbiamo visto, OpenMP utilizza il modello fork-join. Il lavoro parallelo si svolge all'interno di **regioni parallele**, che in C/C++ vengono definite applicando una direttiva a un blocco di codice strutturato.

Quando il compilatore incontra la direttiva `#pragma omp parallel`, crea un pool di thread. Ognuno di essi riceve un ID univoco (da 0 a $N - 1$): **ogni thread riceve un proprio Stack privato e un proprio Instruction Pointer (IP)** separati da quelli del processo padre. Avranno tutti lo stesso **process id (PID)** ma diverso **Thread ID (TID)**. Le direttive in generale in C/C++ sono di questo tipo:

```
#pragma omp <directive> [<clause> [ <clause> [...]]]
```

Ci sono tanti tipi di costrutti omp, tra cui:

- `#pragma omp parallel _some_clauses_here_ :` Una regione parallela può essere una linea singola.
- `#pragma omp parallel _some_clauses_here_ { ... } :` Nessun limite sulla dimensione del codice compreso tra `{...}`.
- `#pragma omp parallel for _some_clauses_here_ { ... } :` Un costruttore specifico per i cicli `for`.
- `#pragma omp sections _some_clauses_here_ { ... } :` Un costrutto più generale per il *work-sharing*.
- `#pragma omp task _some_clauses_here_ { ... } :` Questo permette un parallelismo basato sui task.

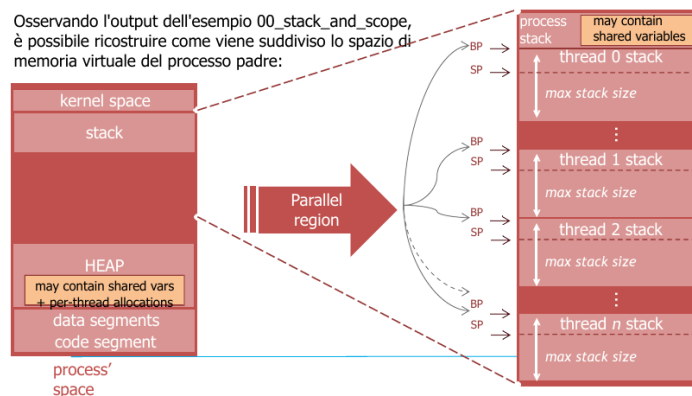
Questi li vedremo più avanti nel dettaglio.

La creazione dello stack per ogni thread non è "gratuita". L'hardware deve allocare fisicamente porzioni di memoria virtuale per ospitare i dati privati di ciascun thread. La dimensione predefinita di questo stack varia a seconda del compilatore (es. ~ 4 MB con GCC, fino a 132 MB con Clang).

Questo genera due scenari di rischio architetturale:

- **Out of Memory (OOM):** Se generi troppi thread o se la dimensione dello stack è troppo grande, potresti esaurire la memoria fisica disponibile nel sistema.
- **Segmentation Fault:** Al contrario, se lo stack predefinito è troppo piccolo e il tuo thread tenta di allocare grossi array locali, sfonderà il limite dello stack causando un errore di segmentazione.

Soluzione operativa: L'utente può controllare esplicitamente questa dimensione esportando la variabile d'ambiente `OMP_STACKSIZE` (es. `export OMP_STACKSIZE=1M`) prima di eseguire il codice.



Di norma, inoltre, il numero di thread che possono essere generati da un processo è limitato dal sistema. Ad esempio, Linux imposta che il numero massimo di thread è la quantità di memoria fisica divisa per la quantità di memoria necessaria per descrivere ed eseguire un thread per un fattore che impedisce che la memoria sia tutta usata per rendere attivi i thread.

Entro questo limite, possiamo modificare il comportamento di OpenMP con `OMP_THREAD_LIMIT`.

6.3.1 Gestione della Regione Parallela

Innanzitutto, vediamo come si costruisce una classica regione parallela:

```
#pragma omp parallel
{
    int id = omp_get_thread_num();

    #pragma omp single
    {
        int nthreads = omp_get_num_threads();
    }

    #pragma omp barrier
    //forza tutti i thread ad arrivare allo stesso punto
    //il rischio è causare rallentamenti
    //per attendere la fine dei thread
    printf("Hello from thread!");
}
```

Possiamo ottenere diversi comportamenti con alcune aggiunte dopo la parola chiave *parallel*:

- **if(condizione)**: questo if permette di aprire la regione parallela solo se la condizione è vera. Utile per evitare l'overhead dovuto alla creazione di troppi thread anche nei casi di in cui vi siano pochi dati. La sezione di codice viene seguita a prescindere ma, nel caso in cui la condizione sia falsa, allora non verrebbero creati thread.
- **num_threads(n)**: questa condizione forza il processo a creare esattamente n thread ignorando le eventuali variabili di ambiente impostate.
- **proc.bind**: Serve a dare direttive sul dove far lavorare il thread in termini di core fisici. Senza questa direttiva il sistema operativo potrebbe decidere di spostarli come più gli aggrada, creando un overhead causato dallo spreco di cache.

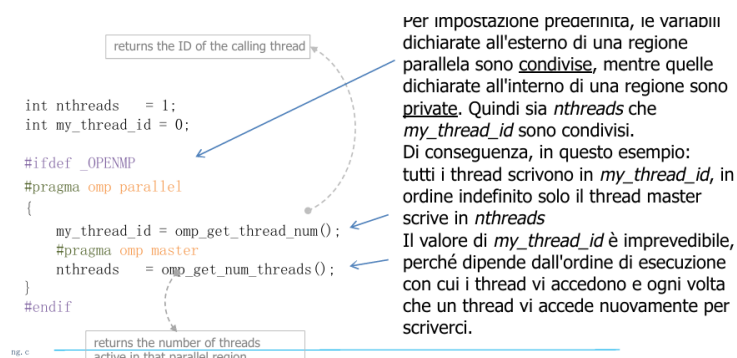
Variabili Condivise o Private

Il cuore di OpenMP è definire correttamente quali variabili risiedono nell'Heap condiviso e quali negli Stack privati.

Possiamo trovare, quindi due tipi di variabili:

- **Shared:** È tutto ciò che è definito nella regione seriale che viene ereditato come condiviso all'interno della regione parallela. Tutti i thread possono dunque leggerle e scriverle.
- **Private:** La clausola `private(x)` ordina al compilatore di allocare un nuovo blocco di memoria **non inizializzato** nello stack di ogni thread. Sebbene si chiami `x` come la variabile originale, punta a un indirizzo fisico completamente diverso e all'inizio non è inizializzata, rischiando dunque di generare errori nel caso non vengano inizializzate. In realtà, l'utilizzo di `private`, proprio a causa di questi problemi è **fortemente sconsigliata** e si predilige la dichiarazione all'interno della regione condivisa stessa così da rendere la cosa più chiara ed evitando errori di distrazione.

Un esempio può essere questo:



Visto che le variabili `private` "nascono vuote" (non inizializzate), OpenMP fornisce strumenti per copiare i valori da e verso l'esterno:

- **firstprivate(x):** La variabile è privata (ogni thread ha la sua copia in stack), ma il suo valore di partenza non è casuale: viene **inizializzato al valore che la variabile `x` originale aveva** prima di entrare nella regione parallela.
- **lastprivate(x):** Ha senso solo nei cicli `for` parallelizzati. La variabile originale (nello stack del thread master) prenderà il valore che la sua copia privata ha calcolato durante **l'ultima iterazione logica** del ciclo.

Per loro natura, le variabili private nascono e muoiono all'interno di una regione condivisa. Tuttavia, potrebbe capitare di volere che queste variabili in un qualche modo rimangano e, dunque, esistono dei costrutti anche per fare questo:

- **threadprivate(var)**: Rende una variabile globale privata per ogni thread. A differenza del normale `private`, questa variabile **sopravvive tra diverse regioni parallele** consecutive (il thread non perde il dato). Nota: non si può usare se il numero di thread cambia dinamicamente.
- **copyin(var)**: Usata insieme a `threadprivate`. Serve per far sì che, all'ingresso di una regione parallela, il thread master copi il valore della sua variabile `threadprivate` dentro le copie di tutti gli altri thread.
- **copyprivate(var)**: Si applica ai costrutti eseguiti da un singolo thread (es. direttiva `single`). Propaga il risultato calcolato da quel singolo thread verso le copie private di tutti gli altri thread del team, prima che attraversino la barriera finale.

6.3.2 Controllo del Numero di Thread

Per impostazione predefinita, il numero di thread generati in una regione parallela corrisponde al numero di core disponibili. Tuttavia, è possibile sovrascrivere questo comportamento in tre modi:

1. **Variabile di ambiente:** `export OMP_NUM_THREADS=n`
2. **Funzione a runtime:** `omp_set_num_threads(n)`
3. **Clausola della direttiva:** `#pragma omp parallel num_threads(n)`

La variabile d'ambiente `OMP_DYNAMIC` può comportarsi in due modi:

- **TRUE:** Se la variabile è impostata a `True` noi diamo le direttive al sistema operativo, tuttavia, potrebbe scegliere di testa sua se darci un numero n di thread o inferiore. Questo permette a noi di dare delle direttive ma ci evita problemi nel caso in cui non sia possibile ottenere quel numero di thread evitando crash.
- **FALSE:** Se questa variabile è impostata a `False`, le direttive che diamo saranno **assolute** e il nostro sistema operativo creerà gli n thread che abbiamo questo. Questo ci dà il vantaggio di avere un totale controllo

sul numero di thread che vengono creati, tuttavia, il sistema operativo non interverrà nemmeno quando, in realtà, l'uso di n thread rallenta solo l'esecuzione del programma a causa della mancanza di risorse hardware.

Questa opzione si può modificare a runtime con `omp_set_dynamic(true)`.

6.3.3 Sincronizzazione dell'Esecuzione

In una regione parallela, potremmo trovarci in varie situazioni. Potremmo volere che alcuni blocchi di codice devono essere **eseguiti da un solo thread alla volta** in quanto intrinsecamente seriali o potremmo volere che sia eseguito **da un solo thread in assoluto** perché intrinsecamente univoci.

OpenMP gestisce queste casistiche con costrutti specifici, che differiscono per l'uso delle **barriere implicite** e per l'overhead. Per comprendere a fondo il comportamento dei costrutti di sincronizzazione, è essenziale porsi due domande chiave per ciascuno di essi:

- Chi esegue il blocco di codice?
- Gli altri thread lo aspettano alla fine?

Vediamoli adesso uno per uno:

- **#pragma omp critical**
 - *Chi esegue il codice?* **Tutti i thread**, ma rigorosamente **uno alla volta**.
 - *Ingresso (Coda):* Nel punto di ingresso è presente una "barriera sotto forma di coda". Se un thread è già all'interno, gli altri trovano un blocco e devono attendere il proprio turno.
 - *Uscita (Via libera):* **Nessuna barriera**. Quando un thread termina il blocco, esce e prosegue immediatamente l'esecuzione del codice successivo, liberando il posto per il prossimo thread in coda.
- **#pragma omp atomic**
 - *Chi esegue il codice?* Tutti i thread, uno alla volta.
 - *Comportamento:* La logica di sincronizzazione è identica a `critical`. Tuttavia, applicandosi a una singola istruzione hardware di aggiornamento in memoria, l'attesa in coda è quasi istantanea.
- **#pragma omp single**

- *Chi esegue il codice?* **Un solo thread** (il primo che raggiunge la direttiva).
- *Ingresso (Salto):* Gli altri thread ignorano completamente il blocco di codice.
- *Uscita (Barriera Implicita):* I thread che hanno saltato il blocco **devono aspettare** alla fine della regione finché il thread lavoratore non ha completato l'operazione. Solo allora ripartono tutti insieme.

- **#pragma omp master / masked**

- *Chi esegue il codice?* Esclusivamente il **thread 0**.
- *Ingresso (Salto):* Gli altri thread ignorano il blocco.
- *Uscita (Nessuna barriera):* A differenza di *single*, **non vi è alcuna sincronizzazione esplicita**. Mentre il master esegue il blocco, gli altri proseguono immediatamente, potendosi trovare ovunque nell'esecuzione del codice successivo.

La direttiva **atomic**

Quando più thread lavorano su variabili condivise, è fondamentale garantire la **correttezza semantica** del codice. Si consideri una semplice assegnazione come $a += b$ dove a è una variabile condivisa e b è 1.

Sebbene in C/C++ questa sembri un'unica istruzione, a livello di processore viene tradotta in **diverse istruzioni Assembly**:

1. **Load:** Il processore legge il valore attuale di a dalla RAM e lo porta in un registro.
2. **Add:** Somma il valore di b al registro.
3. **Store:** Scrive il nuovo valore dal registro di nuovo nella RAM (nella cella di a).

Sebbene una singola istruzione Assembly non possa essere interrotta, un thread potrebbe essere interrotto *tra* la fase di Load e la fase di Store. Se il Thread 1 e il Thread 2 leggono contemporaneamente $a = 5$, entrambi calcolano $5 + 1 = 6$ e scrivono 6. Il risultato finale sarà 6 invece del corretto 7. Un aggiornamento è andato perso!

La direttiva `#pragma omp atomic` "protegge" la variabile condivisa, garantendo che l'intero ciclo di lettura-modifica-scrittura avvenga senza interruzioni da parte di altri thread. Questa direttiva ha un **overhead molto**

inferiore rispetto a una generica regione `critical` poiché sfrutta istruzioni hardware del processore e blocca solo la specifica area di memoria, non il codice. Dunque, deve essere sempre preferita per le operazioni matematiche di base rispetto alla direttiva `#pragma omp critical`.

Per fornire un controllo granulare, OpenMP permette di specificare *che tipo* di operazione atomica si sta eseguendo tramite delle clausole:

- **read**: Forza una lettura atomica della variabile in memoria.

```
#pragma omp atomic read
var = mem;
```

- **write**: Forza una scrittura atomica in una locazione di memoria.

```
#pragma omp atomic write
mem = var;
```

- **update** (*comportamento di default se omissso*): Esegue un aggiornamento atomico leggendo, modificando e riscrivendo il dato.

```
#pragma omp atomic update
mem++; // oppure mem += expr;
```

- **capture**: Esegue l'aggiornamento atomico (`update`) e contemporaneamente "cattura" in una variabile privata il valore originale (prima della modifica) o il valore finale (dopo la modifica), garantendo che la lettura e l'aggiornamento avvengano nello stesso istante indivisibile.

```
#pragma omp atomic capture
// val ottiene il valore VECCHIO, mem viene incrementato
val = mem++;
```

```
#pragma omp atomic capture
// val ottiene il valore NUOVO, mem viene incrementato
val = ++mem;
```

Questa funzione risulta utile quando si vuole fare un aggiornamento atomico ma si ha anche bisogno di conoscere il valore prima o dopo l'aggiornamento, evitando così la necessità di una regione `critical` più pesante.

Ottimizzazione Avanzata dei Costrutti di Sincronizzazione

- **Il problema del Lock Globale in `critical`:** Quando scriviamo semplicemente `pragma omp critical`, il compilatore associa a quel blocco di codice un "lucchetto" (lock) globale predefinito. Supponendo di avere due diverse regioni critiche nel codice, entrambe useranno lo stesso lock globale. Se un thread è all'interno di una regione critica, tutti gli altri thread che vogliono entrare in qualsiasi regione critica (anche se sono regioni diverse) devono aspettare. Questo causa una serializzazione inutile del codice che limitano le prestazioni del sistema. La soluzione è dare un nome alla regione tramite `pragma omp critical (nome)`. In questo modo OpenMP crea lock indipendenti.
- **Interazione tra `critical` e `atomic`:** `atomic` e `critical` operano su livelli di astrazione completamente diversi: `critical` protegge una regione di codice (impedisce che due thread leggano contemporaneamente le stesse righe di codice) mentre `Atomic` protegge una locazione di memoria fisica (impedisce che due thread scrivano nello stesso indirizzo RAM contemporaneamente). Queste due strutture non sono mutualmente esclusive, immaginiamo di avere due regioni critiche diverse che accedono alla stessa variabile. Poiché la variabile a cui accedono è la stessa, pur essendo in regioni diverse, si rischiano delle race condition. In questo caso, è necessario proteggere la variabile condivisa con una direttiva `atomic` all'interno di entrambe le regioni critiche. In questo modo, anche se i thread accedono a regioni critiche diverse, la variabile condivisa sarà comunque protetta da accessi concorrenti.
- **Scelta prestazionale: `single` vs `master`:** Entrambi i costrutti fanno eseguire un blocco di codice a un solo thread. Tuttavia, l'impatto sulle prestazioni (overhead) è molto diverso:
 - **Master:** Il thread master (thread 0) esegue il blocco, mentre gli altri thread ignorano completamente il blocco e proseguono immediatamente. Non c'è alcuna barriera implicita alla fine del blocco, quindi gli altri thread non devono aspettare. L'overhead è quasi zero, poiché basta un semplice controllo `if (thread_id == 0)` per far eseguire il blocco solo al thread master.
 - **Single:** Il primo thread che raggiunge la direttiva esegue il blocco, mentre gli altri thread ignorano il blocco o devono aspettare alla fine della regione parallela. L'overhead è maggiore poiché la logica di coordinamento per determinare quale thread arriva per primo

e gestire la sincronizzazione alla fine del blocco introduce un overhead significativo, soprattutto se i thread arrivano in momenti diversi.

La scelta dipende dal Load Balancing del codice che precede questa direttiva. Se i calcoli precedenti sono sbilanciati (alcuni thread finiscono molto prima di altri), conviene usare `single`. In questo modo, il primo thread che si libera esegue subito il blocco, senza dover per forza aspettare il thread 0 (che magari è in ritardo perché bloccato su un calcolo pesante). Se invece il carico è perfettamente bilanciato, tutti i thread arrivano assieme, quindi è meglio usare `master` per risparmiare sull'overhead del coordinamento.

6.3.4 Ordinamento dei Thread

Di default, l'ordine di esecuzione dei thread in OpenMP è **non deterministico**. Tentare di imporre un ordine temporale rigoroso richiede particolare attenzione, poiché l'uso improprio dei costrutti di sincronizzazione può portare a stalli, codice ignorato o errori di compilazione degradando significativamente le prestazioni.

Supponiamo di avere questo pezzo di codice:

```
int order = 0;
#pragma omp parallel{
    int myid = omp_get_thread_num();
    printf( "greetings from thread num %d\n", my_thread_id );
}
```

Chiaramente, non possiamo garantire l'ordine di stampa dei messaggi.

Tentativo 1: Variabile Order

Se volessimo forzare un ordine di esecuzione, potremmo pensare di usare una variabile condivisa `order` e far sì che ogni thread aspetti il proprio turno:

```
int order = 0;
#pragma omp parallel{
    int myid = omp_get_thread_num();
    #pragma omp critical if ( order == myid ) {
        printf( "greetings from thread num %d\n", my_thread_id );
        order++;
    }
}
```

Tuttavia, la direttiva `CRITICAL` definisce una regione che viene eseguita da tutti i thread, ma da un singolo thread alla volta. Il che potrebbe sembrare un "ordinamento", tuttavia, sebbene i thread si accodino all'inizio della regione, non viene imposto alcun ordine particolare a tale coda. Di conseguenza, il risultato è imprevedibile a parte il fatto che il thread 0 stamperà il messaggio e aumenterà l'ordine. Se il thread 2 viene messo in coda subito dopo, eseguirà la valutazione `if`, che fallirà (l'ordine sarebbe uguale a 1) e il thread 2 non stamperà affatto il messaggio.

Tentativo 2: Ciclo While con Order

Un altro tentativo potrebbe essere il seguente:

```
int order = 0;
#pragma omp parallel{
    int myid = omp_get_thread_num(); int done = 0
    while (!done) {
        #pragma omp critical if (order == myid ) {
            printf( "greetings from thread num %d\n", my_thread_id);
            order++; done=1;
        }
    }
}
```

In questo caso, ogni thread entra in un ciclo infinito in cui tenta di acquisire la regione critica. Se il thread riesce ad acquisire la regione critica e l'ordine è uguale al suo ID, stampa il messaggio, incrementa l'ordine e imposta `done = 1` per uscire dal ciclo. Tuttavia, se un thread non riesce ad acquisire la regione critica o se l'ordine non corrisponde al suo ID, continua a tentare indefinitamente. Questo può portare a stalli (deadlock) se i thread si bloccano reciprocamente aspettando che gli altri rilascino la regione critica, o a un consumo eccessivo di CPU se i thread continuano a tentare di acquisire la regione critica senza successo. Inoltre, se il numero di thread è elevato, potrebbe verificarsi una situazione in cui alcuni thread non riescono mai ad acquisire la regione critica, causando un comportamento imprevedibile e potenzialmente degradante per le prestazioni del sistema.

Tentativo 3: Costrutti nativi

La via corretta per forzare l'ordinamento in OpenMP è utilizzare i costrutti nativi pensati per questo scopo: associare una direttiva `ordered` a un ciclo `for` parallelizzato.

```

int nthreads = 1;
#pragma omp parallel{
    int myid = omp_get_thread_num();
    #pragma omp master
        int nthreads = omp_get_num_threads();
    #pragma omp barrier

    #pragma omp for ordered
    for ( int i = 0; i < nthreads; i++ )
        #pragma omp ordered
            printf( "greetings from thread num %d\n", my_thread_id );
}

```

Senza la direttiva `barrier`, potrebbe succedere che alcuni thread (potenzialmente tutti) arrivino al costrutto di condivisione del lavoro con un valore errato di `nthreads`. Per quanto riguarda la sintassi, la direttiva `omp for` dice a OpenMP di parallelizzare il ciclo `for` (l'ordine è solitamente dato con `i`-esima iterazione eseguita dall'`i`-esimo thread), mentre la clausola `ordered` associata al ciclo `for` indica che le iterazioni del ciclo devono essere eseguite in un ordine specifico che sarà meglio specificato dentro il blocco di codice. Infatti, all'interno del ciclo `for`, la direttiva `omp ordered` indica che il blocco di codice associato a questa direttiva deve essere eseguito in un ordine specifico (in questo caso, l'ordine dei thread).

6.3.5 Assegnazione del lavoro

Quello che vogliamo fare adesso, è assegnare compiti diversi a thread diversi. Il più delle volte vogliamo che:

- **Tutti i thread eseguano lo stesso codice** ma con dati diversi (data parallelism).
- **Tutti i thread eseguano codice diverso** sugli stessi dati (task parallelism).

Banalmente, quello che potremmo fare è scrivere un blocco di codice assegnando manualmente a ogni thread un compito diverso tramite un controllo `if` sul thread ID.

```

#pragma omp parallel{
    if( myid % 3 == 0 )
        result = heavy_work_0( );
}

```

```
    else if ( myid % 3 == 1 )
        result = heavy_work_1( );
    else if ( myid % 3 == 2 )
        result = heavy_work_2( );

    if ( myid < 3 )
        results[myid] = result;
}
```

Tuttavia, questo approccio è poco scalabile e non sfrutta appieno le potenzialità di OpenMP. È anche possibile usare regioni parallele solo al verificarsi di certe condizioni tramite la clausola `if`:

```
#pragma omp parallel if(amount_of_work > threshold){
    if( myid % 3 == 0)
        result = heavy_work_0( );
    else if ( myid % 3 == 1 )
        result = heavy_work_1( );
    else if ( myid % 3 == 2 )
        result = heavy_work_2( );

    if ( myid < 3 )
        results[myid] = result;
}
```

In questo modo, se la quantità di lavoro è inferiore a una certa soglia, il codice verrà eseguito in modalità seriale, evitando l'overhead della creazione dei thread e della sincronizzazione. Se invece la quantità di lavoro supera la soglia, il codice verrà eseguito in modalità parallela, sfruttando le potenzialità di OpenMP per migliorare le prestazioni. Naturalmente, questo approccio è più flessibile e permette di adattare dinamicamente l'esecuzione del codice in base alla quantità di lavoro da svolgere, ottimizzando così le prestazioni complessive del programma a patto che la soglia sia scelta in maniera tale che il beneficio della parallelizzazione superi l'overhead dovuto alla creazione dei thread.

È anche possibile effettuare parallelismo annidato, ovvero creare regioni parallele all'interno di altre regioni parallele. In questo modo, è possibile sfruttare ulteriormente le potenzialità di OpenMP per migliorare le prestazioni del programma. Tuttavia, è importante tenere presente che il parallelismo annidato può introdurre un overhead significativo a causa della creazione di thread aggiuntivi e della sincronizzazione necessaria tra i thread, quindi è

consigliabile utilizzarlo solo quando si è certi che porterà a un miglioramento delle prestazioni complessive del programma.

A seguire, vediamo alcune funzioni di runtime che permettono di ottenere informazioni sui thread e di controllare il comportamento del sistema:

- **int omp_get_num_threads():** Ottiene il numero di thread nella regione parallela da cui viene chiamato.
- **int omp_get_thread_num():** Ottiene l'ID del thread chiamante.
- **void omp_set_num_threads(int):** Imposta il numero di thread da quel punto in poi.
- **int omp_get_max_threads():** Ottiene il numero massimo di thread nella regione parallela successiva, senza una clausola num_threads.
- **double omp_get_wtime():** Ottiene il tempo in secondi trascorso da un punto nel passato (non sincronizzato tra i thread).

Le ICV sono variabili concettuali utilizzate per configurare il runtime di OpenMP. Esse operano su tre livelli di ambito:

- **Ambito Globale:** Si applica all'intero programma ed è configurato una sola volta all'avvio.
- **Ambito del Dispositivo:** Gestisce le differenze di configurazione su piattaforme eterogenee (CPU, GPU, coprocessori) a partire da OpenMP 4.0.
- **Ambito del Task:** La maggior parte delle ICV è locale al task. Alla creazione di un nuovo task, le ICV vengono ereditate dal genitore; eventuali modifiche effettuate dal task figlio hanno effetto solo sulla sua durata e non si propagano al task chiamante.

6.4 Basi della Parallelizzazione dei Cicli con OpenMP

Il costrutto principale per parallelizzare i cicli in C/C++ è la direttiva `#pragma omp parallel for`. Questa istruzione indica al compilatore di suddividere le iterazioni del ciclo tra i thread disponibili.

Durante la parallelizzazione, è fondamentale definire l'ambito (*scope*) delle variabili per evitare conflitti di memoria:

- **shared(...)**: Le variabili elencate sono condivise tra tutti i thread. Ogni thread accede alla stessa locazione di memoria.
- **private(...)**: Ogni thread riceve una copia locale e privata della variabile. Queste copie hanno locazioni di memoria distinte e "muoiono" al termine della regione parallela.

`implicit(none)` (o `default(none)`) è una direttiva di sicurezza caldamente raccomandata: disabilita le ipotesi implicite del compilatore sull'ambito delle variabili e obbliga il programmatore a dichiarare esplicitamente ogni variabile come `shared` o `private`, aiutando a individuare errori di logica prima dell'esecuzione.

Note sui Contatori del Ciclo

Per impostazione predefinita, il contatore di un ciclo `for` parallelizzato (es. la variabile `i`) è considerato **privato** anche se dichiarato esternamente. Tuttavia, per chiarezza e portabilità (standard C99), è preferibile dichiarare il contatore direttamente nell'intestazione del ciclo: `for (int i = 0; ...)`.

Prevenzione dei Data Race

Quando più thread tentano di aggiornare una variabile condivisa contemporaneamente (es. una somma globale), si verifica un *data race*. Per risolvere il problema mantenendo la correttezza del risultato, OpenMP offre strumenti come **#pragma omp atomic**, che protegge una singola operazione di aggiornamento della memoria.

```
#pragma omp parallel for
for ( int i = 0; i < N; i++ )
    #pragma omp atomic
    sum += a[i];
```

6.5 Assegnazione del Lavoro

In OpenMP, la clausola `schedule` specifica come le iterazioni di un ciclo `for` parallelo vengono distribuite tra i thread del pool. La sintassi generale è `schedule(tipo, chunk-size)`.

Le tipologie di scheduling sono:

- **static:** L'iterazione è suddivisa in blocchi di dimensione `chunk-size` distribuiti ai thread in ordine *round-robin* rigido. Se non specificato, i blocchi sono di dimensioni uguali. È basato su pura aritmetica e ha l'overhead minimo.
- **dynamic:** Le iterazioni sono suddivise in blocchi (`chunk-size` predefinito a 1) e distribuite ai thread che ne fanno richiesta. Segue il principio "chi prima arriva, prima viene servito". È utile per bilanciare il carico quando le iterazioni hanno costi computazionali diversi, ma introduce overhead di gestione.
- **guided:** Simile al dinamico, ma la dimensione del blocco iniziale è proporzionale al numero di iterazioni rimanenti divise per il numero di thread. La dimensione decresce progressivamente fino al valore `chunk-size`.
- **runtime:** La politica viene decisa a tempo di esecuzione tramite la variabile d'ambiente `OMP_SCHEDULE`.

6.5.1 Guida alla Scelta dello Scheduler

La scelta dello scheduling corretto è fondamentale per la scalabilità del codice.

Carico di Lavoro	Costo per Iterazione	Strategia Ideale
Costante	Sempre uguale	<code>static</code> (overhead minimo)
Crescente / Decrescente	Varia con l'indice i	<code>dynamic</code> o <code>guided</code>
Casuale / Random	Imprevedibile	<code>dynamic</code> o <code>guided</code>

Tabella 6.1: Confronto tra le politiche di scheduling.

Regola pratica: Lo scheduling `static` è preferibile quando il lavoro computazionale è prevedibile e uniforme, poiché riduce al minimo i tempi di sincronizzazione. Lo scheduling `dynamic` è più efficace quando il tempo richiesto da ogni iterazione è incostante o imprevedibile.

6.6 Clausole Avanzate nei For Paralleli

firstprivate e lastprivate

La clausola `private` standard non inizializza le variabili e non ne salva il valore alla fine. Per ovviare a questo, si utilizzano:

- **firstprivate(list)**: Le variabili elencate sono private, ma vengono inizializzate con il valore che avevano immediatamente prima di entrare nella regione parallela.

```
int offset = 10;
// Ogni thread inizia con una copia privata di "offset" pari a 10
#pragma omp parallel for firstprivate(offset)
for(int i = 0; i < N; i++) {
    a[i] = i + offset;
}
```

- **lastprivate(list)**: Le variabili condivise originali prenderanno il valore calcolato dall'ultimo thread che completa l'iterazione logicamente finale del ciclo.

```
int final_mark;
// "final_mark" manterra' il valore assegnato durante l'ultima it
#pragma omp parallel for lastprivate(final_mark)
for(int i = 0; i < N; i++) {
    final_mark = compute_mark(i);
}
// Qui final_mark ha il valore dell'iterazione i = N-1
```

6.6.1 La clausola **reduction**

Utilizzata per risolvere in modo sicuro e scalabile le operazioni di riduzione (come calcolare una somma totale) evitando *data race*. Ad ogni thread viene assegnata una copia privata temporanea inizializzata con un valore logico neutro (es. 0 per la somma, 1 per la moltiplicazione). Al termine del costrutto, OpenMP combina tutti i risultati parziali nella variabile globale. Operatori supportati includono +, *, -, max, min, &, |, ^.

```
double sum = 0.0;
// Somma sicura senza bisogno di direttive "atomic"
#pragma omp parallel for reduction(+: sum)
for (int i = 0; i < N; i++) {
    sum += a[i];
}
```

6.6.2 La clausola **collapse**

Abilita la parallelizzazione su più livelli di ciclo. Questo fonde n cicli `for` in un unico grande spazio di iterazioni da distribuire ai thread. I cicli devono essere "perfettamente annidati", ovvero senza altre istruzioni intermedie tra di essi.

```
// Fonde i due cicli for per esporre un maggiore livello di parallelismo
#pragma omp parallel for collapse(2)
for (int ii = 0; ii < Nrows; ii++) {
    for (int jj = 0; jj < Ncol; jj++) {
        C[ii][jj] = A[ii][jj] + B[ii][jj];
    }
}
```

6.6.3 La clausola **nowait**

Di default, OpenMP inserisce una barriera di sincronizzazione implicita alla fine di un costrutto di condivisione del lavoro (come un ciclo `for`). La clausola `nowait` ignora questa barriera, permettendo ai thread che hanno terminato le loro iterazioni di proseguire immediatamente con le istruzioni successive.

```
#pragma omp parallel
{
    // I thread che finiscono questo ciclo passano subito al successivo
    #pragma omp for nowait
    for(int i = 0; i < N; i++) {
        a[i] = elabora(i);
    }

    #pragma omp for
    for(int i = 0; i < N; i++) {
        b[i] = elabora_ancora(i);
    }
}
```

6.7 Anatomia di un Data Race

Un *data race* è uno degli errori più comuni e insidiosi nella programmazione concorrente. Genera un comportamento non deterministico, portando a risultati errati che cambiano da un'esecuzione all'altra. Si verifica quando:

- Due o più thread accedono simultaneamente alla stessa locazione di memoria.
- Almeno uno di questi accessi è un'operazione di scrittura (modifica).
- Gli accessi non sono protetti da meccanismi di sincronizzazione (come lock, barriere o direttive `atomic/critical`).

Spesso, i data race sono causati dalla dimenticanza di clausole private o `reduction` nei cicli paralleli.

6.7.1 Il Problema del Livello Macchina (Read-Modify-Write)

Un'istruzione C che appare come una singola operazione atomica (es. `sum += a[i];`) viene in realtà tradotta dal compilatore in più istruzioni macchina:

1. **Fetch**: Lettura del valore dalla memoria in un registro della CPU.
2. **Add**: Esecuzione dell'operazione matematica nel registro.
3. **Store**: Scrittura del risultato dal registro alla memoria.

Se il *Thread 0* e il *Thread 1* eseguono il **Fetch** contemporaneamente, leggeranno lo stesso valore iniziale. Entrambi calcoleranno un risultato parziale e lo sovrascriveranno in memoria (**Store**), ignorando l'aggiornamento dell'altro thread (problema dell'*aggiornamento perso*).

6.7.2 Sincronizzazione vs Scalabilità

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    // Soluzione ingenua per evitare il data race
    #pragma omp critical
    sum += a[i];
}
```

Proteggere l'accesso alla variabile condivisa con costrutti come `#pragma omp critical` o `atomic` risolve il data race, ma **distrugge la scalabilità** del programma. Questi costrutti introducono punti di sincronizzazione che costringono i thread ad attendersi a vicenda, creando colli di bottiglia (*overhead*). Per operazioni di somma globale, la soluzione efficiente e corretta è sempre l'uso della clausola `reduction`.

```
double *a,
sum = 0;
int N;
#pragma omp parallel for reduction(+: sum)
for ( int i = 0; i < N; i++ )
sum += a[i];
```

6.8 Il Problema del False Sharing

Un tentativo comune per evitare i data race senza usare lock o reduction è fornire a ogni thread un indice dedicato in un array condiviso.

```
int nthreads = omp_get_num_threads();
double sum[nthreads]; // Array: un elemento per thread

#pragma omp parallel for
for (int i = 0; i < N; i++) {
    int me = omp_get_thread_num();
    sum[me] += a[i]; // No data race, ma genera false sharing!
}
```

Questa scalabilità non è possibile perché i valori di `sum[nthreads]` risiedono nelle stesse righe della cache; quindi, quando un thread accede e modifica la sua posizione, per mantenere la coerenza la cache deve scrivere e ripulire ogni volta. Questo è chiamato *false sharing*.

Si tratta di un problema relativo alla condivisione della memoria (si condivide esplicitamente e intenzionalmente una regione di memoria tra i thread). Il *False Sharing* è un degrado delle prestazioni che si verifica quando due o più thread modificano variabili indipendenti (diverse da qualsiasi altro thread nel pool parallelo) che risiedono fisicamente nella stessa *cache line*. A differenza del data race, il false sharing non altera la correttezza del risultato, ma compromette gravemente la scalabilità. Il false sharing è un problema quando si verifica un gran numero di volte: thread che accedono a elementi indipendenti nella stessa linea di cache solo una volta ogni tanto è qualcosa di molto comune e non un problema (è lo scopo della memoria condivisa).

I processori caricano i dati dalla memoria RAM alle cache L1/L2 in blocchi chiamati **cache lines** (tipicamente 64 byte). Poiché gli elementi di un array (`sum[0]`, `sum[1]`, ecc.) sono contigui in memoria, vengono caricati tutti nella stessa cache line.

- Supponiamo che entrambi i thread leggano la loro posizione di memoria di destinazione: l'intera riga viene caricata nella cache di entrambi i core e la riga è contrassegnata come "S" (condivisa).
- Se il Core 0 modifica `sum[0]`, il protocollo di coerenza della cache (es. MESI) segna l'intera cache line come (Modified) e invalida (Invalid) le copie presenti nelle cache degli altri core.
- Se il Core 1 deve poi modificare `sum[1]`, deve ricaricare l'intera cache line dalla memoria, invalidando a sua volta la copia del Core 0.
- Se il Core 0 deve modificare nuovamente `sum[0]`, deve anch'esso ricaricare l'intera cache line dalla memoria, invalidando a sua volta la copia del Core 1. E così via.

6.8.1 Risoluzione Tramite Padding

Una tecnica per mitigare il false sharing è il **padding**: distanziare artificialmente le variabili in memoria affinché finiscano su cache lines separate.

```
// Creiamo spazio vuoto tra gli elementi (es. fattore 8)
double sum[nthreads * 8];

#pragma omp parallel for
for (int i = 0; i < N; i++) {
    int me = omp_get_thread_num();
    // Ora sum[0] e sum[8] sono su cache lines differenti
    sum[me * 8] += a[i];
}
```

Sebbene il padding risolva il collo di bottiglia hardware, è considerato una **cattiva pratica** perché:

1. Consuma memoria inutilmente.
2. Si basa su "numeri magici" (come il fattore 8) che dipendono dalla dimensione specifica della cache line di una determinata CPU, rendendo il codice non portabile.
3. La soluzione standard, sicura e portatile per queste operazioni rimane l'uso della clausola `reduction`.

6.9 Direttive Annidate: Un Dettaglio Importante

Quando si dichiara la parallelizzazione di un ciclo all'interno di una regione parallela preesistente, la scelta tra `#pragma omp for` e `#pragma omp parallel for` altera drasticamente il comportamento del programma.

6.9.1 Caso A: Condivisione del Lavoro (**omp for**)

```
#pragma omp parallel
{
    int me = omp_get_thread_num();
    // Corretto: divide il ciclo tra i thread gia' esistenti
    #pragma omp for
    for (int i = 0; i < N; i++) {
        sum[me] += a[i];
    }
}
```

In questo scenario, la direttiva `#pragma omp for` prende i thread che formano il pool della regione parallela esterna e distribuisce loro le iterazioni del ciclo. Il ciclo `for` globale viene eseguito una sola volta, diviso tra i thread.

6.9.2 Caso B: Creazione di Nuove Regioni (**omp parallel for**)

```
#pragma omp parallel
{
    int me = omp_get_thread_num();
    // Errato/Pericoloso: ogni thread tenta di creare un nuovo pool
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        sum[me] += a[i];
    }
}
```

Inserire la parola chiave `parallel` obbliga ogni thread del pool originario a tentare di generare una nuova regione parallela annidata.

- **Se l'annidamento è attivo:** Vengono generati n nuovi pool di thread, e l'intero ciclo `for` viene eseguito n volte (una per ogni thread esterno).

- **Se l'annidamento è inattivo (default):** Ogni thread entra nella nuova regione parallela da solo, eseguendola in modo sequenziale. Anche in questo caso, ogni singolo thread esterno eseguirà la totalità del ciclo `for`, azzerando i benefici della parallelizzazione del loop.

Regola pratica: Utilizzare `omp for` all'interno di regioni `parallel` attive, e riservare `omp parallel for` ai blocchi di codice sequenziale.

6.10 Caso di Studio: Calcolo del Pi-greco

L'algoritmo si basa sul calcolo dell'area sottesa alla curva $f(x) = \frac{4}{1+x^2}$ nell'intervallo $[0, 1]$ suddividendola in n rettangoli (regola del punto medio).

6.10.1 L'Algoritmo Sequenziale di Base

```
double fSum = 0.0;
double fH = 1.0 / (double)n; // Base dei rettangoli
double fX;
int i;

for (i = 0; i < n; i++) {
    fX = fH * ((double)i + 0.5); // Punto medio sull'asse x
    fSum += (4.0 / (1.0 + fX * fX)); // Somma delle altezze
}
double pi = fH * fSum; // Area totale = base * somma_altezze
```

6.10.2 Tentativi di Parallelizzazione ed Errori Comuni

Nel parallelizzare il ciclo `for`, la gestione dell'accumulatore `fSum` determina il successo o il fallimento del codice.

6.10.2.1 Errore 1: Il Data Race (Variabile Condivisa)

Se ci dimentichiamo di gestire esplicitamente `fSum`, la regola di scoping implicito di OpenMP la rende condivisa (poiché dichiarata prima del costrutto parallelo).

```
// ERRORE: Data race su fSum
#pragma omp parallel for private(fX, i)
```

Effetto: Più thread tentano di sommare contemporaneamente alla stessa variabile globale. Molti aggiornamenti vanno persi. Il risultato è imprevedibile, errato e non deterministico (varia ad ogni esecuzione).

6.10.2.2 Errore 2: L'Obligo (Variabile Privata)

Cercando di evitare il data race, potremmo cadere nell'estremo opposto rendendo fSum privata.

```
// ERRORE: Il lavoro viene perso
#pragma omp parallel for private(fX, i, fSum)
```

Effetto: Ogni thread calcola un pezzo di area su una copia privata (non inizializzata per giunta). Al termine del ciclo, le copie private vengono distrutte. La variabile globale fSum rimane intoccata al suo valore iniziale (0.0). Il risultato calcolato sarà sempre 0.

6.10.2.3 Errore 3: Il Furto (Variabile Lastprivate)

Potremmo pensare di recuperare il valore usando lastprivate.

```
// ERRORE: Solo l'ultimo thread salva il risultato
#pragma omp parallel for private(fX, i) lastprivate(fSum)
```

Effetto: OpenMP copia nella variabile globale solo la somma parziale calcolata dal thread che ha gestito l'ultima iterazione logica ($i = n - 1$). Il lavoro di tutti gli altri thread viene scartato. Il risultato corrisponderà solo all'area dell'ultima fetta della curva.

6.10.3 La Soluzione Ottimale: Reduction

La clausola reduction è lo strumento corretto per le operazioni di accumulo globale.

```
// SOLUZIONE CORRETTA E SCALABILE
#pragma omp parallel for private(fX, i) reduction(+:fSum)
for (i = 0; i < n; i++) {
    fX = fH * ((double)i + 0.5);
    fSum += (4.0 / (1.0 + fX * fX));
}
```

Effetto:

1. Ogni thread riceve una copia privata ombra di fSum inizializzata a 0.0.
2. I thread calcolano le somme parziali in totale isolamento (massima scalabilità, nessun data race).
3. Al termine, OpenMP somma in modo sicuro tutti i risultati parziali nella variabile globale fSum.

6.11 NUMA Awareness

Abbiamo già discusso di come un'unica memoria centrale con una larghezza di banda fissa rappresenti un collo di bottiglia per le prestazioni in sistemi con molti core. Per superare questo problema, i moderni sistemi multiprocessore adottano un'architettura NUMA (Non-Uniform Memory Access), in cui la memoria è divisa fisicamente in nodi locali associati a ciascun processore o gruppo di processori in modo che ogni nodo abbia una memoria locale con accesso più veloce rispetto all'accesso alla memoria remota di altri nodi. In questo modo, la larghezza di banda aggregata risultante scala all'aumentare del numero di socket anche se, come sappiamo, la coerenza della cache diventa il nuovo fattore limitante. Chiaramente, abbiamo lo svantaggio che l'accesso alla memoria non è più uniforme: se un thread accede alla memoria locale del suo nodo, ottiene prestazioni migliori rispetto a quando accede alla memoria remota di un altro nodo e questo potrebbe avere delle conseguenze sul modo in cui scriviamo il nostro codice parallelo.

OpenMP fornisce strumenti per gestire la consapevolezza NUMA, consentendo agli sviluppatori di ottimizzare l'allocazione della memoria e l'affinità dei thread per migliorare le prestazioni in ambienti NUMA. OpenMP permette, infatti, di decidere dove eseguire ciascun thread e dove allocare la memoria, in modo da massimizzare l'accesso alla memoria locale e minimizzare l'accesso alla memoria remota.

Abbiamo già discusso, inoltre, la capacità di eseguire più thread su uno stesso core e abbiamo chiamato questo fenomeno **Simultaneous Multithreading (SMT)**. Chiameremo **strand** i diversi thread che condividono lo stesso core e **swthread** i thread software che vengono gestiti da OpenMP. Il posizionamento sui core dei thread è chiamato **affinità dei thread**.

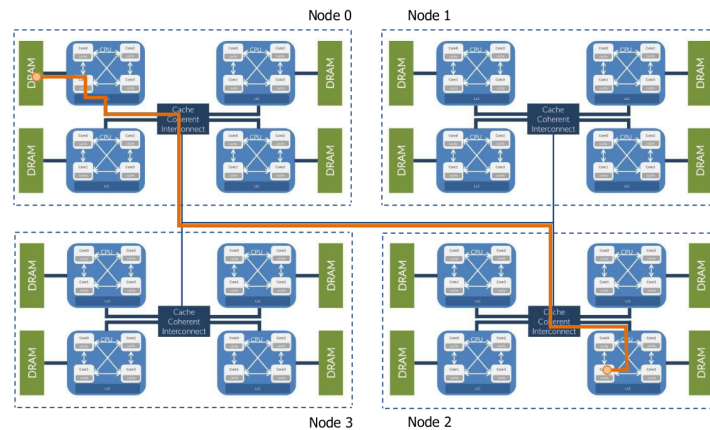
6.11.1 Affinità dei Thread

L'affinità dei thread si riferisce alla capacità di controllare su quali core fisici vengono eseguiti i thread in un sistema multiprocessore. L'obiettivo è quello di massimizzare l'efficienza del sistema minimizzando l'accesso alla memoria remota e sfruttando al meglio la cache locale. Ad esempio, se n thread lavorano su dati condivisi, sarebbe bene che condividessero la cache L2 e, di conseguenza, che lavorino sullo stesso socket. Se invece n thread lavorano su dati completamente indipendenti, sarebbe meglio distribuirli su socket diversi per sfruttare la larghezza di banda aggregata.

Per minimizzare il numero di accessi in memoria, dobbiamo dunque tenere in considerazione:

- **Dove:** ovvero in quale banca di memoria si trovano i dati.

- **Chi:** ovvero quale thread sta accedendo a quei dati e come sono distribuiti i dati sui core.
- **Cosa:** come è distribuito il lavoro tra i thread e se è possibile sfruttare la cache locale.



L'obiettivo primario nella gestione di un sistema NUMA è la distribuzione ottimale del carico di lavoro per eliminare la **contesa delle risorse** in termini di potenza di calcolo, cache e memoria. Per massimizzare le prestazioni, ogni swthread OpenMP dovrebbe avere una risorsa di calcolo dedicata e l'accesso più rapido possibile ai propri dati.

Per ottenere ciò, è necessario intervenire su quattro fronti:

- **Mappatura esplicita:** associare i thread ai core fisici (hwthread).
- **Allocazione mirata:** posizionare i dati nella memoria fisica più "vicina" al core che li elabora.
- **Località dei dati:** minimizzare i tempi di latenza riducendo l'accesso alla memoria remota.
- **Migrazione dinamica:** se necessario, spostare memoria o thread tra i nodi NUMA per ristabilire l'equilibrio durante l'esecuzione.

Il posizionamento ottimale dei thread non è universale, ma dipende strettamente dalla natura dell'algoritmo e dalla condivisione dei dati.

Thread "Distanti" (Spread)

Posizionare i thread su unità di calcolo lontane (es. socket diversi) offre vantaggi e svantaggi specifici:

- **PRO:** Aumenta la **larghezza di banda aggregata**, poiché ogni thread può sfruttare i canali di memoria di nodi diversi.
- **PRO:** Migliora l'uso della **cache locale**, riservando l'intera cache di un core/socket ai dati di un singolo thread.
- **CONTRO:** Peggiora le prestazioni dei **costrutti di sincronizzazione** (es. barriere) a causa della latenza di comunicazione tra socket.
- **CONTRO:** Vanifica i vantaggi della cache se i thread devono leggere frequentemente gli stessi dati.

Thread "Vicini" (Close)

Posizionare i thread vicini (es. sullo stesso core tramite SMT o sullo stesso socket) cambia radicalmente il profilo prestazionale:

- **PRO:** Riduce drasticamente la **latenza di sincronizzazione**.
- **CONTRO:** Riduce la **larghezza di banda totale** disponibile, poiché più thread competono per lo stesso canale di memoria.
- **Variabile:** L'impatto sulla cache può essere positivo (se condividono dati) o negativo (se si "sfrattano" a vicenda i dati dalla cache - *cache thrashing*).

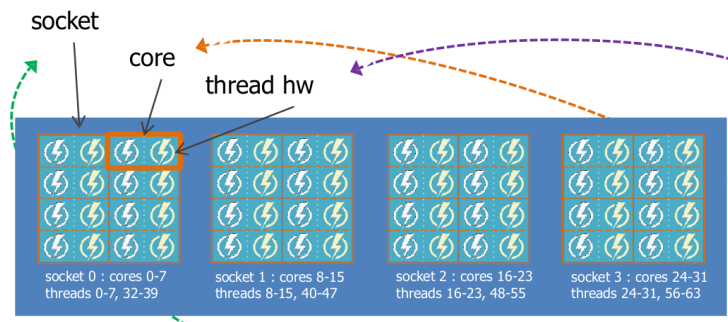
OpenMP offre 2 concetti di base per gestire l'affinità dei thread:

- **Places:** ovvero a quali entità fisiche ci riferiamo con la nostra richiesta di affinità, cioè dove vengono eseguiti i thread software.
- **Binding:** ovvero se esiste una relazione tra thread e places, e se sì, quale tipo di relazione.

6.11.2 Placecs

OpenMP fornisce diverse modalità di affinità dei thread, che possono essere specificate. I places rappresentano il dove vengono eseguiti gli **swthread**. I nomi dei places sono:

- **Threads:** ogni posizione corrisponde a un hwthread o strand sui core.



- **Cores:** ogni posizione corrisponde a un core fisico (con tutti i suoi hwthread).
- **Sockets:** ogni posizione corrisponde a un socket fisico (con tutti i suoi core).

Esempio socket

Immaginiamo di avere la seguente configurazione hardware:

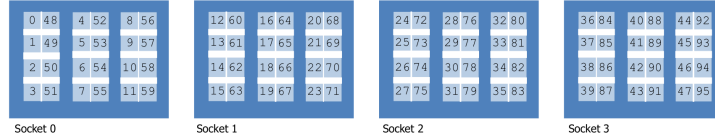
- 4 sockets;
- 12 core per socket;
- 2 hwthread per core.
- Ogni socket corrisponde a un nodo NUMA.

Per i nodi NUMA varrà la seguente configurazione:

- NUMA node0 CPU: 0-11, 48-59;
- NUMA node1 CPU: 12-23, 60-71;
- NUMA node2 CPU: 24-35, 72-83;
- NUMA node3 CPU: 36-47, 84-95.

I primi valori (0-47) rappresentano i core fisici, mentre i secondi (48-95) rappresentano gli hwthread, cioè quelli che vengono attivati solo in caso di SMT. Se volessimo massimizzare le prestazioni, potremmo scegliere di posizionare i thread solo sui core fisici (0-47) evitando di attivare gli hwthread (48-95) così da assegnare un core a ogni thread. Se invece volessimo massimizzare

il numero di thread, potremmo scegliere di attivare anche gli hwthread (48-95) così da raddoppiare il numero di thread disponibili, ma a scapito delle prestazioni a causa della contesa per le risorse del core.



Vale che se ci sono $nsocket$ con $ncore_per_socket$, allora abbiamo un totale di:

$$nore = nsocket \times ncore_per_socket$$

E possiamo gestire un numero di thread pari a:

$$nthread = nsocket \times nthread_sMT$$

Per dare a OpenMP indicazioni sull'affinità dei thread, possiamo usare la variabile d'ambiente `OMP_PLACES = {sockets | cores | threads}` con, rispettivamente, le seguenti interpretazioni:

- **Threads:** si riferisce ai core logici, ovvero tiene conto dei thread SMT come posti distinti.;
- **Cores:** si riferisce ai core fisici, ovvero tiene conto solo dei core fisici considerando nell'esempio i core 0,48 come lo stesso posto. Il thread che sarà assegnato a questo posto potrà essere eseguito su uno qualsiasi dei due core fisici.
- **Sockets:** si riferisce ai socket fisici.

È possibile specificare anche una mappatura personalizzata dei places tramite la sintassi `OMP_PLACES = {list}` dove `list` è una lista di intervalli di CPU (core o hwthread) che definiscono i places. Ad esempio:

- `OMP_PLACES = {0,1}` definisce una posizione composta dal hwthread 0 della core 0 e dal hwthread 1 del core 1;
- `OMP_PLACES = {0,48}` definisce una posizione composta dal hwthread 0 e dal hwthread 48 dello stesso core 0 dove 48 è quello attivato tramite SMT.
- `OMP_PLACES = {0 : 2 : 48}` definiscono una posizione composta da 2 core che sono 0 e 48. La sintassi qui è `inizio:contatore:passo` dove `inizio` è il primo core della posizione, `contatore` è il numero di core da includere nella posizione e `passo` è la distanza tra i core inclusi. In questo caso, la posizione include i core 0 e 48, saltando i core intermedi.

- $OMP_PLACES = \{0 : 4 : 1\} : 4 : 12$ è un esempio più complesso che definisce 4 posizioni, ognuna composta da 4 core consecutivi. La prima posizione include i core da 0 a 3, la seconda posizione include i core da 12 a 15, la terza posizione include i core da 24 a 27 e la quarta posizione include i core da 36 a 39. Questo perché la prima parentesi ci dice di prendere 4 core consecutivi a partire da 0, poi fuori ci viene detto di creare 4 posizioni con un passo di 12 core tra una posizione e l'altra.

Queste posizioni sono statiche e non possono cambiare durante l'esecuzione del programma. Se un thread viene assegnato a una posizione, rimarrà in quella posizione per tutta la durata del programma.

Adesso quello che ci rimane da capire è come vengono posizionati gli swthread sui places. Vedremo che questo dipende dalle politiche di binding, ovvero dalle regole che governano l'associazione tra thread e places.

6.11.3 Binding

Il binding definisce come i **swthread** vengono mappati ai **places**. Esistono diverse politiche di binding che possono essere specificate tramite la variabile d'ambiente `OMP_PROC_BIND`:

- **FALSE**: Il posizionamento dei thread è completamente affidato al sistema operativo e non è garantito che un thread rimanga nello stesso place durante l'esecuzione. Questa è la politica predefinita se non viene specificata alcuna politica di binding.
- **TRUE**: I thread vengono posizionati sui places dal sistema operativo in modo deterministico e non possono migrare durante l'esecuzione. In questo modo, ogni thread rimane sempre nello stesso place per tutta la durata del programma, garantendo una maggiore prevedibilità delle prestazioni.
- **CLOSE**: Gli swthread vengono posizionati in posizioni il più vicino possibile tra loro, cercando di minimizzare la distanza tra i thread. Ad esempio, se abbiamo 4 thread e 4 core, i thread saranno posizionati sui core 0, 1, 2 e 3. Anche nel caso in cui vi siano più hwthread per core, i thread saranno posizionati sui core più vicini possibile, cercando di sfruttare prima i core fisici e poi gli hwthread attivati tramite SMT.
 - Se $T \leq P$: Ogni thread viene assegnato a un place univoco in modo consecutivo. Il thread k viene mappato al place $(m + k) \pmod{P}$, dove m è la posizione del thread master.

- **Se $T > P$:** Poiché i posti non bastano, OpenMP distribuisce i thread in sottoinsiemi S_i . Ogni place riceve un numero di thread compreso tra $\lfloor T/P \rfloor$ e $\lceil T/P \rceil$. In questo modo, i thread "eccedenti" vengono ammassati sugli stessi posti, partendo da quello del master.
- **SPREAD:** Gli swthread vengono posizionati in posizioni il più uniformemente possibile, cercando di massimizzare la distanza tra i thread. Ad esempio, se abbiamo 4 thread e 4 core, i thread saranno posizionati sui core 0, 12, 24 e 36.
 - **Se $T \leq P$:** L'elenco dei places viene diviso in T sotto-partizioni. Ogni sotto-partizione contiene circa P/T posti consecutivi. Ogni thread viene assegnato al **primo posto** della propria sotto-partizione. Questo garantisce che i thread siano fisicamente il più lontano possibile (ad esempio, uno per ogni socket).
 - **Se $T > P$:** L'elenco dei places viene diviso in P sotto-partizioni (una per ogni posto reale). Ogni place ospiterà più thread con ID consecutivi, in numero pari a $\lfloor T/P \rfloor$ o $\lceil T/P \rceil$. Anche in questo caso di "sovraccarico", OpenMP cerca di mantenere una distribuzione bilanciata su tutta la macchina.
- **MASTER/PRIMARY:** Tutti i swthread vengono posizionati sulla stessa posizione del thread master (thread 0). Ad esempio, se il thread master è posizionato su un intero socket, tutti gli swthread saranno posizionati sullo stesso socket, condividendo la cache e la memoria locale di quel socket.
 - **OMP_PLACES=threads:** Tutti i thread software vengono forzati sullo stesso singolo hwthread. Si verifica una condizione di **over-subscription** estrema (tutti i thread lottano per lo stesso processore logico); è una configurazione usata quasi esclusivamente per test di contesa.
 - **OMP_PLACES=cores:** Tutti i thread vengono assegnati allo stesso core fisico. Se il core supporta SMT, i thread si distribuiranno sui diversi hwthread di quel core prima di andare in over-subscription.
 - **OMP_PLACES=sockets:** Tutti i thread vengono confinati all'interno dello stesso socket fisico. In questo caso, OpenMP ha la libertà di distribuire i thread sui vari core disponibili all'interno di quel socket, garantendo comunque che tutti condividano la memoria locale e la cache L3 del processore.

Per assegnare questi valori, scriviamo banalmente:

```
export OMP_PROC_BIND = {value}
```

places binding			
	THREADS	CORES	SOCKETS
CLOSE	i swt vengono posizionati su hwt vicini, saturando tutti gli hwt SMT in ogni core prima di utilizzare nuovi core	i swt vengono posizionati su hwt vicini, utilizzando 1 hwt/core prima di iniziare a utilizzare SMT	i swt vengono posizionati round-robin per socket, 1/core; dopo la saturazione, SMT viene utilizzato round-robin +1 hwt/socket
SPREAD	i swt sono posizionati in modo round-robin, su core liberi	simile al ← Si evita la SMT fino alla saturazione	simile al ← swt sono posizionati tramite socket round-robin e hwt
MASTER/PRIMARY	tutti i swt sono posizionati sullo stesso hwt sullo stesso core sullo stesso socket	tutti gli swt sono posizionati sullo stesso core sullo stesso socket, utilizzando tutti i suoi hwt	tutti gli swt sono posizionati sullo stesso socket, saturando tutti gli hwt a partire da quelli SMT

Possiamo, inoltre, specificare il binding a livello di regione parallela tramite la clausola `proc_bind` associata alla direttiva `omp parallel`:

```
#pragma omp parallel proc_bind( {value} ) {
    // codice parallelo
}
```

Sono inoltre disponibili anche altri comandi:

- **Ispezione dei PLACES:**

- `omp_get_num_places()`: Restituisce il numero totale di *places* definiti nel sistema.
- `omp_get_place_num()`: Restituisce l'ID del *place* a cui è assegnato il thread chiamante.
- `omp_get_place_num_procs(int place_num)`: Restituisce il numero di processori logici contenuti in un determinato *place*.
- `omp_get_place_proc_ids(int place_num, int *ids)`: Riempie un array con gli ID dei processori associati a un specifico *place*.

- **Reporting e Debug dell’Affinità:**

- `omp_display_affinity(const char *format)`: Stampa a video le informazioni di affinità per i thread correnti.

- * `omp_get_num_places()`: Restituisce il numero totale di *places* definiti nel sistema.
- * `omp_get_place_num()`: Restituisce l'ID del *place* a cui è assegnato il thread chiamante.
- * `omp_get_place_num_procs(int place_num)`: Restituisce il numero di processori logici contenuti in un determinato *place*.
- * `omp_get_place_proc_ids(int place_num, int *ids)`: Riempie un array con gli ID dei processori associati a un specifico *place*.
- `omp_capture_affinity(char *buffer, size_t size, const char *format)`: Cattura le informazioni di affinità in una stringa di testo.
- `omp_set_affinity_format(const char *format) / omp_get_affinity_format()`: Impostano o leggono il formato utilizzato per la visualizzazione delle informazioni di affinità.

6.11.4 Policy di Allocazione della Memoria

Oltre a controllare dove vengono eseguiti i thread, è fondamentale gestire anche dove vengono allocati i dati in memoria, soprattutto in un sistema NUMA. Troviamo diverse politiche di allocazione della memoria.

Policy "touch-first"

Supponiamo di operare su un sistema SMP in cui abbiamo che ogni socket è fisicamente connesso a un banco di RAM e poi fisicamente connesso a un altro socket. In questo modo, l'accesso alla memoria non è uniforme: la larghezza di banda necessaria a un core per accedere a un banco di memoria non fisicamente connesso ad esso è probabilmente significativamente inferiore a quella necessaria per accedere al banco più vicino.

Quando andiamo ad allocare un blocco di memoria, possiamo farlo attraverso due operazioni principalmente:

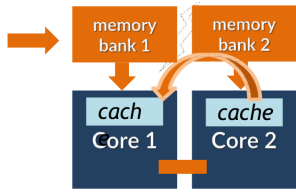
- **Malloc**: Qui segnaliamo che verrà utilizzata una certa quantità di memoria così che l'occupazione di memoria del processo aumenti. Tuttavia, la mappatura non avviene realmente fino a quando non accediamo a quella memoria per la prima volta. Cioè, fino a quando non la **tocchiamo**.

- **Calloc:** Qui segnaliamo che verrà utilizzata una certa quantità di memoria così che l'occupazione di memoria del processo aumenti. A differenza di `malloc`, `calloc` inizializza i byte allocati a zero di fatto toccandoli mappandoli immediatamente. Inoltre, per quanto riguarda la `calloc`, questa funzione ci assicura che la memoria allocata sia contigua, mentre con `malloc` non è garantito che la memoria allocata sia contigua.¹

Vediamo adesso il seguente esempio:

```
double *a = (double*)calloc( N, sizeof(double);
for ( int i = 0; ii < N; ii++ )
a[i] = initialize(i);

#pragma omp parallel for reduction(+: sum)
for ( int i = 0; i < N; i++ )
    sum += a[i];
```



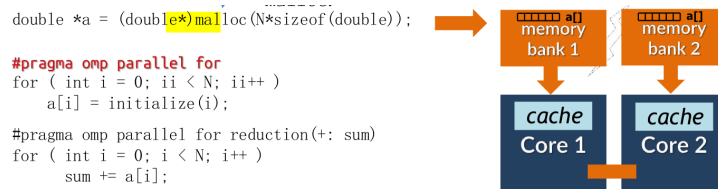
Qui, abbiamo che la prima cosa che viene fatta è una chiamata a `calloc` per allocare un blocco di memoria. Poiché `calloc` inizializza i byte allocati a zero, questa operazione comporta un accesso alla memoria che mappa immediatamente il blocco di memoria allocato al nodo NUMA del thread che ha effettuato la chiamata. In questo caso, se il thread che ha effettuato la chiamata a `calloc` è posizionato su un socket specifico, il blocco di memoria allocato sarà mappato al nodo NUMA associato a quel socket, garantendo così un accesso più veloce alla memoria locale per quel thread. Tuttavia, analizziamo adesso il ciclo `for` che viene parallelizzato e analizzato dal core 1 e dal core 2. Poiché i dati si trovano nella memoria del core 1, otteniamo che questi vengono effettivamente elaborati molto in fretta da quest'ultimo, tuttavia, quando il core 2 tenta di accedere a quei dati, deve accedere alla memoria remota del core 1, subendo una penalizzazione in termini di prestazioni a causa della latenza più elevata e della larghezza di banda inferiore associata all'accesso alla memoria remota.

Questa è dunque la politica **Touch-First** per cui la memoria viene allocata sul nodo NUMA del thread che effettua la chiamata di allocazione. Se i dati vengono elaborati principalmente da quel thread, questa politica può offrire prestazioni ottimali. Tuttavia, se i dati devono essere condivisi tra più thread su nodi NUMA diversi, questa politica può portare a prestazioni degradate a causa dell'accesso alla memoria remota. In questi casi, potrebbe essere necessario considerare altre politiche di allocazione della memoria o strategie di affinità dei thread per ottimizzare le prestazioni complessive del programma.

¹La contiguità è data dal fatto che i dati si trovano tutti nella stessa memoria

Policy "Touch-By-All"

Supponiamo di essere ancora una volta nella stessa situazione di prima ma, anziché usare la `calloc` usiamo la `malloc`.



In questo caso, la chiamata a `malloc` riserva un set di indirizzi virtuali, ma non impegna memoria fisica. La mappatura avviene a livello di singole pagine (granularità di 4 KB) solo quando i thread effettuano il primo accesso (**First-Touch**). Grazie all'inizializzazione parallela, le pagine toccate dal Core 1 verranno allocate nel suo nodo NUMA locale, mentre quelle toccate dal Core 2 finiranno nel proprio. Questo distribuisce i dati fisicamente, abbattendo la latenza e sfruttando la larghezza di banda di entrambi i controller di memoria.

Quindi, quando il core 1 accede per la prima volta a quel blocco di memoria, esso viene mappato al nodo NUMA del core 1. Allo stesso modo, quando il core 2 accede per la prima volta a quel blocco di memoria, esso viene mappato al nodo NUMA del core 2. In questo modo, entrambi i core hanno accesso alla memoria locale, riducendo così la latenza e aumentando la larghezza di banda disponibile per entrambi i thread.

Questa è dunque la politica **Touch-By-All** per cui la memoria viene allocata senza essere mappata a nessun nodo NUMA specifico, e viene mappata al nodo NUMA del thread che accede per primo a quella memoria. Questa politica può essere vantaggiosa quando i dati devono essere condivisi tra più thread su nodi NUMA diversi, poiché consente a ciascun thread di accedere alla memoria locale, migliorando così le prestazioni complessive del programma. Tuttavia, se i dati vengono elaborati principalmente da un singolo thread, questa politica potrebbe non offrire prestazioni ottimali a causa della latenza associata all'accesso alla memoria remota per gli altri thread.

6.11.5 Strumenti per l'Analisi della Topologia e il Pinning

Per ottimizzare l'affinità è fondamentale conoscere la gerarchia hardware del sistema. Gli strumenti principali si dividono in due categorie:

1. Scoperta della Topologia

- **lscpu** / **/proc/cpuinfo**: Forniscono informazioni generali sull'architettura (numero di core, socket, thread SMT).
- **lstopo (hwloc)**: Fornisce una rappresentazione grafica o testuale dettagliata della gerarchia delle cache, dei core e dei nodi NUMA.
- **numactl --hardware**: Mostra l'inventario dei nodi NUMA disponibili e la "distanza" (latenza) tra di essi.
- **likwid-topology**: Uno strumento avanzato per visualizzare la topologia dei thread e le relazioni con le cache.

2. Gestione dell'Affinità (Pinning)

Questi tool permettono di gestire l'affinità dall'esterno, senza modificare il codice:

- **taskset**: Permette di impostare la maschera di affinità di un processo tramite una maschera esadecimale o una lista di CPU (`--cpu-list 0,2,4-8`).
- **numactl**:
 - `--cpunodebind=n`: Forza l'esecuzione sui core del nodo NUMA *n*.
 - `--membind=n`: Forza l'allocazione della memoria esclusivamente sui banchi del nodo NUMA *n*.
- **likwid-pin**: Uno strumento molto potente per il pinning di applicazioni multithread, capace di gestire gruppi di core basandosi sulla topologia (es. "pinnami un thread per ogni socket").

6.11.6 Workflow per la NUMA-awareness

Per rendere un'applicazione multithread efficiente su architetture NUMA, è possibile seguire una metodologia strutturata in cinque fasi:

1. **Esplorazione della topologia**: Utilizzo di strumenti come **lstopo** o **numactl -H** per identificare la gerarchia dei core e dei nodi di memoria.

2. **Definizione di Places e Binding:** Configurazione delle variabili `OMP_PLACES` (es. *cores*) e `OMP_PROC_BIND` (es. *spread*) in base alle necessità di condivisione dati dell'algoritmo.
3. **Implementazione della politica di First-Touch:** Sostituzione di allocazioni seriali (`calloc`) con allocazioni tramite `malloc` seguite da una **inizializzazione parallela**. Questo assicura che ogni pagina di memoria sia mappata nel nodo NUMA locale al thread che la elaborerà (concetto di *touch-by-all*).
4. **Validazione del binding:** Verifica sperimentale del posizionamento dei thread tramite funzioni di runtime come `omp_display_affinity()` o strumenti esterni come `likwid-pin`.
5. **Analisi delle prestazioni:** Misurazione dello speedup scalando il numero di thread e confronto tra la versione ottimizzata e quella *non-aware* per quantificare il guadagno ottenuto eliminando gli accessi alla memoria remota.

Capitolo 7

Tecniche di Compilazione Ottimizzata e Profiling

Come già detto più volte, per scrivere codice funzionante e ottimizzato, è importante capire bene come lavori il compilatore e come questo manipola il nostro codice per estrapolarne il meglio.

7.1 Compilatore

Il **compilatore** è un programma che traduce il linguaggio sorgente in un linguaggio target equivalente. Questo è importante per il calcolo HPC perché funge da intermediario tra codice e hardware. Inoltre, un buon compilatore riesce a ottimizzare altamente il codice che abbiamo scritto per una specifica architettura proprio per garantire delle ottime prestazioni. Comprenderne i limiti, infatti, ci permetterà di scrivere codice che può essere ottimizzato ancora meglio dal compilatore stesso.

Un **interprete**, al contrario di quanto fa un compilatore, è un programma di elaborazione del linguaggio che esegue **direttamente** un programma in un **linguaggio sorgente**.

Questo risulta generalmente in un codice più portabile ma che, nel caso del calcolo HPC non torna molto utile poiché l'ottimizzazione su macchine specifiche è un grosso vantaggio in termini di prestazioni generali del programma.

7.1.1 Pipeline di Compilazione

La compilazione fa parte di una **pipeline** che traduce il codice sorgente in un codice eseguibile dalla macchina. Questa pipeline comprende:

- **Preprocessore:** Analizza il sorgente includendo gli header ed espandendo le macro. Gli Header non sono altro che dei file al cui interno contengono solo la dichiarazione della funzione ma non il suo corpo.
- **Front-end o Compilazione (in base alla slide che stai leggendo):** Traduce il sorgente in un linguaggio ad alto livello di astrazione, assembly nello specifico. Qui vengono anche effettuate le prime analisi sintattiche e semantiche del programma.
- **Assembler:** Produce il codice macchina dall'assembly generato alla fase precedente risolvendo label interni e lasciando quelli esterni ancora irrisolti.
- **Linker:** Risolve i riferimenti e gli indirizzi di memoria tra diverse sezioni del codice e librerie esterne.



I moderni compilatori sono anche in grado di applicare trasformazioni al codice per migliorarne le prestazioni:

- **Constant folding:** calcola espressioni costanti a tempo di compilazione.
- **Dead code elimination:** rimuove codice irraggiungibile o mai utilizzato.
- **Inlining delle funzioni:** sostituisce le chiamate con il corpo della funzione.
- **Loop unrolling:** replica il corpo del ciclo per ridurre il controllo del loop.
- **Common subexpression elimination:** evita di ricalcolare la stessa espressione.
- **Vettorizzazione automatica:** usa istruzioni SIMD (SSE, AVX) su array.
- **Instruction scheduling:** riordina le istruzioni per sfruttare la pipeline della CPU.
- **Register allocation:** minimizza gli accessi in memoria usando i registri della CPU.

I compilatori moderni (come GCC o Clang) offrono diversi "preset" di ottimizzazione che bilanciano velocità di compilazione, facilità di debug e performance del codice eseguibile.

- **-O0 (Nessuna ottimizzazione):**

- Offre una compilazione molto veloce e un debug semplificato, poiché ogni istruzione C corrisponde a poche istruzioni assembly.
- Il codice risultante è più lento ma completamente prevedibile, rendendolo ideale per l'uso durante lo sviluppo con strumenti come *gdb*.

- **-O1 (Ottimizzazioni di base):**

- Riduce le dimensioni dell'eseguibile e i tempi di esecuzione senza allungare significativamente i tempi di compilazione.
- Include l'eliminazione di variabili inutili e l'ottimizzazione dei salti.
- Per loop semplici, può risultare già 2 o 5 volte più veloce rispetto a -O0.

- **-O2 (Ottimizzazioni raccomandate):**

- Rappresenta lo standard per i rilasci ufficiale e il punto di partenza corretto per il codice in produzione.
- Attiva quasi tutte le ottimizzazioni considerate "sicure", come la vettorizzazione, la *Common Subexpression Elimination* (CSE) e le trasformazioni dei loop.

- **-O3 (Ottimizzazioni aggressive):**

- Introduce l'inlining aggressivo e altre tecniche che possono aumentare la dimensione del codice (*code bloat*).
- È necessario testare attentamente le prestazioni poiché su codici di grandi dimensioni può risultare più lento di -O2 a causa di effetti negativi sulla cache.

- **-Ofast (Aggressivo + FP non-standard):**

- Include tutte le ottimizzazioni di -O3 più il flag *-ffast-math*.
- Permette il riordinamento delle operazioni in virgola mobile violando lo standard IEEE 754.

- **Attenzione:** Può alterare i risultati numerici; non deve mai essere usato in codice scientifico senza una validazione accurata dei risultati.
- **-Os (Ottimizzazione per dimensioni):** Sebbene nel calcolo HPC questa non risulti particolarmente utile, è giusto segnalarela. Si tratta di ottimizzazioni che non cercano di aumentare le prestazioni del sistema in termini di tempo ma in termini di dimensione del codice. Spesso utilizzato per i microcontrollori.
- **-march=native:** Istruisce il compilatore a generare codice specifico per l'architettura della CPU su cui sta girando, sfruttando tutti i set di istruzioni (ISA) disponibili. Infatti, il compilatore generalmente produce un codice portabile, cioè un codice che può essere eseguito su qualsiasi CPU appartenente a quella classe di architettura.

7.1.2 Il Memory Aliasing

Il **Memory Aliasing** si verifica quando due o più riferimenti (puntatori) puntano alla stessa posizione di memoria. Questo è uno dei maggiori blocchi alle prestazioni in C/C++.

```
void func1 ( int *a, int  *b ) {
    *a += *b;    -> *a e    *b  adesso contengono 2
    *a += *b;    -> *a e    *b  adesso contengono 4
}

void func2 ( int *a, int *b    )
    *a += 2 * *b; -> *a e *b  adesso contengono 3
}
```

Consideriamo queste due funzioni. In effetti, se a e b puntano a due indirizzi diversi, il risultato delle due funzioni è lo stesso. Il discorso cambia, invece, quando a e b puntano allo stesso indirizzo come mostrato in figura.

Allo stesso modo accade nel seguente codice:

```
void scale(double *a, double *b, int n, double s){
    for(int i = 0; i<n; i++)
        a[i] = s * b[i-1];
}
```

Se il compilatore non può garantire che due puntatori siano distinti, non può ottimizzare gli accessi alla memoria perché teme che una scrittura su uno possa influenzare il valore dell'altro.

Questo costringe la CPU a ricaricare continuamente i dati dalla memoria centrale anziché usare i registri o la cache. Per risolvere questo problema usiamo il qualificatore *restrict*. Esistono anche due flag:

- `-fstrict-aliasing`: questo flag dice al compilatore che non possono esserci aliasing tra tipi diversi. È di default nel caso di `-O2`
- `-fno-strict-aliasing`: serve a dire al compilatore di non assumere che non ci siano sovrapposizioni in nessun caso.

Esistono molti altri quantificatori che aiutano il compilatore:

- **Const**: Questo dice al nostro compilatore che il valore di questa variabile non cambierà nello scope corrente permettendogli, dunque, di tenere il valore in registro senza leggerlo più volte dalla memoria.
- **restrict**: Ne abbiamo già parlato prima, dice al compilatore che quel puntatore punta a un indirizzo di memoria esclusivamente puntato da lui.
- **volatile**: informa il compilatore che quel valore potrebbe cambiare inaspettatamente, così facendo, il compilatore ne impedisce il caching in registro e riordinamento delle istruzioni.
- **register**: suggerisce al compilatore che quella variabile verrà usata spesso e quindi deve tenerla in registro. Suggerimento che potrebbe essere tranquillamente ignorato dal compilatore poiché in genere se ne occupa lui in maniera più efficiente.

Esistono anche diverse classi di storage:

- **extern**: variabili globali che esistono per tutta la durata del programma.
- **auto**: variabili locali allocate sullo stack e distrutte subito dopo l'uscita dal blocco.
- **static**: variabili che persistono tra le varie chiamate a funzioni. Sono visibili solo all'interno del file corrente e permettono di evitare reinizializzazioni e abilita il caching.

In generale, nel caso in cui volessimo analizzare l'assembly del nostro codice, esistono diversi strumenti tra cui **godbolt**, **perf annotate** o, banalmente, **gcc -S -fverbose-asm**.

7.2 Profiling

Il profiling rappresenta il cuore del Performance Engineering. Consiste nell'analizzare l'esecuzione di un programma utilizzando carichi di lavoro (workload) reali per identificare con precisione gli hotspots (punti critici) e i bottleneck (colli di bottiglia) del codice.

Senza una misurazione oggettiva, l'ottimizzazione rischierebbe di basarsi su intuizioni spesso errate; profilare ci permette invece di quantificare i problemi per intervenire dove è realmente necessario.

Il ciclo iterativo del Performance Engineering si articola in quattro fasi:

- **Profilare:** Individuare i colli di bottiglia basandosi su dati di esecuzione reali.
- **Analizzare:** Studiare il codice per comprendere la causa radice del rallentamento (es. problemi di cache, algoritmi inefficienti, contesa di memoria).
- **Ottimizzare:** Implementare le modifiche mirate per risolvere il problema identificato.
- **Verificare:** Misurare nuovamente le prestazioni per convalidare il miglioramento ed escludere eventuali regressioni.

È fondamentale non procedere all'ottimizzazione basandosi sull'intuito, bensì su misurazioni concrete. Spesso sappiamo che il 90% del tempo di esecuzione è concentrato nel 10% del codice: è proprio da quegli hotspot che dobbiamo partire per massimizzare l'impatto del nostro lavoro. Idealmente, un processo di profiling efficace deve fornire risposte a domande precise:

- **Hotspots:** Quali sezioni di codice consumano la maggior parte del tempo totale?
- **Cache Efficiency:** Quanti *cache miss* si verificano e in quali loop la memoria diventa il collo di bottiglia?
- **Pipeline Utilization:** La CPU sta sfruttando correttamente la *pipeline* o si verificano stalli frequenti?
- **Throughput:** Quante istruzioni vengono effettivamente ritirate per ciclo di clock (IPC)?
- **Performance Bound:** Il codice è **compute bound** (limitato dalla capacità di calcolo) o **memory bound** (limitato dalla velocità della memoria)?

7.2.1 PMU

I moderni processori contengono hardware dedicato al monitoraggio delle proprie prestazioni. Sono circuiti integrati nella CPU che contano eventi micro architetturali con precisione. Questi sono parecchio utili:

- **Efficienza:** Garantisce un **overhead minimo** ($< 1\%$), permettendo misurazioni in tempo reale senza degradare significativamente le prestazioni del sistema.
- **Precisione Fisica:** Crea una correlazione diretta tra l'esecuzione del software e le caratteristiche fisiche della CPU, individuando le cause profonde dei colli di bottiglia.
- **Portabilità:** Grazie all'astrazione fornita dai tool moderni, è possibile raccogliere metriche hardware senza dover gestire manualmente i registri specifici di ogni diversa microarchitettura.

La PMU opera coordinando due tipologie principali di registri hardware, che lavorano in coppia per ogni "canale" di monitoraggio attivo:

- **Performance Event Select Register (PESR):** Rappresenta il "pannello di controllo" dell'evento.
 - **Configurazione:** Definisce *quale* evento monitorare (Event Select) e con quale specifica variante (Unit Mask).
 - **Filtraggio:** Contiene i bit di controllo per stabilire se il conteggio deve avvenire solo in modalità Utente, solo in Kernel, o in entrambe.
 - **Attivazione:** Funge da interruttore per far partire o fermare il monitoraggio del relativo contatore.
- **Performance Monitor Counter (PMC):** È il contatore digitale vero e proprio associato al PESR.
 - **Accumulazione:** Incrementa il proprio valore ogni volta che l'evento configurato nel PESR si verifica a livello microarchitetturale.
 - **Sampling (Interrupt su Overflow):** Questa è la sua funzione più potente. Quando il registro raggiunge il suo valore massimo (overflow), può generare un interrupt hardware. Questo permette al sistema operativo di "fotografare" esattamente quale istruzione stava girando in quel momento, permettendo il profiling statistico (campionamento).

I tipi di eventi monitorabili sono:

- **Istruzioni ritirate (INST_RETIRED)**: Indica il numero di istruzioni completate con successo dal processore. È una metrica fondamentale per misurare il lavoro utile effettivamente svolto.
- **Cicli di clock (CPU_CLK_UNHALTED)**: Rappresenta i cicli totali in cui la CPU è in stato attivo e non in stop. Combinato con le istruzioni ritirate, permette di calcolare l'IPC (*Instructions Per Cycle*).
- **Cache miss (LLC_MISSES, L1D_MISSES)**: Conteggia gli accessi falliti ai vari livelli di cache, come la L1 o la LLC (*Last Level Cache*). Alti valori qui indicano solitamente un codice *memory bound*.
- **Branch misprediction (BR_MISP_RETIRED)**: Misura gli errori di predizione del *branch predictor* hardware. Ogni errore causa lo svuotamento della pipeline, con una pesante penalità in termini di cicli persi.
- **TLB miss (D-TLB, I-TLB MISSES)**: Registra i fallimenti nella traduzione hardware degli indirizzi virtuali. Un numero elevato di questi eventi suggerisce che il programma sta accedendo alla memoria in modo troppo sparso su molte pagine diverse.
- **Stalli della pipeline (RESOURCE_STALLS)**: Indica i cicli persi a causa di dipendenze di dati tra istruzioni o conflitti strutturali all'interno della CPU.

I contatori possono essere usati in due modalità:

- **Counting mode**: Contiamo il numero esatto di eventi in un intervallo di codice. Avremo overhead minimo, risultati precisi, ideale per misure globali.
- **Sampling mode**: Il PMC genera un interrupt ogni N eventi, registrando l'istruzione corrente. Permette di identificare gli hotspot nel codice.

7.2.1.1 Performance Events for Linux (Perf)

Perf è l'infrastruttura standard di profiling su Linux, integrata nel kernel (v2.6+). Astrae le differenze hardware tra CPU offrendo un'interfaccia a riga di comando unificata. Perf è composto da due componenti:

- **Kernel SYSCALL:** Questo, attraverso la chiamata `perf_event_open()` permette un accesso unificato agli eventi del SO e agli eventi hardware PMU del processore.
- **User-space tools:** Questi sono i comandi utilizzabili da CLI per raccogliere e analizzare i dati.

`perf` permette di spiare dettagli “intimi” del sistema e quindi richiede privilegi. È necessario abbassare temporaneamente le restrizioni del kernel:

```
# Configurazione temporanea dei permessi
sudo bash -c 'echo 0 > /proc/sys/kernel/kptr_restrict'
sudo bash -c 'echo -1 > /proc/sys/kernel/perf_event_paranoid'
```

Esistono vari tipi di eventi che possiamo monitorare tramite `perf`:

- **Software events:** Si tratta di eventi generati interamente dal kernel del sistema operativo tramite software. Includono operazioni come:
 - *context-switches*: i cambi di contesto tra processi.
 - *cpu-migrations*: lo spostamento di un task da una CPU a un'altra.
 - *page-faults*: errori di pagina nella gestione della memoria.
 - *task-clock*: il tempo di CPU effettivamente consumato dal task.
- **Hardware events (PMU):** Sono eventi mappati su reali contatori hardware presenti nel processore. Hanno il vantaggio di essere portatili tra diverse architetture e comprendono:
 - *cycles*: cicli di clock della CPU.
 - *instructions*: numero di istruzioni eseguite.
 - *cache-references* / *cache-misses*: accessi e fallimenti nei vari livelli di cache.
 - *branch-misses*: errori di predizione dei salti.
- **Hardware cache events:** Forniscono un accesso strutturato e specifico agli eventi legati alle gerarchie di cache. Esempi tipici sono:
 - *L1-dcache-loads* / *L1-dcache-load-misses*: letture e fallimenti nella cache dati di primo livello.
 - *LLC-loads* / *LLC-load-misses*: accessi alla *Last Level Cache* (solitamente L3).

- **Tracepoint events:** Sono eventi implementati tramite l'infrastruttura *ftrace* del kernel. Permettono di tracciare in modo dettagliato chiamate di sistema (*syscall*), operazioni di input/output (I/O) ed eventi legati allo *scheduler*.

Per identificare gli eventi disponibili sul sistema, si utilizza il comando `perf list`, che può essere filtrato tramite `grep` per isolare categorie specifiche (es. `perf list | grep cache`).

1. Counting Mode: **perf stat**

Il comando `perf stat` esegue il programma in modalità *counting*, fornendo un riepilogo aggregato degli eventi al termine dell'esecuzione.

- **Utilizzo:** `perf stat ./programma`.
- **Metriche Chiave:** Una delle più importanti è l'IPC (Instructions Per Cycle).
 - **IPC tra 1 e 4:** Indica un buon utilizzo della pipeline (architettura superscalare).
 - **IPC > 1:** Segnala la presenza di stalli frequenti (es. attesa della memoria o errori di predizione).
- **Opzioni utili:**
 - `-e <event>`: Filtra la visualizzazione per mostrare solo eventi specifici.
 - `-r <n>`: Ripete la misura *n* volte per calcolare una media robusta.
 - `-p <pid>`: Analizza un processo già in esecuzione.
 - `-a -C <core_id>`: Monitora l'attività su core specifici.

2. Sampling Mode: **perf record e report**

A differenza del counting, la modalità *sampling* tramite `perf record` genera un campione statistico che permette di localizzare esattamente gli hotspot nel codice.

- **Simboli di Debug:** È fondamentale compilare il codice con il flag `-g` per includere i simboli di debug; senza di essi, `perf` non potrebbe correlare i campioni alle righe del codice sorgente C.

- **Workflow:**

1. `perf record ./programma`: Registra i campioni in un file (`perf.data`).
2. `perf report`: Visualizza un istogramma delle funzioni in cui il processo spende più tempo.
3. `perf annotate`: Permette di scendere nel dettaglio e vedere i campioni riga per riga (sia sorgente C che assembly).
4. `perf diff`: Consente di confrontare due diverse esecuzioni (es. prima e dopo un'ottimizzazione) per verificarne l'efficacia.

3. Call Graph Analysis

Mentre il "flat profiling" mostra solo quali funzioni sono lente, la Call Graph Analysis permette di risalire alla gerarchia delle chiamate per capire *chi* ha invocato la funzione critica.

- **Comando:** `perf record -g --call-graph dwarf ./prog` (l'opzione *dwarf* usa i simboli di debug per ricostruire lo stack).
- **Visualizzazione:** `perf report --call-graph --stdio`.
- **Utilità:** Identifica le cosiddette **funzioni "trampolino"** (o wrapper), ovvero funzioni che presentano un alto overhead *totale* (somma delle funzioni chiamate) ma un basso overhead *proprio* (lavoro svolto direttamente). Questo è essenziale per capire se il problema è la funzione stessa o il modo in cui viene invocata ripetutamente.

7.2.1.2 Altri strumenti di Profiling

Sebbene `perf` sia uno strumento eccellente per l'analisi a livello di sistema, esistono alternative che offrono vantaggi specifici come l'integrazione programmatica, la facilità d'uso per l'heap profiling o analisi specializzate per il calcolo parallelo.

PAPI (Performance Application Programming Interface)

PAPI è una libreria open source che offre un'interfaccia portabile per accedere agli hardware performance counter direttamente dall'interno del codice applicativo.

- **Vantaggi rispetto a `perf`:** Consente un "profiling chirurgico" (misurando solo specifiche sezioni di codice), garantisce portabilità su diverse architetture (x86, ARM, IBM POWER) ed è ideale per creare benchmark ripetibili.
- **Integrazione e Utilizzo:** Richiede l'inizializzazione degli eventi, seguita dall'inserimento delle funzioni `PAPI_start()` e `PAPI_stop()` attorno alla porzione di codice da analizzare. Per la compilazione, è necessario linkare la libreria aggiungendo il flag `-lpapi`.

gperftools (Google Performance Tools)

È una suite sviluppata da Google, considerata più semplice di `perf`, che include un profiler per la CPU (basato sul campionamento) e un heap profiler.

- **Modalità d'uso:**
 - *Senza modifiche al codice (Preload):* Utilizzando la variabile d'ambiente `LD_PRELOAD=/path/libprofiler.so` seguita dall'esecuzione del programma.
 - *Con integrazione nel codice:* Includendo la libreria (`#include <gperftools/profiler.h>`), avvolgendo il codice tra `ProfilerStart()` e `ProfilerStop()`, e compilando con il flag `-lprofiler`.
- **Analisi dei dati:** Si effettua tramite il tool `pprof`, che permette di generare output testuali, dettagli riga per riga, o visualizzazioni grafiche consultabili direttamente nel browser.

Confronto e Regola Pratica di Profiling

La scelta dello strumento dipende dal livello di dettaglio e dall'obiettivo dell'analisi:

- **`perf stat`:** Non richiede modifiche al codice e fornisce una misura globale degli eventi hardware, ideale per comprendere il comportamento complessivo (es. IPC, cache miss rate).
- **`perf record` + `perf report`:** Profiling per campionamento utile per identificare le catene di chiamate (call chains) e scoprire esattamente in quali funzioni l'applicazione spende più tempo (hotspot).
- **PAPI:** Ottimo per benchmark scientifici e misurazioni precise, sebbene richieda la modifica del codice sorgente.

- **gperftools**: Ottimo compromesso tra semplicità di integrazione e livello di dettaglio per identificare rapidamente i colli di bottiglia in applicazioni complesse.

Workflow Consigliato

Per un'analisi efficace delle prestazioni, la regola pratica suggerisce di procedere per gradi:

1. Iniziare sempre con **perf stat** per avere una visione d'insieme delle performance.
2. Utilizzare **perf record** per localizzare e isolare gli hotspot.
3. Impiegare **PAPI** o **gperftools** per effettuare misurazioni chirurgiche e dettagliate solo all'interno delle sezioni critiche individuate.

Capitolo 8

OpenMPI

8.1 Il Problema della Scalabilità

Il passaggio al modello a memoria distribuita è dettato da limiti fisici e architetturali.

- **Memoria Condivisa (es. OpenMP):** Scala efficacemente fino a qualche decina di core sullo stesso nodo. Quando il numero di core necessari supera quelli fisicamente condivisi o il problema diventa troppo grande, questo modello cessa di funzionare.
- **Memoria Distribuita (es. MPI):** Ogni processo possiede la propria RAM privata. La comunicazione non avviene tramite variabili condivise, ma attraverso lo scambio esplicito di messaggi (*msg*) tra processi indipendenti.

8.2 Cos'è MPI?

MPI (*Message Passing Interface*) è lo standard per la programmazione parallela basata sullo scambio esplicito di messaggi tra processi indipendenti. Non è un linguaggio, ma una specifica di interfaccia multi-piattaforma e vendor-neutral.

Le caratteristiche principali dello standard sono:

1. **Standard Aperto:** Gestito dall'MPI Forum, che riunisce vendor, centri di ricerca e università.

2. **Portabilità Totale:** Lo stesso codice sorgente è eseguibile su diverse architetture, dal singolo laptop ai cluster Linux, fino ai supercomputer più potenti.
3. **Evoluzione Continua:** Dalle prime specifiche MPI-1 (1994) alle più recenti MPI-4 (2021), che introducono modelli a sessioni e collettive persistenti.

Il Modello di Esecuzione: SPMD

MPI adotta il paradigma **SPMD** (*Single Program, Multiple Data*).

- **Binario Unico:** Tutti i processi eseguono lo stesso file eseguibile.
- **Rank:** La distinzione dei ruoli avviene a runtime tramite il *rank*, ovvero l'identificatore numerico unico di ogni processo.
- **Esempio:** In un blocco `if (rank == 0)`, il processo "root" può decidere di raccogliere i risultati, mentre gli altri processi ("worker") eseguono i calcoli e inviano i dati.

8.3 Struttura di un Programma e Comunicatori

L'esecuzione parallela avviene all'interno di un blocco delimitato da chiamate di inizializzazione e chiusura.

- `MPI_Init`: Inizializza il runtime; deve essere la prima chiamata MPI.
- `MPI_Comm_rank`: Restituisce il rank del processo corrente.
- `MPI_Comm_size`: Restituisce il numero totale di processi nel comunicatore.
- `MPI_Finalize`: Libera le risorse; nessuna chiamata MPI è permessa dopo di essa.

Il Contesto: I Comunicatori

Un comunicatore è un oggetto che racchiude un gruppo di processi e un contesto di comunicazione. Ogni messaggio viaggia all'interno di un comunicatore: due messaggi con lo stesso tag ma su comunicatori diversi non

si confondono mai. Questo è il meccanismo fondamentale con cui le librerie MPI proteggono i propri messaggi interni dall'interferenza con i messaggi dell'utente. Il comunicatore predefinito è `MPI_COMM_WORLD`, che include tutti i processi. È possibile creare sotto-comunicatori tramite `MPI_Comm_split`, partizionando i processi in base a un *colore* (gruppo di appartenenza) e una *chiave* (ordinamento interno).

8.4 Tipi di Dati e Messaggi

MPI definisce un insieme di tipi primitivi che corrispondono ai tipi del linguaggio host. Sono usati per specificare il tipo e la quantità dei dati in ogni operazione di comunicazione e per gestire la comunicazione tra architetture diverse.

- **Tipi Primitivi:** Corrispondono ai tipi C (es. `MPI_INT`, `MPI_DOUBLE`, `MPI_CHAR`).
- **Tipi Derivati:** Permettono di descrivere layout di memoria non contigui (strutture, colonne di matrici) senza buffer temporanei.

Tipo MPI	Tipo C	Dimensione tipica
MPI_INT	int	4 byte
MPI_LONG	long int	8 byte (64-bit)
MPI_FLOAT	float	4 byte
MPI_DOUBLE	double	8 byte
MPI_CHAR	char	1 byte
MPI_BYTE	—	1 byte (raw, no conversion)
MPI_LONG_LONG_INT	long long int	8 byte
MPI_UNSIGNED	unsigned int	4 byte

Nota: le dimensioni effettive dipendono dalla piattaforma. MPI_BYTE è utile per trasferire buffer grezzi senza conversione di tipo.

MPI_Type_contiguous

count copie contigue dello stesso tipo. Il caso più semplice: un array.

```
MPI_Type_contiguous(N, MPI_DOUBLE, &newtype);
```

MPI_Type_create_struct

Il più generale: specifica offset, tipo e count di ciascun campo. Usato per comunicare struct C arbitrarie.

```
MPI_Type_create_struct(count, array_of_blocklengths[], array_of_displacements[], array_of_types[], &newtype)
```

MPI_Type_vector

count blocchi di blocklength elementi, separati da stride (in numero di elementi). Perfetto per colonne di matrici row-major.

```
MPI_Type_vector(count, blocklength, stride, oldtype, &newtype)
```

MPI_Type_indexed

Lista arbitraria di blocchi con offset e count variabili. Utile per sparse patterns.

```
MPI_Type_indexed(count, blens[], displs[], oldtype, &newtype)
```

```

/* Esempio: comunicare la colonna 0 di una matrice N×M */
double A[N][M]; // matrice row-major in memoria

MPI_Datatype col_type;

/* 1. Definisci il tipo: N blocchi da 1 double, stride = M */
MPI_Type_vector(N, 1, M, MPI_DOUBLE, &col_type);

/* 2. OBBLIGATORIO: registra il tipo prima dell'uso */
MPI_Type_commit(&col_type);

/* 3. Usa il tipo come qualsiasi tipo MPI */
if (rank == 0)
    MPI_Send(&A[0][0], 1, col_type, 1, 0, MPI_COMM_WORLD);
else
    MPI_Recv(buf, N, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

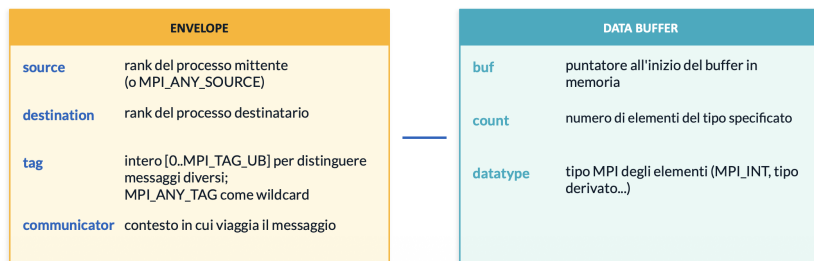
/* 4. Libera le risorse quando il tipo non serve più */
MPI_Type_free(&col_type);

```

Figura 8.1: Un tipo derivato deve essere registrato con `MPI_Type_commit` prima di poter essere usato nelle chiamate di comunicazione. Dopo l'uso va liberato con `MPI_Type_free` per evitare memory leak nell'implementazione MPI.

Anatomia di un Messaggio MPI

Un messaggio è composto da due parti logiche: l'**envelope** (intestazione per il matching, cioè identifica il messaggio nel sistema) e il **data buffer** (payload effettivo).



Il matching avviene sull'envelope: il runtime cerca un messaggio in arrivo con source, tag e communicator compatibili.

8.5 Modalità di Comunicazione Point-to-Point

Le funzioni fondamentali della comunicazione point-to-point sono `MPI_Send` e `MPI_Recv`.

```

/* firma completa */
int MPI_Send(
    void *buf,           // DATA BUFFER: puntatore ai dati
    int count,           // DATA BUFFER: n. elementi
    MPI_Datatype datatype, // DATA BUFFER: tipo MPI
    int dest,            // ENVELOPE: rank destinatario
    int tag,             // ENVELOPE: etichetta
    MPI_Comm comm);      // ENVELOPE: comunicatore

messaggio
MPI_Status status;

int MPI_Recv(
    void *buf,           // DATA BUFFER: dove scrivere
    int count,           // DATA BUFFER: capacità massima
    MPI_Datatype datatype, // DATA BUFFER: tipo MPI
    int source,          // ENVELOPE: rank mittente
    int tag,             // ENVELOPE: etichetta attesa
    MPI_Comm comm,       // ENVELOPE: comunicatore
    MPI_Status *status); // info sul msg ricevuto

/* esempio: P0 invia 10 double a P1 */
double buf[10] = { ... };
if (rank == 0)
    MPI_Send(buf, 10, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD);
else
    MPI_Recv(buf, 10, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

```

buf / count / datatype
 I tre componenti del data buffer. count è in numero di ELEMENTI, non in byte. Per 10 MPI_DOUBLE si trasferiscono 80 byte, ma si scrive count=10.

dest / source / tag / comm
 I componenti dell'envelope. source nella Recv può essere MPI_ANY_SOURCE. tag può essere MPI_ANY_TAG. comm deve essere esattamente uguale su entrambi i lati.

Bloccanti
 Send ritorna quando il buffer può essere riusato. Recv ritorna quando i dati sono completamente copiati in buf. Nel mezzo il thread è bloccato — vedremo le varianti non-bloccanti più avanti.

MPI_STATUS_IGNORE
 Passare MPI_STATUS_IGNORE se non vi servono informazioni sul messaggio ricevuto. Se usate MPI_ANY_SOURCE o MPI_ANY_TAG dovete passare &status per sapere da chi è arrivato e con quale tag.

Esistono diverse modalità di invio con semantiche differenti. In MPI, il termine *blocking* (bloccante) significa che la funzione non ritorna finché il buffer di memoria non può essere riutilizzato. Tuttavia, il modo in cui questo avviene dipende dalla modalità scelta.

Standard Mode (MPI_Send) È la modalità "non deterministica". Il sistema decide autonomamente se bufferizzare il messaggio o sincronizzarsi.

- Se bufferizzato: il mittente prosegue subito.
- Se non bufferizzato: il mittente attende che il destinatario sia pronto.

Nota: Non è sicuro assumere che il buffer sia libero immediatamente.

Buffered Mode (MPI_Bsend) Il completamento è sempre **locale**. Il programmatore deve allocare un buffer di memoria con `MPI_Buffer_attach`. Una volta che i dati sono stati copiati nel buffer di sistema, la funzione ritorna. È l'unica modalità che garantisce il distacco tra mittente e ricevitore.

Synchronous Mode (MPI_Ssend) Il completamento è **remoto**. La funzione termina solo quando il processo di ricezione ha iniziato a prelevare i dati. Questo garantisce un "rendez-vous" (sincronia) tra i due processi, fondamentale per il debugging dei flussi logici.

Ready Mode (MPI_Rsend) È una modalità a "zero overhead". Si può usare solo se si ha la certezza matematica che il destinatario abbia già postato la sua `MPI_Recv`. Se il destinatario non è pronto, l'invio fallisce silenziosamente o causa errori imprevedibili.

Il concetto di Sincronia in MPI

In un sistema parallelo, meno i processi devono aspettarsi l'un l'altro, più il sistema è efficiente. Tuttavia, la sincronia fornita da `MPI_Ssend` è spesso necessaria per evitare che il sistema esaurisca la memoria dei buffer temporanei in caso di messaggi troppo frequenti o troppo grandi.

8.5.1 Garanzie di Comunicazione: Ordering, Fairness e Progress

MPI fornisce garanzie precise sulla gestione dei messaggi, fondamentali per scrivere programmi corretti ed evitare comportamenti imprevedibili.

- **Ordering (Non-overtaking):** I messaggi non si sorpassano. Se il processo A invia due messaggi *M1* e *M2* allo stesso processo B, utilizzando lo stesso comunicatore e lo stesso tag, e B lancia due `MPI_Recv` compatibili, è garantito che *M1* verrà ricevuto prima di *M2*.
- **No Fairness:** MPI non garantisce equità. Se più operazioni di ricezione sono compatibili con un messaggio in arrivo, lo standard non specifica quale verrà soddisfatta per prima. Un processo potrebbe teoricamente monopolizzare i messaggi in modo indefinito (starvation).
- **Progress:** Un'implementazione conforme deve garantire che, se tutti i processi in un comunicatore eseguono operazioni bloccanti compatibili, almeno una di esse deve completare in un tempo finito.

8.5.2 Gestione della Ricezione: `MPI_Status` e `MPI_Get_count`

Ogni ricezione bloccante accetta come parametro una struttura `MPI_Status`. Questa è fondamentale quando si utilizzano wildcard come `MPI_ANY_SOURCE` o `MPI_ANY_TAG`, poiché permette di ispezionare le caratteristiche reali del messaggio appena arrivato.

```
MPI_Status status;
int count;
MPI_Recv(buf, MAXN, MPI_DOUBLE, MPI_ANY_SOURCE, MPI_ANY_TAG,
         MPI_COMM_WORLD, &status);

// Lettura dei campi della struttura:
status.MPI_SOURCE; // Restituisce il rank effettivo del mittente
```

```
status.MPI_TAG;    // Restituisce il tag effettivo del messaggio
status.MPI_ERROR;  // Codice di errore (MPI_SUCCESS se ok)
```

Attenzione: il numero effettivo di elementi ricevuti è un dato *opaco* all'interno della struttura. Per leggerlo è obbligatorio usare l'apposita funzione `MPI_Get_count(&status, MPI_DOUBLE, &count)`. Se queste informazioni non servono, si può passare la costante `MPI_STATUS_IGNORE` al posto dell'indirizzo di `status`.

8.5.3 Comunicazione Bloccante vs Non-Bloccante

Le operazioni MPI si dividono in bloccanti e non bloccanti. La distinzione non riguarda la correttezza del risultato, ma il momento in cui il thread di controllo viene restituito al programma chiamante.

Blocking (`MPI_Send` / `MPI_Recv`) Ritorna solo quando il buffer può essere riusato. Durante l'attesa per il trasferimento dati o la sincronizzazione, il thread è bloccato e la CPU viene sprecata senza fare calcolo utile.

Non-Blocking (`MPI_Isend` / `MPI_Irecv`) Ritorna immediatamente restituendo un *handle* di tipo `MPI_Request`. La comunicazione avviene in background, permettendo la **sovrapposizione tra calcolo e comunicazione** (*latency hiding*). Per completare l'operazione e assicurarsi che i dati siano pronti, bisogna esplicitamente chiamare `MPI_Wait` o `MPI_Test` sulla richiesta.

Note Cruciali sulle operazioni Non-Bloccanti

- **Dipendenza dei dati:** Quando si usa la comunicazione non bloccante e si esegue del calcolo in background, bisogna prestare estrema attenzione affinché tale calcolo *non dipenda in alcun modo* dal buffer coinvolto nella comunicazione. Toccare il buffer prima della `MPI_Wait` genera risultati inconsistenti.
- **Interoperabilità:** È perfettamente legale e possibile utilizzare insieme primitive bloccanti e non bloccanti. Ad esempio, una `MPI_Isend` da un processo può essere tranquillamente accoppiata con una `MPI_Recv` bloccante su un altro processo.

8.5.4 Deadlock e la Soluzione **MPI_Sendrecv**

L'errore più comune nella programmazione point-to-point si verifica quando due processi eseguono una `MPI_Send` simmetrica l'uno verso l'altro prima di chiamare le rispettive `MPI_Recv`. Se i messaggi superano l'*eager limit* (la dimensione massima per cui il sistema tenta di bufferizzare automaticamente), le `MPI_Send` si comportano in modo sincrono: entrambi i processi si bloccano in attesa della ricezione dell'altro, causando un **Deadlock**.

Soluzioni:

1. Alternare l'ordine asimmetricamente (es. i processi pari fanno prima `Send` e poi `Recv`, i dispari prima `Recv` e poi `Send`).
2. Usare chiamate non bloccanti (`MPI_Isend`).
3. Usare la funzione `MPI_Sendrecv`.

La primitive **MPI_Sendrecv**

`MPI_Sendrecv` esegue un'operazione di invio e una di ricezione in maniera **atomica**. Il runtime si occupa di gestire l'ordine interno per evitare qualsiasi rischio di deadlock.

- **Asimmetria di destinazione/sorgente:** La *source* da cui si riceve e la *destination* a cui si invia non devono essere per forza vincolanti o simmetriche. Un processo può ricevere dal rank *A* e contestualmente inviare al rank *B*.
- **Full-Duplex:** Se `dest = source`, di fatto si stanno inviando simultaneamente due messaggi incrociati (uno in entrata e uno in uscita) sullo stesso comunicatore con lo stesso partner di comunicazione.

8.5.5 Compilazione ed Esecuzione in Ambiente HPC

La compilazione dei programmi MPI utilizza script wrapper che aggiungono automaticamente gli header e i flag di linking corretti.

- **Compilazione:** `mpicc -O2 -o eseg eseguibile.c` (per il C)
o `mpicxx` (per il C++).
- **Esecuzione Locale:** `mpirun -np 4 ./eseg` avvia 4 istanze sul nodo corrente.

- **Esecuzione su Cluster:** Sistemi di produzione HPC moderni (es. SLURM) usano il comando `srun` (spesso inserito in uno script batch lanciato con `sbatch`) che si occupa dell’allocazione dinamica delle risorse hardware e del *binding* dei processi.

8.6 Misurazione delle Prestazioni: `MPI_Wtime`

MPI fornisce un timer ad alta risoluzione, sincronizzato con il runtime: `MPI_Wtime()`, che restituisce il tempo *wall-clock* trascorso in secondi (come `double`). `MPI_Wtick()` ne restituisce la risoluzione.

Best practice: Inserire sempre una `MPI_Barrier(MPI_COMM_WORLD)` prima di prendere il tempo di inizio (`t_start`). Senza la barriera, i processi non partono allineati e si rischia di misurare tempi sfasati, calcolando erroneamente le performance sul processo più veloce o più lento.

8.6.1 Modelli di Costo e Scalabilità (Legge di Amdahl e Modello $\alpha - \beta$)

8.6.1.1 L’impatto sulla Scalabilità

Nel contesto MPI, la Legge di Amdahl assume implicazioni pratiche: la “frazione seriale” non è composta solo dal codice non parallelizzabile, ma include **tutta la comunicazione, la latenza di rete, le barriere di sincronizzazione e gli sbilanciamenti di carico (load imbalance)**.

In ambito HPC, si fa spesso riferimento alla **Legge di Gustafson** ($S(p) = p - (1 - \alpha)(p - 1)$), basata sullo *scaled speedup*: crescendo la dimensione del problema al crescere dei processi, la frazione seriale si mantiene percentualmente costante.

8.6.2 Il Modello di costo $\alpha - \beta$

Per scrivere codice efficiente, il tempo di trasferimento di un messaggio di n byte è stimato con una funzione lineare:

$$T(n) = \alpha + \frac{n}{\beta}$$

- α (**Latenza o Startup cost**): Costo fisso per avviare la comunicazione (overhead software, protocollo di rete). Domina i messaggi piccoli.
- β (**Banda di picco**): Velocità di trasferimento in byte/sec. Diventa rilevante solo per messaggi grandi.

Le 4 Regole d'Oro derivate dal Modello $\alpha - \beta$

1. **Pochi messaggi grandi battono tanti messaggi piccoli:** A causa del termine fisso α , inviare 1000 messaggi da 1 KB è infinitamente più lento che inviare 1 messaggio da 1 MB.
2. **Aggregare le comunicazioni:** Se si devono inviare dati sparsi (es. N variabili scalari separate), bisogna "impacchettarle" (Pack/Unpack o tipi derivati) ed effettuare una singola Send.
3. **La latenza domina sui messaggi piccoli:** Sotto una certa soglia (es. 50 KB), il tempo di invio è dominato dalla latenza di startup (α).
4. **Sovrapporre calcolo e comunicazione:** Utilizzare operazioni non bloccanti (MPI_Isend) per nascondere il tempo di latenza (α) sotto calcoli utili della CPU. Questa è la tecnica più potente per scalare su migliaia di processi.

8.6.3 Il problema delle Deadlock

Supponiamo ci siano due processi che cercano di scambiarsi dei dati mediante l'uso di funzioni bloccanti come MPI_Send e MPI_Recv:

```
/* Esempio di potenziale Deadlock con messaggi grandi */
if (rank == 0) {
    MPI_Send(sbuf, N, MPI_DOUBLE, 1, 0, comm);
    MPI_Recv(rbuf, N, MPI_DOUBLE, 1, 0, comm, &st);
} else {
    MPI_Send(sbuf, N, MPI_DOUBLE, 0, 0, comm);
    MPI_Recv(rbuf, N, MPI_DOUBLE, 0, 0, comm, &st);
}
```

Se si utilizza la classica MPI_Send bloccante per scambiare messaggi di grandi dimensioni, si rischia di incappare in un *deadlock* (stallo). Poiché la funzione attende che il destinatario sia pronto a ricevere il messaggio prima di sbloccarsi, entrambi i processi finiscono per fermarsi all'infinito, aspettandosi a vicenda.

Questo problema non si presenta con i messaggi di piccole dimensioni ma si presenta dopo che il messaggio ha passato una certa dimensione limite detta *eager limit*. Implementazioni come OpenMPI, infatti, gestiscono queste

situazioni avvalendosi di buffer di sistema interni: se il messaggio è piccolo, viene copiato nel buffer e la `MPI_Send` ritorna immediatamente altrimenti si utilizza il buffer dedicato ma, come abbiamo detto, rischiamo stalli.

Per risolvere in modo sicuro lo scambio simmetrico di messaggi, si può utilizzare la funzione `MPI_Sendrecv`, che garantisce che l'invio e la ricezione avvengano in modo protetto. Indipendentemente dallo stato dei buffer di sistema o dalle dimensioni (anche gigantesche) dei messaggi, il runtime di MPI orchestra internamente i flussi di dati prevenendo qualsiasi rischio di deadlock. Di questa funzione ne esistono due varianti principali:

MPI_Sendrecv

```
int MPI_Sendrecv(void *sendbuf, int sendcount, MPI_Datatype sendtype, int dest, int sendtag,
                 void *recvbuf, int recvcount, MPI_Datatype recvtype, int source, int recvtag,
                 MPI_Comm comm, MPI_Status *status)
```

sendbuf/sendcount/sendtype Buffer, dimensione e tipo del messaggio da inviare.

dest / sendtag Destinatario e tag del messaggio inviato.

recvbuf/recvcount/recvtype Buffer, capacità massima e tipo del messaggio atteso.

source / recvtag Mittente atteso e tag atteso. Accettano `MPI_ANY_SOURCE` e `MPI_ANY_TAG`.

status Come nella Recv: contiene `MPI_SOURCE` e `MPI_TAG` del messaggio ricevuto.

MPI_Sendrecv_replace — stesso buffer per inviare e ricevere

```
int MPI_Sendrecv_replace(void *buf, int count, MPI_Datatype type, int dest, int sendtag, int source,
                        int recvtag, MPI_Comm comm, MPI_Status *status)
```

buf Unico buffer: prima contiene i dati da inviare, poi viene sovrascritto con i dati ricevuti.

Utile per shift circolari: `MPI_Sendrecv_replace(buf, N, MPI_DOUBLE, left, 0, right, 0, comm, IGNORE)`

Il runtime garantisce che la Send completi prima di sovrascrivere buf.

8.6.4 Comunicazioni non bloccanti

Il limite principale delle comunicazioni bloccanti è l'attesa inattiva: il processo si ferma in attesa che il messaggio arrivi o venga consegnato, causando rallentamenti e sprecando cicli di CPU.

Per ovviare a questo problema si introducono le comunicazioni non bloccanti, tramite le funzioni `MPI_Isend` e `MPI_Irecv` (dove la *I* sta per *Immediate*):

MPI_Isend / MPI_Irecv / MPI_Wait / MPI_Test

```
int MPI_Isend(void *buf, int count, MPI_Datatype type, int dest, int tag, MPI_Comm comm,
              MPI_Request *req);
int MPI_Irecv(void *buf, int count, MPI_Datatype type, int source, int tag, MPI_Comm comm,
              MPI_Request *req);
```

`int MPI_Wait (MPI_Request *req, MPI_Status *status);` // bloccante

`int MPI_Test (MPI_Request *req, int *flag, MPI_Status *status);` // non-bloccante

MPI_Request* OUTPUT: handle dell'operazione. Passatelo a `MPI_Wait` per aspettare il completamento.

MPI_Wait Blocca finché l'operazione completa. Dopo il ritorno: `request=NULL`, buffer libero.

MPI_Test Ritorna immediatamente: `flag=1` se completato, `flag=0` se ancora in corso. Non blocca.

Utilizzando queste funzioni, il processo avvia la comunicazione e riprende immediatamente l'esecuzione, permettendo di sovrapporre il trasferimento dei dati al tempo di calcolo (*computation-communication overlap*). Il processo si metterà in attesa esplicita solo nel momento in cui sarà strettamente necessario utilizzare i dati ricevuti (o riutilizzare il buffer di invio). Questo si ottiene tramite la funzione `MPI_Wait`, che forza il processo a fermarsi finché la specifica operazione non è conclusa.

È fondamentale, tuttavia, **non toccare per alcun motivo** i buffer di invio o ricezione nell'intervallo di tempo compreso tra l'avvio della chiamata asincrona e la relativa `MPI_Wait`, poiché i dati al loro interno si trovano in uno stato inconsistente.

Il pattern corretto e tipico da applicare è il seguente:

```
/* Pattern corretto: Irecv PRIMA di Isend */
MPI_Request sreq, rreq;

MPI_Irecv(rbuf, N, MPI_DOUBLE, src, 0, comm, &rreq); // 1. posta Recv prima
MPI_Isend(sbuf, N, MPI_DOUBLE, dst, 0, comm, &sreq); // 2. poi Send

do_computation(); // 3. calcola nel frattempo

MPI_Wait(&sreq, MPI_STATUS_IGNORE); // 4. aspetta Send
MPI_Wait(&rreq, MPI_STATUS_IGNORE); // 5. aspetta Recv
/* rbuf contiene ora i dati ricevuti */
```

Un'altra situazione comune è non conoscere a priori la lunghezza del messaggio in arrivo. Per gestire questa incertezza, esiste la funzione `MPI_Probe` (e la sua variante non bloccante `MPI_Iprobe`). Queste funzioni si limitano a ispezionare la coda dei messaggi in arrivo senza effettivamente estrarli, permettendoci di leggerne i metadati (come la dimensione) per allocare dinamicamente i buffer corretti prima di chiamare la *Receive*:

Firma `MPI_Probe` / `MPI_Iprobe`

```
/* Firma */
int MPI_Probe(
    int source,
    int tag,
    MPI_Comm comm,
    MPI_Status *status);

/* Versione non-bloccante */
int MPI_Iprobe(
    int source,
    int tag,
    MPI_Comm comm,
    int *flag,
    MPI_Status *status);

/* flag=1: messaggio disponibile
   flag=0: nessun messaggio */
```

Pattern d'uso: 4 passi

```
/* Pattern: size sconosciuta */
MPI_Status st;
int count;

/* 1. Aspettare che arrivi */
MPI_Probe(MPI_ANY_SOURCE, 0, comm, &st);

/* 2. Scoprire quanti elementi */
MPI_Get_count(&st, MPI_DOUBLE, &count);

/* 3. Allocare il buffer giusto */
double *buf = malloc(count*8);

/* 4. Ricevere - usare st.MPI_SOURCE
   non MPI_ANY_SOURCE! */
MPI_Recv(buf, count, MPI_DOUBLE,
         st.MPI_SOURCE, 0, comm, IGNORE);
```

Infine, come accennato, le comunicazioni non bloccanti necessitano sempre di primitive di sincronizzazione (*wait*) per garantire il corretto comple-

tamento delle operazioni ed evitare perdite di memoria. Oltre alla prima `MPI.Wait` analizzata, esistono altre comode varianti per gestire array di richieste:

MPI.Waitall — tutti devono completare	MPI.Waitany — il primo che finisce
<pre> /* MPI.Waitall — aspetta TUTTE */ int MPI_Waitall(int count, MPI_Request reqs[], MPI_Status stats[]); /* Scatter non-bloccante */ MPI_Request reqs[size-1]; for (int p=1; p<size; p++) MPI_Isend(&array[p*chunk], chunk, MPI_DOUBLE, p, 0, comm, &reqs[p-1]); do_my_chunk(); /* calcola chunk 0 mentre invii gli altri */ MPI_Waitall(size-1, reqs, MPI_STATUSES_IGNORE); </pre>	<pre> /* MPI.Waitany — aspetta LA PRIMA */ int MPI_Waitany(int count, MPI_Request reqs[], int *index, MPI_Status *status); /* Master raccoglie in ordine arrivo */ MPI_Request reqs[nworkers]; for (int w=0; w<nworkers; w++) MPI_Irecv(&results[w], 1, MPI_DOUBLE, w+1, 0, comm, &reqs[w]); for (int i=0; i<nworkers; i++) { int idx; MPI_Waitany(nworkers, reqs, &idx, MPI_STATUS_IGNORE); process(results[idx]); } </pre>

8.6.5 Linee guida per la scelta: Bloccanti vs. Non-Bloccanti

Le funzioni non bloccanti non rappresentano sempre la scelta migliore, poiché introducono una maggiore complessità nel codice e rendono il debugging nettamente più ostico. La scelta della strategia comunicativa deve quindi basarsi su criteri pratici:

- **Scegliere le funzioni BLOCCANTI quando:**
 - Non ci sono computazioni utili da poter eseguire nell'intervallo tra l'invio (`MPI.Send`) e la ricezione (`MPI.Recv`), rendendo l'asincronia di fatto superflua.
 - I messaggi scambiati sono di piccole dimensioni (indicativamente < 64 KB, al di sotto dell'*eager limit*), in quanto `MPI.Send` sfrutta la bufferizzazione interna e ritorna in modo quasi istantaneo.
 - Si è in fase di sviluppo o debugging, poiché la semantica lineare semplifica drasticamente la riproducibilità degli errori e l'individuazione dei bug.
- **Scegliere le funzioni NON-BLOCCANTI quando:**
 - Si hanno calcoli indipendenti da eseguire contemporaneamente al tempo di comunicazione, così da nascondere i tempi morti della rete (*latency hiding*), come nel caso degli algoritmi iterativi con *halo exchange* o nelle computazioni *stencil*.

- Un processo *Master* deve distribuire dati a molti nodi *Worker*: avviare un ciclo veloce di `MPI_Isend` seguito da una singola `MPI_Waitall` risulta molto più efficiente e scalabile rispetto a una sequenza bloccante di `MPI_Send`.

Regola d'oro: È sempre buona norma iniziare lo sviluppo implementando una logica lineare e robusta con le funzioni **bloccanti**. Solo successivamente si esegue la profilazione del codice (*profiling*), passando alle varianti **non bloccanti** esclusivamente nei colli di bottiglia dove l'attesa di comunicazione domina nettamente sul tempo di calcolo.

8.7 Comunicazioni Collettive in MPI

Le operazioni collettive si differenziano dalle comunicazioni point-to-point (che sono selettive e coinvolgono solo un mittente e un destinatario). Le comunicazioni collettive coinvolgono obbligatoriamente **TUTTI** i processi appartenenti a un determinato *communicator*.

Una caratteristica cruciale di queste operazioni è che sono **sincronizzanti**: tutti i processi devono raggiungere il punto della chiamata nel codice prima che l'esecuzione possa procedere regolarmente.

NOTA - ERRORE COMUNE: `MPI_Bcast` dentro un `if`. Se rank 0 chiama `Bcast` e rank 1 non lo chiama, il programma si blocca indefinitamente.

8.7.1 Operazioni di Distribuzione e Raccolta

Broadcast (`MPI_Bcast`) Permette al processo *root* di distribuire (copiare) un dato o un array a tutti gli altri processi del comunicatore. L'operazione è altamente ottimizzata e avviene in tempo $O(\log P)$ tramite un algoritmo ad albero binario. Tutti i processi devono invocare la funzione passando gli stessi identici parametri.

MPI_Bcast

```
int MPI_Bcast(void *buf, int count, MPI_Datatype type, int root, MPI_Comm comm)
buf Sul root: dato da distribuire. Sugli altri: buffer dove ricevere.
count Numero di elementi.
type Tipo MPI del dato.
root Rank del processo che ha il dato originale.
Tutti i processi chiamano MPI_Bcast con gli stessi parametri.
```

```
int N = 0;

/* Solo il root legge il parametro */
if (rank == 0) N = atoi(argv[1]);

/* Tutti chiamano Bcast - root distribuisce, gli altri ricevono */
MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);

/* Ora tutti i processi hanno N - possono allocare i propri array */
double *array = malloc(N * sizeof(double));

/* Bcast funziona anche su array */
/* MPI_Bcast(array, N, MPI_DOUBLE, 0, comm) */
```

Scatter (MPI_Scatter) A differenza del broadcast che invia a tutti la stessa copia del dato, lo scatter divide un array residente sul *root* in parti (chunk) di dimensioni uguali e distribuisce un frammento diverso a ciascun processo.

Gather (MPI_Gather) È l'operazione speculare allo scatter. Raccoglie i chunk di dati dai vari processi e li concatena in un unico array sul processo *root*, rispettando l'ordine dei rank.

Nota architetturale: La sequenza Scatter → calcolo locale → Gather costituisce il **pattern parallelo fondamentale**: distribuire il problema, elaborare i dati in parallelo e raccogliere i risultati finali.

8.7.2 Operazioni di Riduzione (MPI_Reduce e MPI_Allreduce)

Le operazioni di riduzione servono a combinare i dati distribuiti su più processi utilizzando un operatore matematico o logico predefinito (es. MPI_SUM, MPI_MAX, MPI_MIN, MPI_PROD). Tali operatori sono commutativi e associativi.

Reduce standard (MPI_Reduce) Applica l'operatore ai valori forniti da tutti i processi, ma deposita il risultato finale **esclusivamente** nel buffer di ricezione (*recvbuf*) del processo *root*. Sui processi non-*root*, il buffer di ricezione viene ignorato.

MPI_Reduce

```
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,
               MPI_Datatype type, MPI_Op op, int root, MPI_Comm comm)
```

sendbuf Contributo locale — il valore di questo processo.
recvbuf Risultato finale — solo sul root. Ignorato sugli altri.
op Operatore: MPI_SUM, MPI_MAX, MPI_MIN, MPI_PROD, MPI_LAND, MPI_LOR, MPI_BAND..
 Reduce è commutativo e associativo — l'ordine in cui i valori vengono combinati non è garantito dallo standard.

```
/* Ogni processo calcola la somma del proprio chunk */
double local_sum = 0.0;
for (int i = 0; i < chunk; i++) local_sum += local[i];

/* MPI_Reduce somma tutte le local_sum sul root */
double total_sum = 0.0;
MPI_Reduce(&local_sum, &total_sum, 1, MPI_DOUBLE,
           MPI_SUM, 0, MPI_COMM_WORLD);

/* Solo rank 0 ha total_sum — gli altri hanno 0 (recvbuf non scritto) */
if (rank == 0)
    printf("Somma totale: %.0f\n", total_sum);
```

Allreduce (MPI_Allreduce) Esegue la medesima operazione logica della Reduce, ma il risultato finale viene distribuito a **TUTTI** i processi del comunicatore. Di conseguenza, la firma della funzione non prevede il parametro `root`. È fondamentale utilizzarla in scenari in cui tutti i processi devono prendere una decisione basata su un dato globale, come nel caso di controlli di convergenza o normalizzazioni.

Regole Pratiche ed Errori Comuni (Deadlock e Inefficienze) Le collettive hanno una semantica molto rigida. Ignorare queste regole porta a blocchi o risultati errati:

- **Collettiva dentro un costrutto condizionale (if):** È l'errore più comune. Inserire una chiamata come `MPI_Bcast` o `MPI_Reduce` all'interno di un `if (rank == 0)` fa sì che gli altri processi non partecipino mai all'operazione, causando un **deadlock** (blocco indefinito) dell'intero programma.
- **Leggere il buffer di ricezione su processi non-root:** In una `MPI_Reduce`, cercare di leggere il `recvbuf` su processi con rank diverso dal root restituirà solo "spazzatura" (valori di memoria indefiniti).
- **Simulare Allreduce con Reduce + Bcast:** Scrivere codice che esegue una `MPI_Reduce` sul root seguita da una `MPI_Bcast` per diffondere il risultato è una pratica estremamente **inefficiente**. Lo standard raccomanda di utilizzare direttamente `MPI_Allreduce`, che è ottimizzata internamente per questo scopo.

8.8 Comunicatori

La slide definisce il comunicatore come un gruppo di processi con un contesto di comunicazione isolato. In genere, all'avvio di un programma MPI esiste un unico comunicatore chiamato `MPI_COMM_WORLD` e questo include tutti i processi che sono stati lanciati. Facendo una `Send` o una `Recv` usando `Comm world` stiamo mandando o ricevendo il messaggio a tutti i possibili processi.

Tuttavia, può capitare di volere un comunicatore personalizzato che appartenga a specifici processi e che sia in un certo senso privato solo per loro. Ci sono varie ragioni per fare una scelta di questo tipo:

1. **Isolare sottogruppi di processi:** Immaginiamo di avere una griglia di processi (es. una matrice) e di voler calcolare la somma degli elementi solo riga per riga o colonna per colonna. Creando un comunicatore per ogni riga, possiamo dire a MPI di fare una riduzione (`MPI_Reduce`) solo tra i processi di quella specifica riga, ignorando gli altri.
2. **Isolamento del contesto:** Se una libreria esterna che usiamo nel programma scambia messaggi e anche noi scambiamo messaggi nel codice principale, c'è il rischio che i messaggi si sovrappongano o vengano confusi (anche usando i tag). Un comunicatore custom crea un "canale privato" o namespace separato e i messaggi inviati su `comm_custom` non possono essere intercettati o confusi con quelli inviati su `MPI_COMM_WORLD`.
3. **Leggibilità e struttura:** Rende il codice parallelo modulare e molto più facile da gestire.

È bene ricordare che ogni comunicatore ha un suo **spazio dei rank** e che questo non coincide con quello di `MPI_COMM_WORLD`. Infatti, per trovare il rank nello specifico comunicare, si usa la funzione `MPI_COMM_RANK`. MPI gestisce in automatico tre tipi di comunicatori:

1. **`MPI_COMM_WORLD` (Il mondo globale)**
 - **Cos'è:** È il comunicatore principale che racchiude *tutti* i P processi che sono stati inizializzati al momento del lancio del programma tramite il comando `mpirun` o `mpiexec`.
 - **Caratteristiche:** È sempre disponibile dopo la chiamata a `MPI_Init` e rappresenta il punto di partenza universale. Se interroghiamo la sua dimensione tramite `MPI_Comm_size (MPI_COMM_WORLD, &size)`, la variabile `size` conterrà il numero totale di processi allocati per l'intera esecuzione.

2. **MPI_COMM_SELF** (Il comunicatore specchio)

- **Cos'è:** È un comunicatore rigidamente limitato al *singolo processo corrente*. Ogni processo ha il proprio `MPI_COMM_SELF` indipendente dagli altri.
- **Proprietà fisse:** Per qualsiasi processo che lo interroghi, la `size` (dimensione) del comunicatore sarà **sempre 1** e il `rank` del processo al suo interno sarà **sempre 0**.
- **Casi d'uso:** Viene utilizzato per effettuare operazioni MPI puramente locali (che non devono coinvolgere la rete o altri nodi), oppure come tassello base quando si vogliono costruire comunicatori personalizzati partendo da zero in modo programmatico.

3. **MPI_COMM_NULL** (Il comunicatore nullo)

- **Cos'è:** Rappresenta un oggetto comunicatore non valido o inesistente. Equivalente al concetto di puntatore `NULL` in C.
- **Comportamento con `MPI_Comm_split`:** Quando si usa la funzione `MPI_Comm_split` per dividere un comunicatore in sottogruppi, un processo può auto-escludersi passando come parametro di colore il valore speciale `MPI_UNDEFINED`. Di conseguenza, la funzione restituirà nel parametro di output (`&sub`) il valore `MPI_COMM_NULL`.
- **Importanza algoritmica:** Non è possibile inviare o ricevere messaggi su `MPI_COMM_NULL` (genererebbe un errore a runtime). Serve invece come **sentinella** logica: effettuando un controllo condizionale come `if (sub != MPI_COMM_NULL)`, il processo può verificare matematicamente se fa parte o meno del nuovo sottogruppo di calcolo prima di eseguire funzioni collettive (come la `MPI_Bcast` mostrata nell'esempio).

Per fare questa divisione utilizziamo la funzione `MPI_COMM_SPLIT` che divide i processi di un comunicatore in sottogruppi in base a un parametro `color`. Processi con lo stesso `color` finiscono nello stesso nuovo comunicatore.

MPI_Comm_split

```
int MPI_Comm_split(MPI_Comm comm, int color, int key, MPI_Comm *newcomm)
```

color Processi con lo stesso `color` formano un sottogruppo. Usare `MPI_UNDEFINED` per escludere un processo.

key Determina l'ordine dei rank nel nuovo comunicatore. Tipicamente si usa `rank` o `colonna`.

Tutti i processi del comunicatore originale devono chiamare `Comm_split`. Liberare sempre con `MPI_Comm_free` quando non serve più.

È importante ricordare che tutti i processi del comunicatore originale devono chiamare la `split` anche se questi vogliono escludersi. Inoltre è sempre

necessario liberare manualmente la memoria del comunicatore mediante la `MPI_COMM_FREE`. Immaginiamo il seguente codice:

```
/* Griglia 2x4: dividere per RIGA */
int row = rank / 4;
int col = rank % 4;

MPI_Comm row_comm;
MPI_Comm_split(MPI_COMM_WORLD, row, col, &row_comm);
/* color=row → processi della stessa riga → stesso comm */
/* key=col → rank nel nuovo comm = posizione nella riga */

/* Operazioni collettive indipendenti per riga e colonna */
MPI_Reduce(&val, &row_sum, 1, MPI_DOUBLE, MPI_SUM, 0, row_comm);

MPI_Comm_free(&row_comm);
```

Su 8 processi i valori saranno:

- **Color:** Questo valore sarà 0 per tutti i processi da 0 a 3 mentre sarà 1 per tutti i processi da 4 a 7.
- **Key:** Questo andrà a numerare i processi con lo stesso color da 0 a 3.

8.8.1 Topologia Cartesiana

Fino ad ora, per MPI i processi erano semplicemente una lista lineare da 0 a $n - 1$. Con la topologia cartesiana, invece, diciamo a MPI di organizzare i processi come una griglia geometrica (a 2D, 3D o più dimensioni). Questo è utilissimo perché molti problemi reali (es. simulazioni di fluidi, elaborazione immagini, calcolo di matrici) sono strutturati geometricamente. Per farlo, usiamo la chiamata `MPI_Cart_create` che ha la seguente firma:

MPI_Cart_create / MPI_Cart_coords / MPI_Cart_rank

```
int MPI_Cart_create(MPI_Comm comm, int ndims, int dims[], int periods[], int reorder, MPI_Comm *comm_cart);
dims[] Dimensioni della griglia: dims[0]=righe, dims[1]=colonne.
periods[] Periodicità: 1=wrap-around sui bordi, 0=bordi aperti (MPI_PROC_NULL).
reorder 1=MPI può riassegnare i rank per efficienza. 0=ordine invariato.
int MPI_Cart_coords(MPI_Comm comm, int rank, int ndims, int coords[]);
int MPI_Cart_rank (MPI_Comm comm, int coords[], int *rank);
Cart_coords Dato un rank, restituisce le coordinate nella griglia. Cart_rank Date le coordinate, restituisce il rank. Inverso di Cart_coords.
```

In pratica abbiamo i seguenti parametri:

- **comm (MPI_Comm):** Il comunicatore di partenza (di solito `MPI_COMM_WORLD`) che contiene l'insieme originale di processi da cui attingere per creare la griglia.
- **ndims (int):** Il numero di dimensioni della struttura cartesiana che si desidera creare (ad esempio, 1 per una linea/anello, 2 per una matrice/griglia bidimensionale, 3 per un cubo).

- **dims[] (int *)**: Un array di interi di dimensione pari a `ndims`. Ogni elemento specifica il numero di processi da allocare lungo quella specifica dimensione.
 - *Esempio*: Se `ndims = 2`, definire `dims[2] = {2, 3}` significa richiedere una griglia di 2 righe e 3 colonne (6 processi totali).
- **periods[] (int *)**: Un array di valori logici (booleani espressi come 0 o 1) di dimensione pari a `ndims`. Specifica se la griglia è periodica (connessa ad anello/toro) oppure se possiede dei bordi aperti lungo ciascuna dimensione.
 - 1 (True): Abilita il *wrap-around* (il vicino successivo all'ultimo elemento torna all'elemento zero).
 - 0 (False): I bordi sono lineari e interrotti (i processi di confine non hanno vicini oltre il limite).
- **reorder (int)**: Un flag booleano (0 o 1) che indica a MPI se può ottimizzare l'assegnazione dei rank hardware.
 - 1 (True): MPI è autorizzato a riordinare i processi e cambiare i loro rank fisici originari per mapparli in modo ottimale sulla reale topologia dei nodi e degli switch del cluster (riducendo la latenza di rete).
 - 0 (False): L'ordine relativo dei rank dei processi deve rimanere identico a quello del comunicatore di partenza.
- ***comm_cart (MPI_Comm *)**: Il puntatore di output in cui MPI salverà l'handle del nuovo comunicatore cartesiano appena creato, dotato delle proprietà geometriche configurate.

Inoltre, abbiamo anche altre due funzioni:

- **MPI_Cart_coords**: Quando la griglia è creata, ogni processo mantiene il suo rank numerico tradizionale, ma possiede anche delle coordinate spaziali (x, y) che possono essere ottenute tramite questa funzione.
- **MPI_Cart_rank**: È l'operazione inversa a `coords`, cioè permette di passare dalle coordinate spaziali al rank banalmente.

Proprio grazie alla struttura geometrica del problema, possiamo lavorare individuando velocemente i vicini tramite `MPI_Cart_shift`: Per farlo, usiamo la chiamata `MPI_Cart_create` che ha la seguente firma:

MPI_Cart_shift

```
int MPI_Cart_shift(MPI_Comm comm, int direction, int disp, int *rank_source, int *rank_dest)
direction Dimensione: 0=righe (su/giù), 1=colonne (sinistra/destra).
disp +1 = avanti, -1 = indietro nella direzione scelta.
rank_source Vicino da cui ricevereste (indietro rispetto a disp).
rank_dest Vicino a cui inviereste (avanti rispetto a disp).
```

Vediamone un esempio per capire meglio. Vediamone un esempio per capire meglio. Supponiamo di avere `MPI_Cart_shift(cart, 0, +1, &up, &down)`. Se la direzione è 0 (quindi verticale) e lo spostamento è +1 (verso il basso):

- Il vicino che sta sotto di te è la tua destinazione logica (down), perché se ti muovi in avanti incontri lui.
- Il vicino che sta sopra di te è la tua sorgente logica (up), perché se lui si muovesse in avanti di +1 colpirebbe te.

Cosa succede ai bordi? Se la griglia non è periodica (`periods = 0`), i processi che si trovano sul bordo non hanno un vicino esterno. In quel caso, MPI assegnerà a quella variabile il valore speciale `MPI_PROC_NULL`. MPI sa gestire questa costante: se passi `MPI_PROC_NULL` a una funzione di invio/ricezione, la comunicazione viene semplicemente ignorata senza mandare in crash il programma.

Infine, un'ultima funzione molto utile è `MPI_Dims_create`. Supponiamo di aver dichiarato a mano come prima la dimensione del numero di processi come, ad esempio supponiamo `int dims[2] = {2, 3}` ma che, anziché avere 6 processi, ne abbiamo 4 o magari 8. Questo porterebbe ad un crash a runtime poiché le dimensioni sono *hardcoded*.

La funzione `MPI_Dims_create` risolve proprio questo problema calcolando in automatico le dimensioni ottimali per la griglia in base al numero di processi disponibili a runtime:

MPI_Dims_create

```
int MPI_Dims_create(int nnodes, int ndims, int dims[])
nnodes Numero totale di processi da distribuire nella griglia.
ndims Numero di dimensioni.
dims[] In ingresso: 0 = lascia decidere a MPI, != 0 = fisso. In uscita: dimensioni calcolate.
```

Un esempio è il seguente:

```

/* Senza Dims_create: hardcoded - non funziona con P diverso */
int dims[2] = {2, 3}; /* solo per 6 processi */

/* Con Dims_create: automatico per qualsiasi P */
int dims[2] = {0, 0}; /* 0 = lascia decidere a MPI */
MPI_Dims_create(size, 2, dims);
/* Con 6 proc: dims = {2, 3} */
/* Con 12 proc: dims = {3, 4} */
/* Con 16 proc: dims = {4, 4} */

/* Fissare una dimensione, lasciare libera l'altra */
int dims[2] = {2, 0}; /* forza 2 righe, MPI calcola le colonne */
MPI_Dims_create(size, 2, dims);
/* Con 12 proc e dims[0]=2: dims = {2, 6} */

```

Quest'ultimo scenario mette a confronto le tre metodologie di gestione dei processi viste finora, definendo i criteri di scelta ottimale in base alla struttura del problema da parallelizzare.

1. Quando utilizzare **MPI_Comm_split**

- **Partizioni logiche indipendenti:** Si sceglie quando l'algoritmo prevede la scomposizione del dominio in macro-blocchi o sotto-problemi isolati che non necessitano di comunicare tra loro (ad esempio, operare in parallelo sulle singole righe o colonne di una matrice, oppure separare le fasi di una pipeline di calcolo).
- **Operazioni collettive parziali:** È indispensabile quando si ha la necessità di invocare funzioni collettive (come `MPI_Reduce`, `MPI_Bcast`, `MPI_Scatter`) limitando il loro raggio d'azione esclusivamente a un sottoinsieme specifico di processi, evitando di coinvolgere l'intera rete.

2. Quando utilizzare la Topologia Cartesiana

- **Struttura spaziale N -dimensionale:** Rappresenta la scelta ottimale se il problema ha una forte connotazione geometrica e i processi devono scambiare dati prevalentemente con i propri vicini di casa immediati (es. risoluzione di equazioni a derivate parziali via differenze finite, simulazioni di fluidodinamica, automi cellulari o elaborazione di immagini distribuiti).
- **Gestione semplificata dei confini:** Consente di scrivere codice lineare ed elegante eliminando una fitta giungla di costrutti condizionali (`if`). Grazie all'uso della costante `MPI_PROC_NULL`, MPI ignora in modo trasparente e sicuro i messaggi inviati oltre i bordi di una griglia non periodica.

3. Quando NON utilizzare strutture personalizzate

- **Assenza di relazioni geometriche o di gruppo:** Se i processi comunicano in modo asimmetrico o se la topologia dei dati non segue uno schema regolare, è preferibile rimanere all'interno di `MPI_COMM_WORLD` utilizzando comunicazioni punto-a-punto (`MPI_Send` e `MPI_Recv`) dirette.
- **Problemi a bassa complessità o con pochi processi:** La creazione di nuovi comunicatori e topologie introduce un costo computazionale aggiuntivo in termini di tempo e memoria (*overhead*). Se la comunicazione è banale o i dati scambiati sono minimi, il sovraccarico introdotto dalle funzioni di setup non è giustificato da un reale guadagno in termini di performance o leggibilità.